

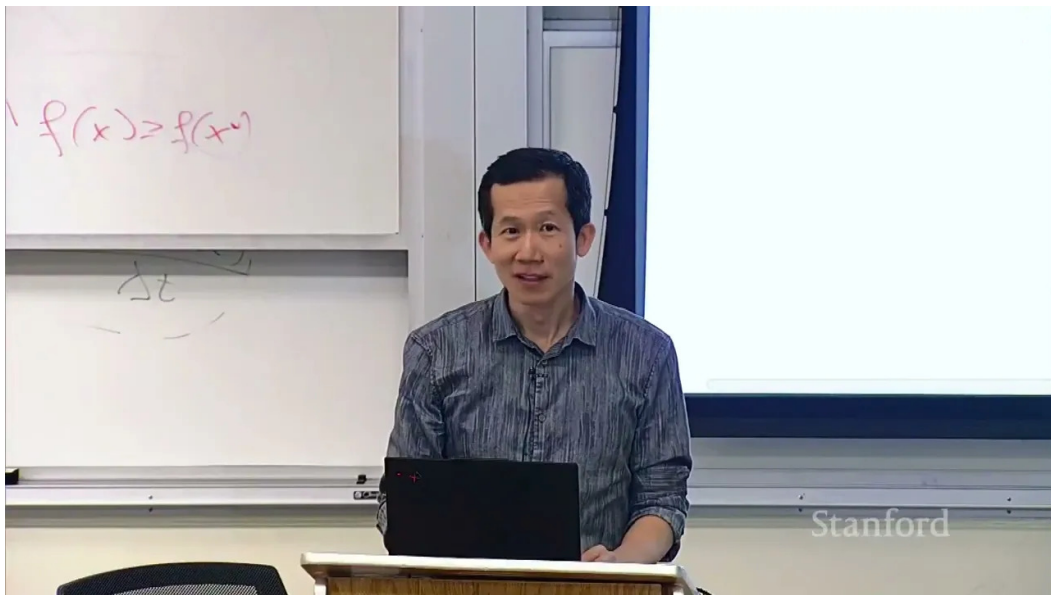
课程笔记

从零构建语言模型：课程全景与分词

Stanford CS336 Spring 2026 · Lecture 1 高密度中文讲义

Percy Liang (讲授) / lecture-to-notes (整理)

2026 年 7 月 11 日



视频作者 / 频道: Stanford Online

发布日期: 2026-04-14

视频时长: 01:19:21

视频链接: <https://www.youtube.com/watch?v=JuoVZkPBkKk>

课程主页: <https://cs336.stanford.edu/>

官方可执行课件: `lecture_01.py`

目录

1 为什么还要“从零构建”语言模型	3
1.1 抽象提升生产率，也可能切断因果理解	3
1.2 工业化之后，公开课堂能学什么	4
1.3 本章小结	6
2 效率主线与“苦涩教训”	6
2.1 同一套效率问题贯穿全栈	7
2.2 本章小结	7
3 语言模型发展史：接口在变，基本机制仍在	7
3.1 从统计模型到神经基础模型	7
3.2 开放生态为何重要	8
3.3 语言模型接口的四次变化	9
3.4 本章小结	10
4 课程地图一：从 Tokenizer 到训练循环	10
4.1 Tokenization：模型究竟在哪些“原子”上计算	10
4.2 模型架构：从原始 Transformer 到现代组件	11
4.3 训练配方：参数如何被可靠地设置	12
4.4 本章小结	12
5 课程地图二：系统、并行与推理	12
5.1 资源核算与 Roofline 视角	13
5.2 Kernel：把数学表达式组织成硬件友好程序	13
5.3 并行：通信是跨 GPU 的数据搬运	14
5.4 推理：Prefill 与 Decode 是两种工作负载	15
5.5 本章小结	15
6 课程地图三：缩放定律与可预测训练	15
6.1 用小实验优化大训练配方	16
6.2 等算力曲线：更大模型还是更多 token	16
6.3 本章小结	17
7 课程地图四：评测、数据与对齐	17
7.1 评测服务于两类不同目的	17

7.2	数据不是“从天上掉下来”的	17
7.3	数据处理是一条有损决策链	18
7.4	对齐：当评价比生成更容易	19
7.5	一条贯穿五个作业的主线	19
7.6	本章小结	20
8	Tokenization 基础：可逆表示与计算预算	20
8.1	现实 Tokenizer 的几个反直觉现象	21
8.2	压缩率与词表大小	21
8.3	本章小结	22
9	三种直接方案为什么都不理想	22
9.1	字符级：语义直观，但词表大且长尾严重	22
9.2	字节级：词表固定，却把序列拉得过长	23
9.3	词级：压缩好，却无法封闭词表	24
9.4	本章小结	24
10	Byte Pair Encoding：从字节学习可变粒度词表	24
10.1	基本直觉	24
10.2	最小实现与复杂度	26
10.3	用合并表编码新文本	26
10.4	Tokenizer-free 的长期方向	27
10.5	本章小结	28
11	总结与延伸	28
11.1	讲者总结	28
11.2	结构化提炼	28
11.3	建议练习	29
11.4	拓展阅读	29
11.5	本章小结	29

1 为什么还要“从零构建”语言模型

CS336 的出发点不是否定 API、预训练权重或高层框架，而是指出：语言模型的抽象层仍然会**泄漏**。当研究者只会调用闭源模型时，一旦遇到上下文、推理效率、数据、稳定性或训练目标层面的限制，就很难判断失败来自哪里，更无法改造底层机制。课程因此采用“通过构建获得理解”的路线：亲手完成 tokenizer、Transformer、优化器、训练循环、GPU kernel、分布式训练、数据处理与对齐。

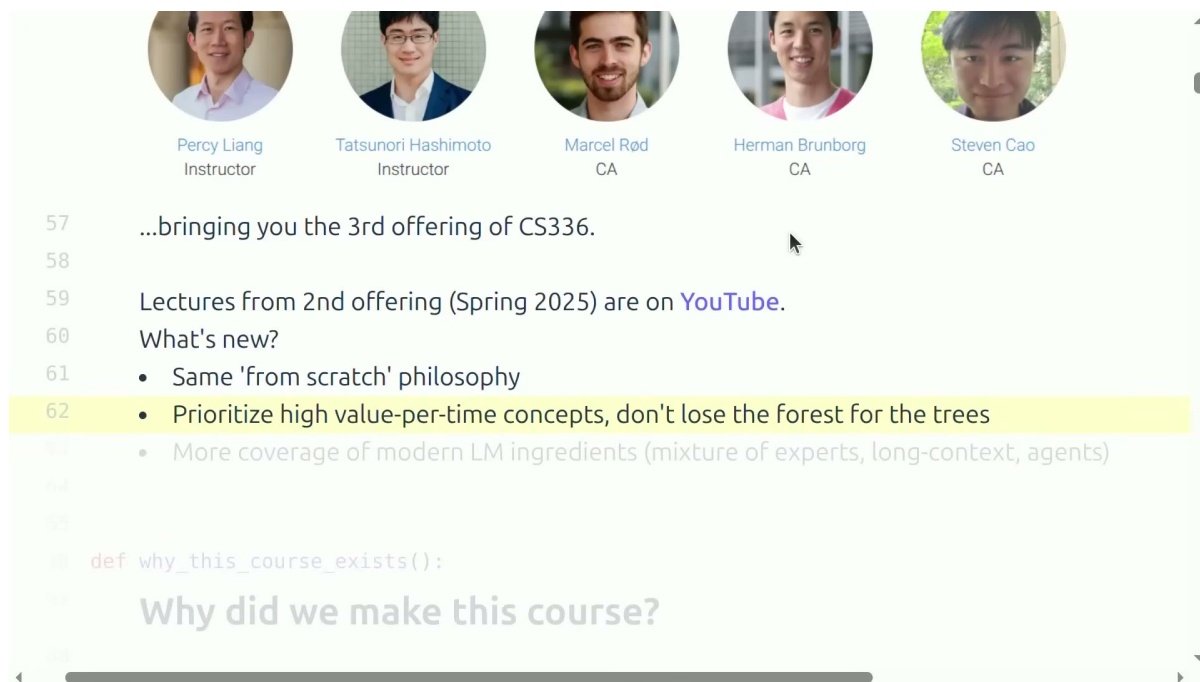


图 1：2026 年第三次开课：延续从零构建理念，同时强化现代架构、长上下文与智能体内容。¹

1.1 抽象提升生产率，也可能切断因果理解

讲者用研究工作流的变化说明断层：早期研究者训练自己的模型，随后转向下载并微调 BERT，再后来主要通过提示 GPT、Claude 或 Gemini 完成任务。抽象层越高，试验速度越快；但语言模型不像编程语言或操作系统那样形成稳定边界，模型能力、推理接口、数据和系统实现仍在共同变化。

¹视频画面时间区间：00:02:45–00:03:00。

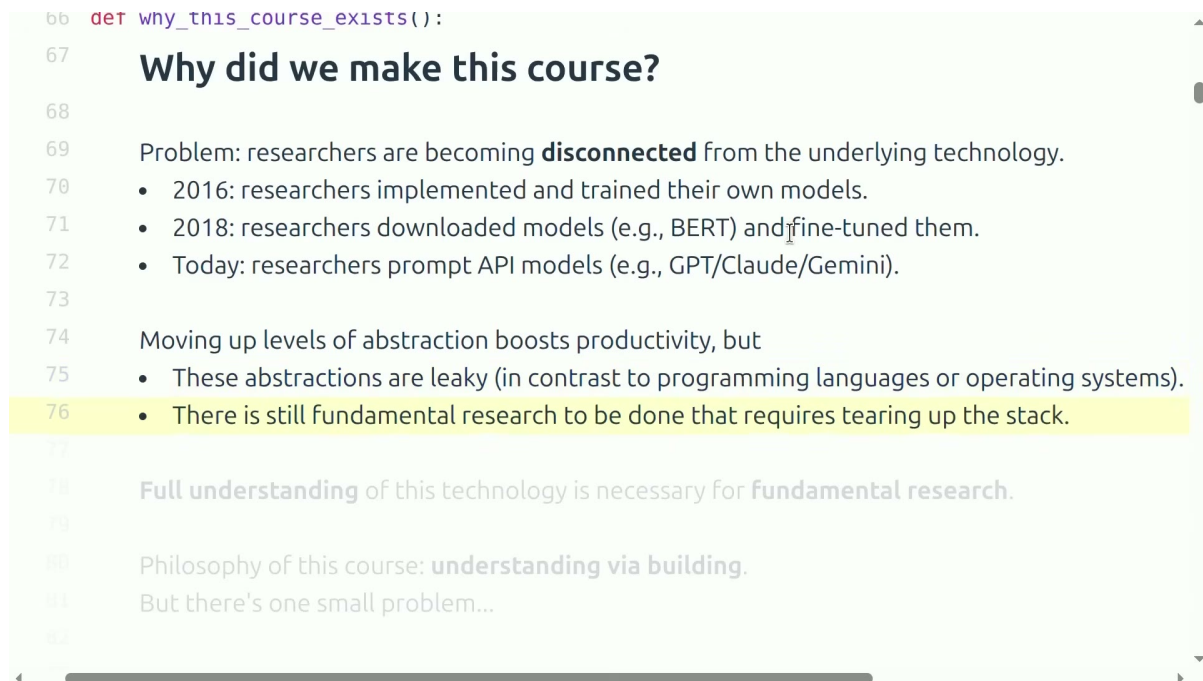


图 2: 课程存在的理由: 研究者逐渐与底层技术脱节, 而当前抽象仍不足以屏蔽底层差异。²

课程真正要培养的能力

目标不是记住一套 2026 年的最佳实践, 而是获得定位问题的能力: 看到损失异常时能追到数值尺度, 看到吞吐低时能追到数据搬运, 看到 token 成本高时能追到 tokenizer, 看到长上下文失效时能区分建模、训练分布和服务系统。

1.2 工业化之后, 公开课堂能学什么

前沿模型已经成为资本和基础设施密集型工程: 训练预算达到上亿美元, 集群规模达到十万 GPU 量级, 而闭源技术报告常省略数据、架构、训练和失败细节。小模型实验又不总能代表大模型: 注意力与 MLP 的 FLOPs 比例会随形状变化, 有些行为只在临界规模之后出现。

²视频画面时间区间: 00:04:15–00:04:30。

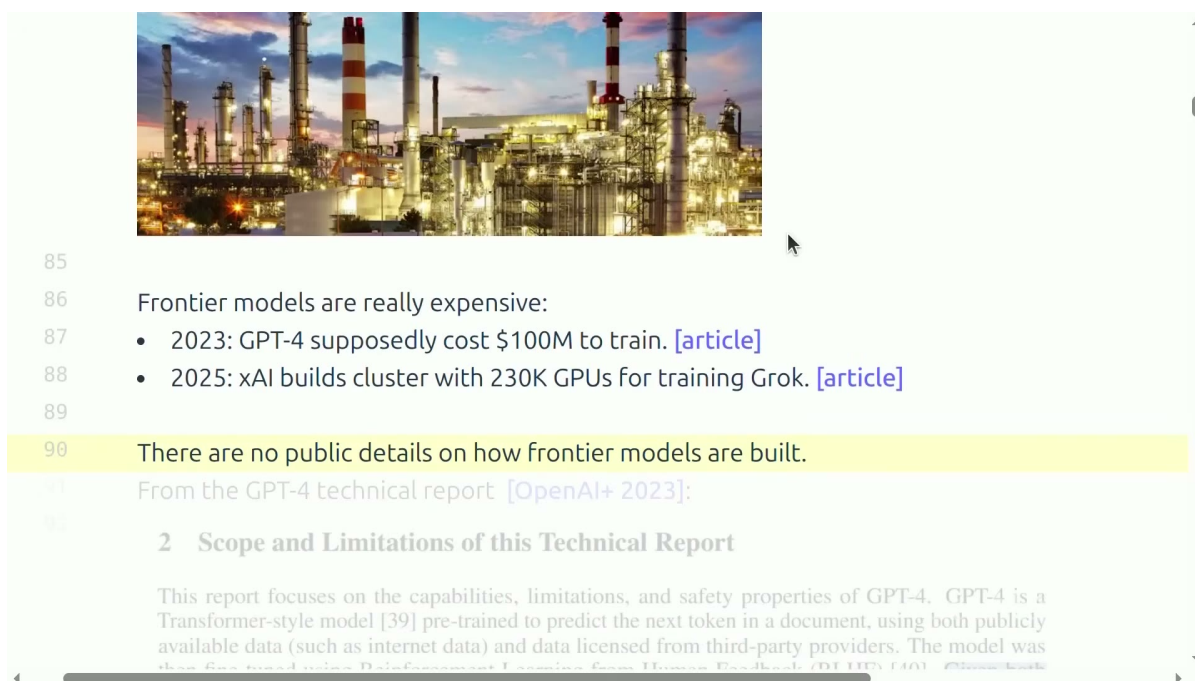


图 3: 前沿模型工业化: 训练成本和集群规模急剧上升, 但关键建模细节并未公开。³

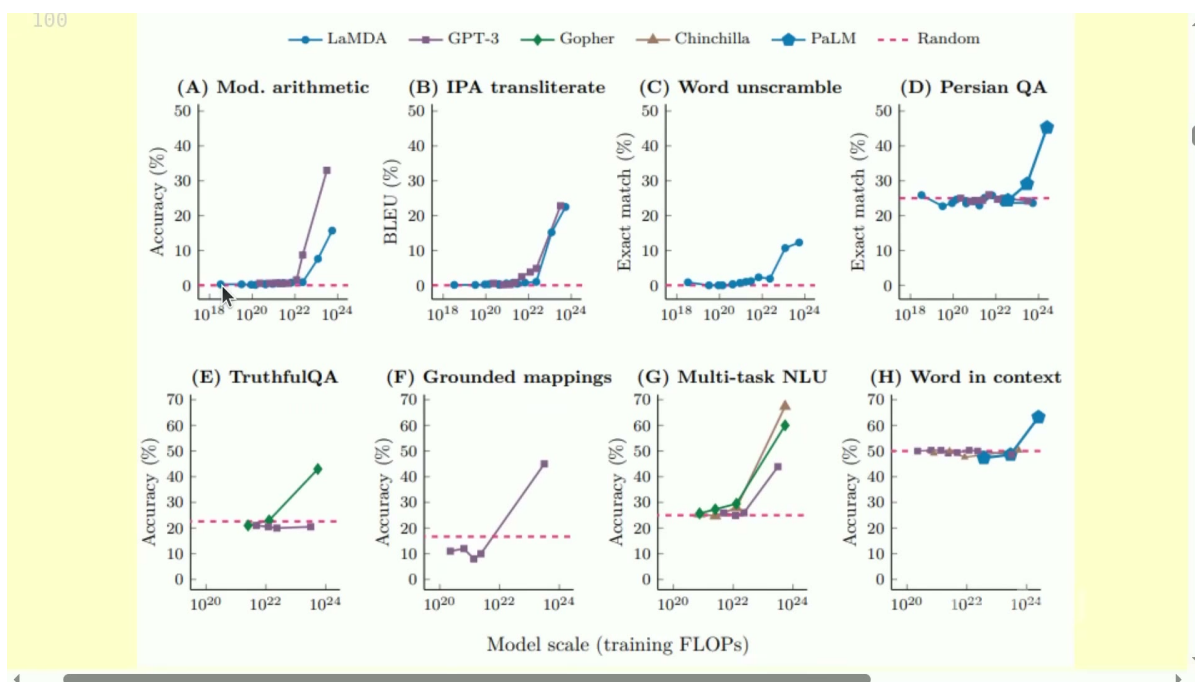


图 4: 规模变化可能改变能力曲线; 在小规模上看不到的现象, 不能简单外推为不存在。⁴

课程把知识分成三层:

- **Mechanics (机制):** Transformer、并行、kernel 等怎样工作;
- **Mindset (思维方式):** 认真核算资源、持续 profiling、把扩展性放在中心;
- **Intuitions (经验直觉):** 哪些数据和建模选择更准。

³视频画面时间区间: 00:05:15–00:05:30。

⁴视频画面时间区间: 00:07:00–00:07:15。

机制和思维方式通常能跨规模迁移；经验直觉却可能随数据、硬件和规模改变。课程承诺前两者，只对第三者给出有边界的经验。

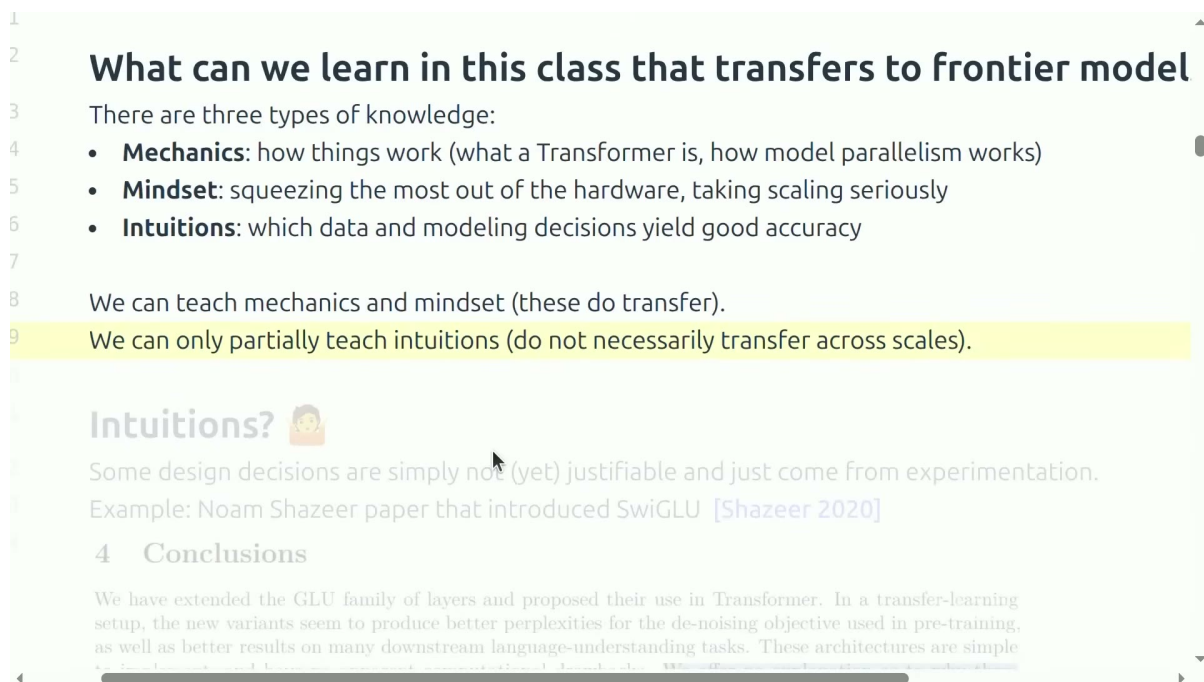


图 5：可迁移知识的三层结构：机制和效率思维较稳健，建模直觉需要谨慎外推。⁵

1.3 本章小结

“从零构建”是一种研究方法：通过实现建立跨层因果模型。小规模课堂无法复刻前沿训练，却能训练机制理解和效率思维；任何从小模型得到的质量直觉都应标注适用边界。

2 效率主线与“苦涩教训”

“苦涩教训”常被误读为“规模决定一切，算法不重要”。讲者给出的正确解释是：真正长期有效的是能够随资源扩展的算法。硬件、数据和算力增长不会替代算法改进；相反，规模越大，浪费一个百分点的成本越高。

$$\text{Accuracy} \approx \text{Efficiency} \times \text{Resources}.$$

- Accuracy：给定任务分布上的可用质量；
- Efficiency：单位数据、计算、显存和通信所换取的质量；
- Resources：可用数据、硬件、时间与资金。

⁵视频画面时间区间：00:08:00–00:08:15。

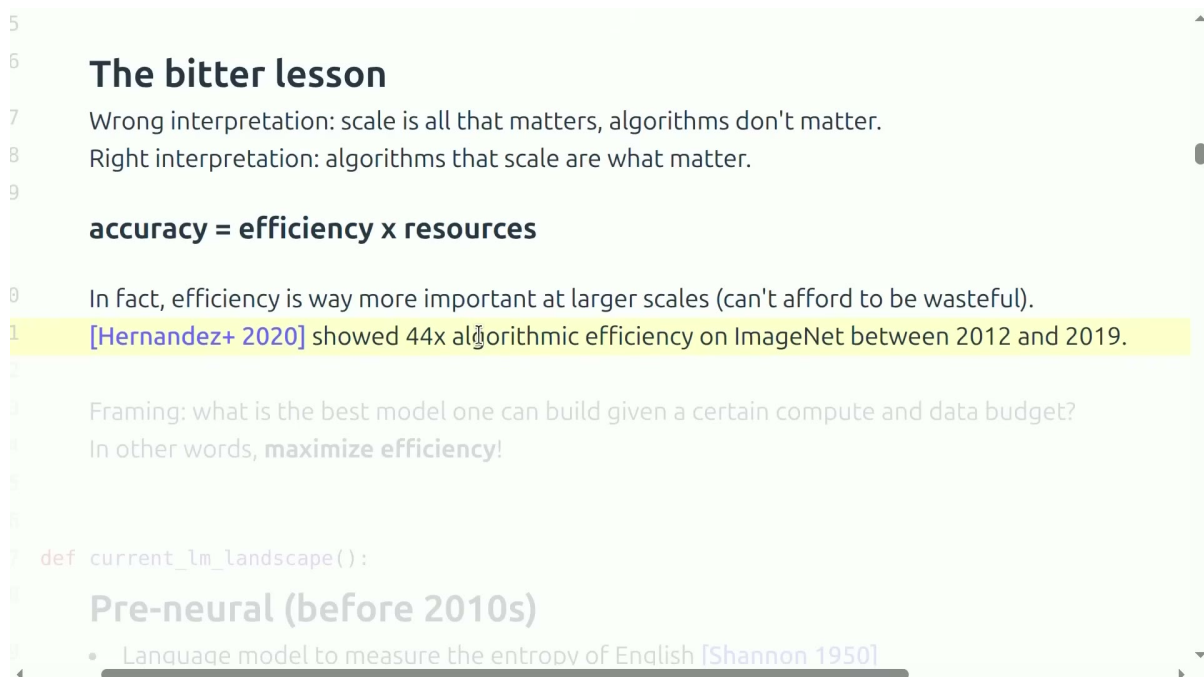


图 6: “苦涩教训”的课程版解释: 质量来自资源与算法效率的乘积, 而非单独依赖规模。⁶

2.1 同一套效率问题贯穿全栈

Tokenizer 用更有信息量的块缩短序列; 架构用 GQA、局部注意力或 MoE 调整计算与容量; 系统用 kernel fusion 和并行降低数据搬运; 数据过滤避免把昂贵更新浪费在低质量样本; 缩放定律用小实验选择大训练配方。它们看似属于不同章节, 本质都是同一个问题: **在固定资源预算下训练出最好的模型。**

效率不等于少做 FLOPs

真实效率同时取决于计算、显存容量、显存带宽、互连通信、并行度和可用内核。减少 FLOPs 若增加不规则访存或同步, 墙钟时间可能反而变差。课程后续会不断把理论计算量与硬件利用率分开核算。

2.2 本章小结

规模化并没有让算法失去意义, 而是放大了算法效率的价值。理解 CS336 最好的方式, 是把每一个组件都翻译成资源分配问题, 再用质量和墙钟成本共同验收。

3 语言模型发展史: 接口在变, 基本机制仍在

3.1 从统计模型到神经基础模型

早期语言模型用 n-gram 估计局部条件概率。2010 年代逐步形成今天的神经组件: LSTM 改善长期依赖, 神经语言模型学习分布式表示, seq2seq 和 attention 服务机器翻译, Adam 改变优化实

⁶视频画面时间区间: 00:10:45–00:11:00。

践，Transformer 用自注意力统一序列建模，MoE 与模型并行打开规模化通路。

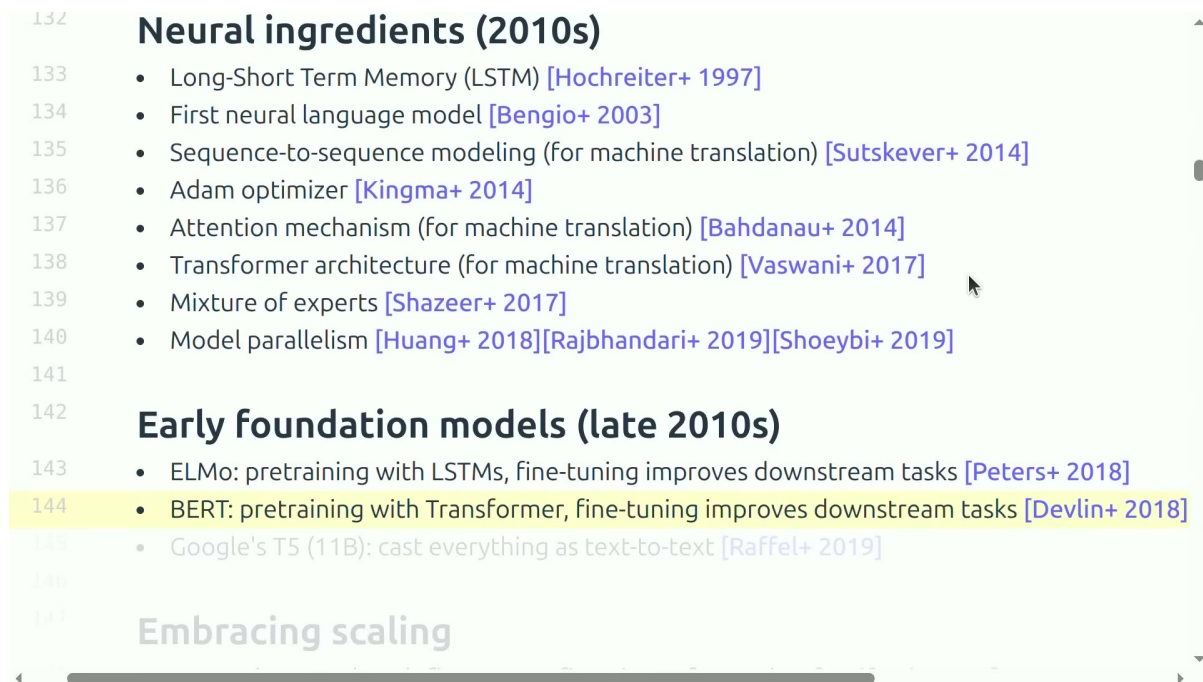


图 7：神经语言模型关键技术时间线：LSTM、神经 LM、seq2seq、Adam、attention、Transformer、MoE 与并行。⁷

随后，ELMo 与 BERT 建立“预训练后微调”的范式；GPT-2 暴露零样本能力；缩放定律给大训练带来可预测性；GPT-3 展示上下文学习；PaLM 和 Chinchilla 则推动计算最优训练讨论。这里的重点不是背诵模型名，而是理解接口与资源约束如何共同演化。

3.2 开放生态为何重要

开放权重只允许部署和评测；开放论文增加设计说明；开放代码允许复现实作；开放数据和开发过程才使因果追踪更完整。课程材料大量依赖 LLaMA、Mistral、DeepSeek、Qwen、Kimi、GLM 等开放权重模型，以及 OLMo、Nemotron、Marin 等更完整的开放项目。

⁷ 视频画面时间区间：00:13:30–00:13:45。

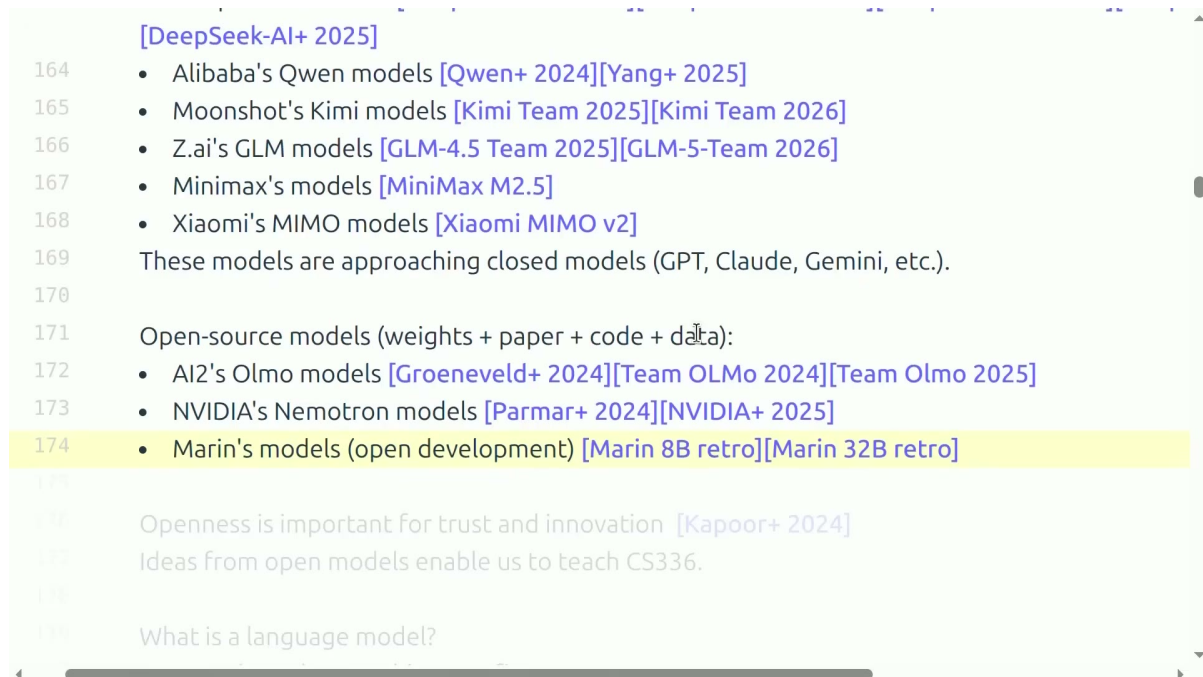


图 8: 开放模型版图: 开放权重、论文、代码和数据的程度不同, 能支持的研究问题也不同。⁸

“开放权重”不等于“开源训练”

拿到参数并不能回答数据如何清洗、失败实验如何处理、超参数为何选择、训练中断如何恢复。评估复现性时, 应明确区分权重、架构、训练代码、数据和开发日志五个层次。

3.3 语言模型接口的四次变化

2018 年, 语言模型是“下载后微调的表示”; 2020 年, 它成为“通过 prompt 调用的生成器”; 2022 年, 它成为“可以对话的系统”; 2026 年, 智能体把模型变成能自主采取行动、调用工具和维护长轨迹的组件。接口变化很大, 但 attention、kernel、优化等基础仍然存在; 变化最快的是上下文长度和推理效率要求。

⁸视频画面时间区间: 00:16:45–00:17:00。

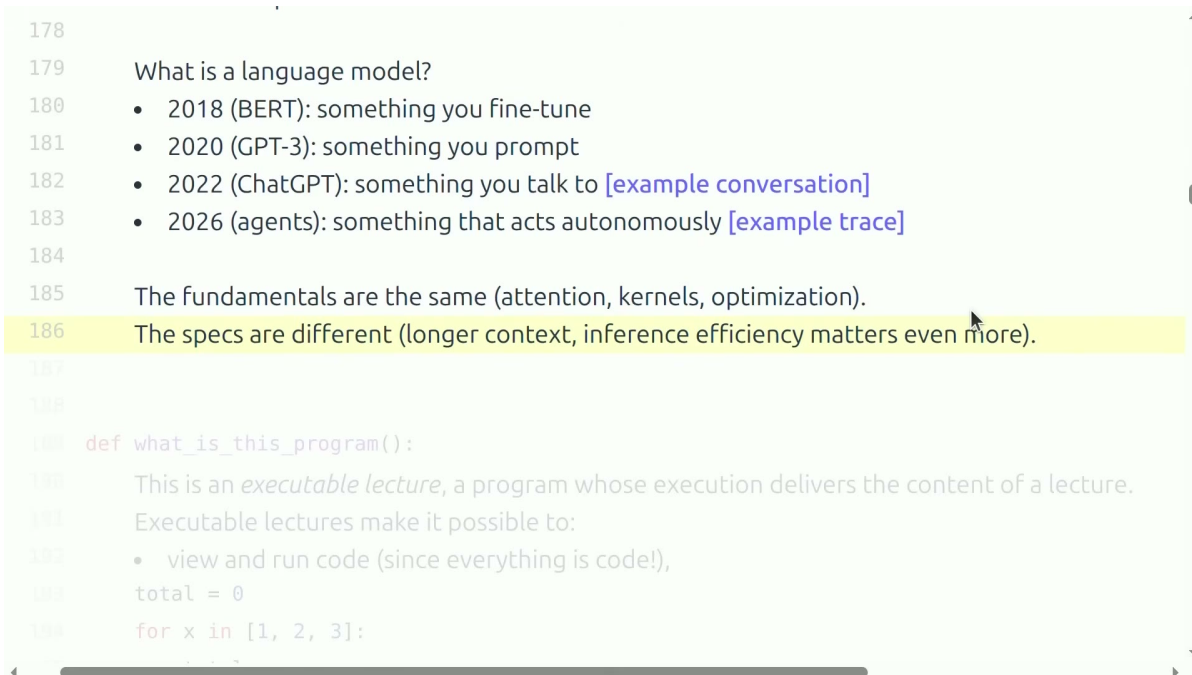


图 9: 语言模型接口从 fine-tune、prompt、chat 演化到 agent；底层机制稳定，系统规格持续变化。⁹

3.4 本章小结

语言模型的表面接口从概率估计一路演化到智能体，但表示、注意力、优化和系统执行仍是共同底座。开放生态让这些底座可被教学、验证和改进，也是从零构建课程能够成立的前提。

4 课程地图一：从 Tokenizer 到训练循环

CS336 用五个作业串起完整管线。Assignment 1 要求实现 BPE tokenizer、Transformer、交叉熵、AdamW 和训练循环，并在固定 B200 时间预算下优化困惑度。它不是把现成库拼起来，而是要求学生同时面对**表达力**、**稳定性**与**效率**三者的平衡。

4.1 Tokenization: 模型究竟在哪些“原子”上计算

原始输入是字节，模型处理的是整数 token 序列。Tokenizer 决定一个字符、词根、空格或代码片段占多少计算位置。BPE 把高频相邻字节组合成更大的块，常把约 1000 个字节压成约 250 个 token；同时，它把更多建模容量留给罕见或结构复杂的片段。

⁹视频画面时间区间：00:19:00–00:19:15。

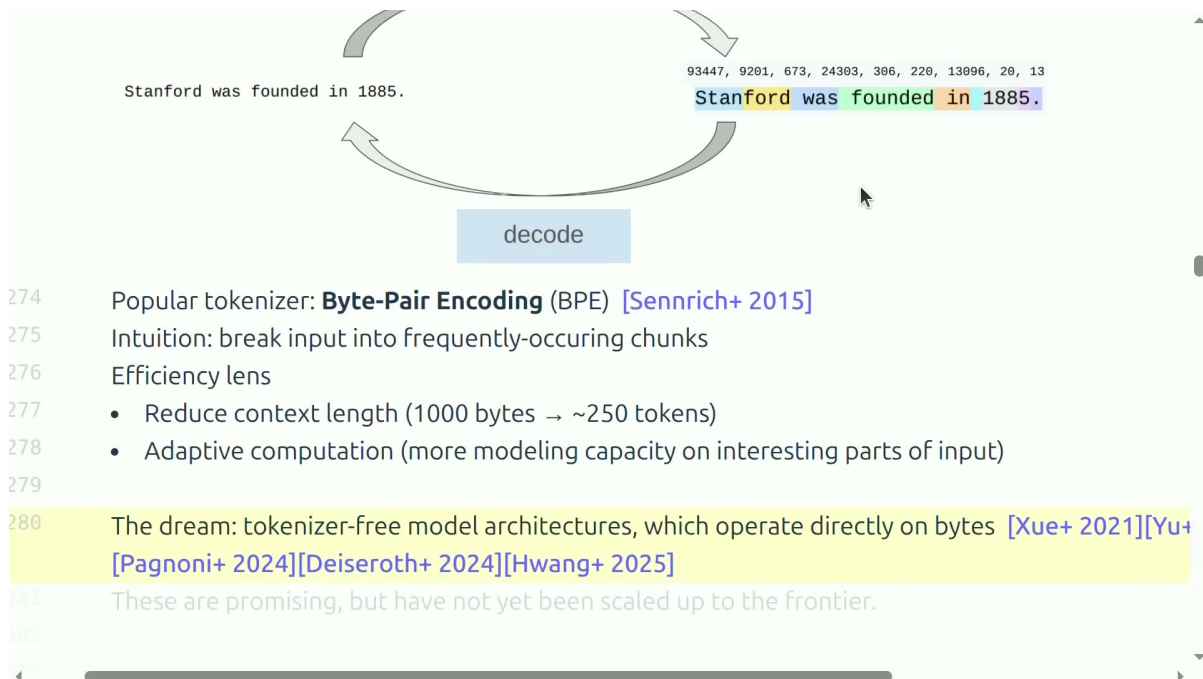


图 10: 课程中的 Tokenization: 在可逆编码前提下缩短上下文, 并进行数据驱动的自适应分块。¹⁰

4.2 模型架构: 从原始 Transformer 到现代组件

课程从原始 Transformer 出发, 再讨论 SwiGLU、RoPE、RMSNorm、QK-Norm、Pre-Norm、稀疏 / 局部注意力、GQA、MLA、状态空间模型和 MoE。架构问题不是组件越新越好, 而是每个选择如何改变计算、内存、稳定性、上下文传播和推理缓存。

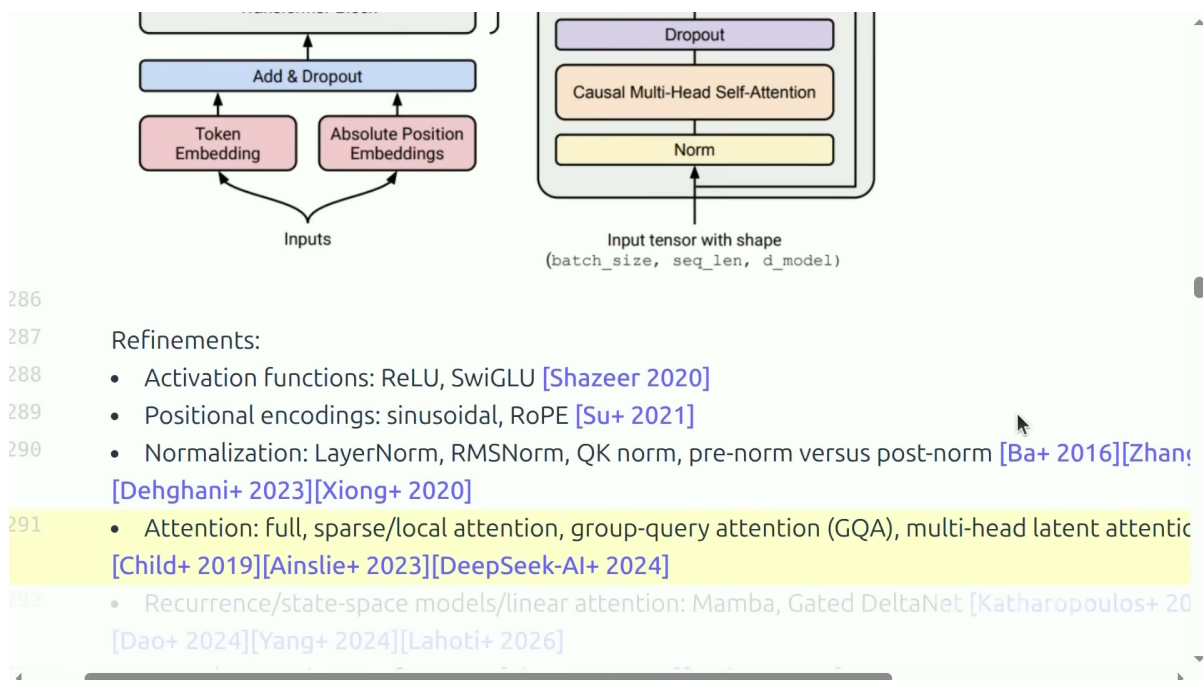


图 11: 现代语言模型架构的主要改进轴: 激活、位置、归一化、注意力、递归 / 状态空间与 MoE。¹¹

¹⁰ 视频画面时间区间: 00:29:15–00:29:30。

4.3 训练配方：参数如何被可靠地设置

训练配方包含损失函数、优化器、初始化、学习率日程、正则化、batch size 和 MoE 负载均衡。它们不能孤立调节：初始化改变激活尺度，学习率改变权重衰减的累计作用，batch size 改变梯度噪声，MoE 路由又改变专家负载。

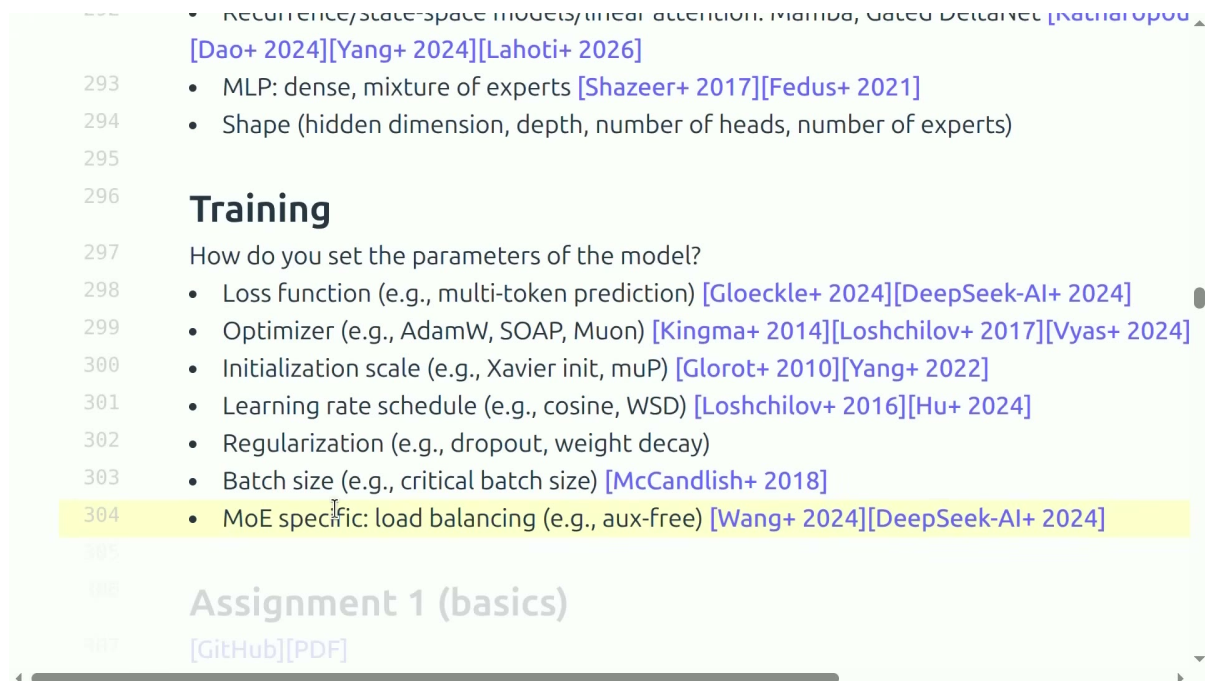


图 12: 训练配方的耦合组件：损失、优化器、初始化、学习率、正则化、批大小与专家负载。¹²

Assignment 1 的验收不只看正确性

Tokenizer 要可逆且高效；Transformer 要通过形状和梯度测试；优化器要匹配公式；训练循环还要在固定时间预算下获得低困惑度。正确实现只是起点，最终要建立“质量—速度—资源”联合指标。

4.4 本章小结

基础作业已经包含一个完整语言模型系统：Tokenizer 决定计算单位，架构决定容量与信息流，训练配方决定参数轨迹。三者共同作用，不能把最终质量归因于单个组件。

5 课程地图二：系统、并行与推理

Assignment 2 把注意力从数学对象变成硬件程序：学生要实现 Triton kernel、分布式数据并行、优化器状态分片，并系统地 benchmark 和 profile。核心原则是减少数据搬运，而不只是减少算术操作。

¹¹ 视频画面时间区间：00:30:45–00:31:00。

¹² 视频画面时间区间：00:33:15–00:33:30。

5.1 资源核算与 Roofline 视角

模型参数必须从 HBM 搬到计算单元。以 B200 为例，bf16 峰值计算吞吐约为 PFLOP/s 量级，而显存带宽为 TB/s 量级；一个算子能否“喂饱”计算单元，取决于每搬运一个字节能做多少 FLOPs。

训练稠密 Transformer 的粗略算力可写为：

$$C \approx 6ND.$$

- C ：一次完整训练的浮点运算量；
- N ：模型参数量；
- D ：训练 token 数；
- 系数 6：前向、反向和参数梯度的常用近似合计。

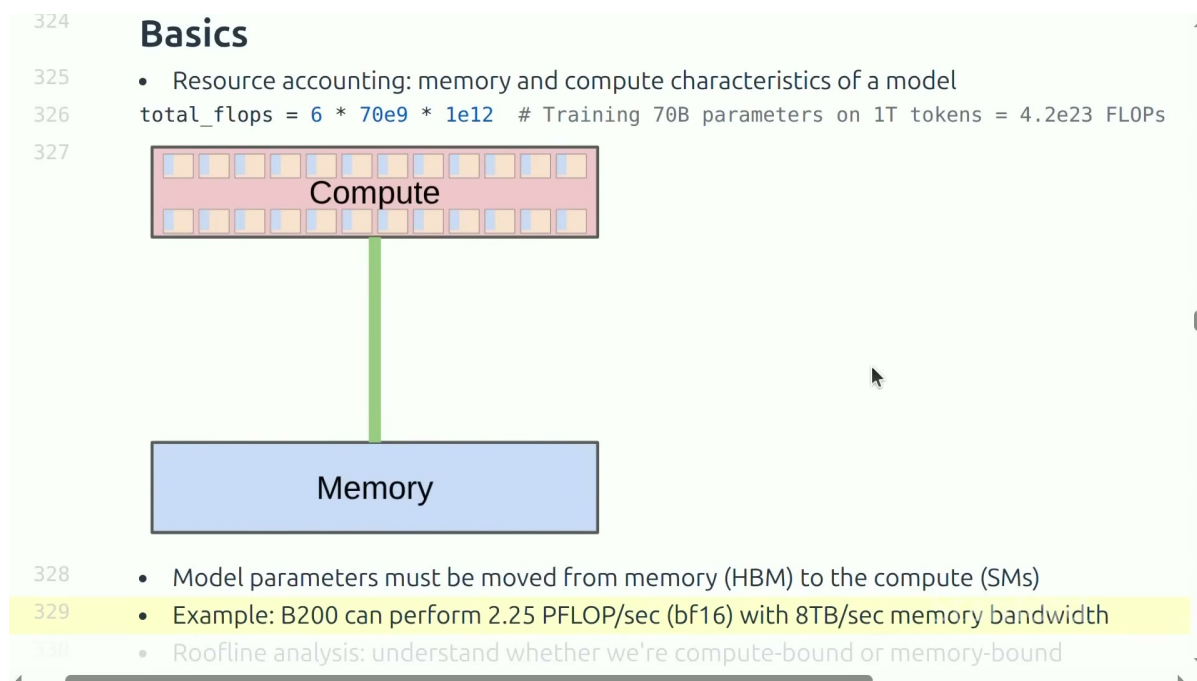


图 13: 资源核算的基本图景：计算单元与 HBM 之间的数据通路可能比理论 FLOPs 更早成为瓶颈。¹³

5.2 Kernel：把数学表达式组织成硬件友好程序

PyTorch 中每个 primitive 通常触发标准 kernel；自定义 kernel 可以融合中间步骤、复用片上存储、减少 HBM 往返。优化 FlashAttention 的本质便是改变循环与分块顺序，使注意力在不物化巨大中间矩阵的情况下完成相同计算。

¹³ 视频画面时间区间：00:37:30–00:37:45。

- Can write custom kernels to make GPUs go brrr
- Principle: organize computation to minimize data movement
- Naive: read HBM; compute A; write HBM; read HBM; compute B; write HBM
- Fused: read HBM; compute A and B; write HBM
- Strategies: operator fusion (matmul + activation), tiling (FlashAttention)
- Warp divergence, memory coalescing, bank conflicts, occupancy, bulk-async memory transfers
- Write kernels in CUDA/**Triton**/CUTLASS/ThunderKittens

Parallelism

- What if we have 1024 GPUs?
- Data movement between GPUs is even slower, but same 'minimize data movement' principle holds
- Use classic collective operations (e.g., gather, reduce, all-reduce)
- Shard memory (parameters, activations, gradients, optimizer states) across GPUs
- How to split computation: {data,tensor,pipeline,sequence,expert} parallelism

Inference

图 14: Kernel 优化的共同原则：融合操作、复用片上数据、减少 HBM 与计算单元之间的往返。¹⁴

5.3 并行：通信是跨 GPU 的数据搬运

当模型扩展到多 GPU，数据并行、张量并行、流水线并行、序列并行和专家并行会切分不同维度。GPU 互连通常比片上计算慢，因此并行方案的优劣不能只看每卡 FLOPs；还要看通信体积、同步频率、负载均衡和流水线气泡。

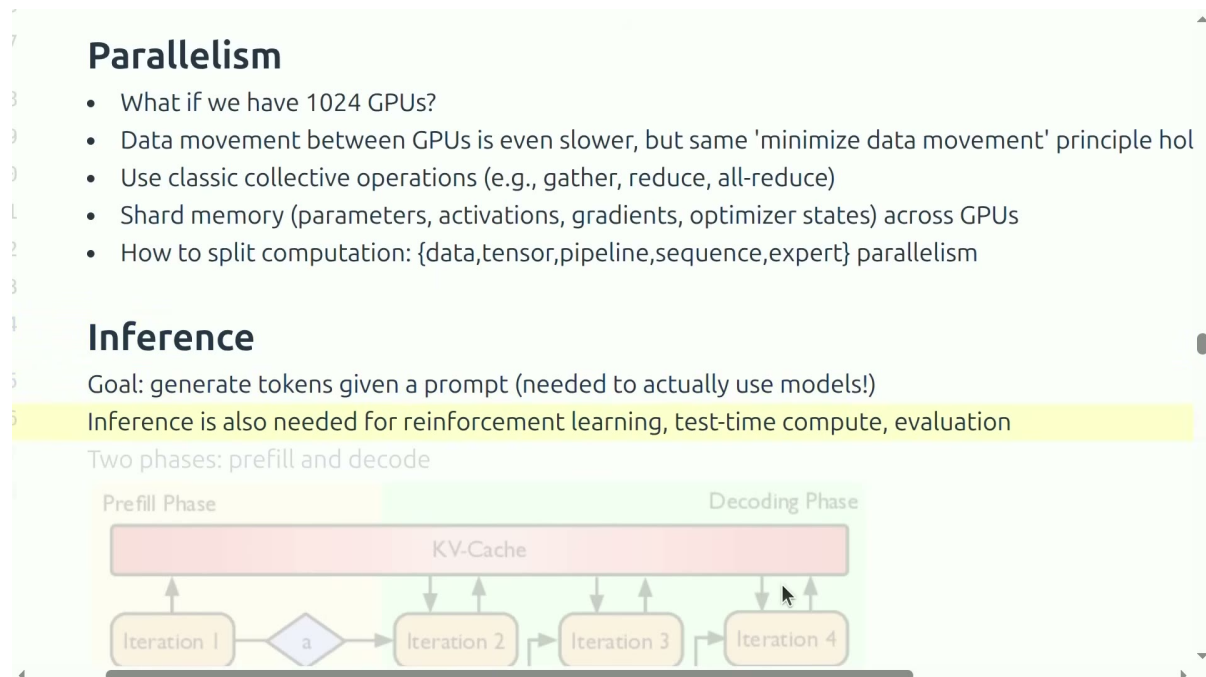


图 15: 多 GPU 并行仍遵循“最小化数据搬运”：共享参数、激活、梯度和优化器状态都带来通信。¹⁵

¹⁴ 视频画面时间区间：00:40:30–00:40:45。

5.4 推理：Prefill 与 Decode 是两种工作负载

Prefill 一次处理整段 prompt，矩阵规模大、并行度高，常偏计算受限；Decode 每次只生成一个 token，必须读取参数和 KV cache，且时间步之间串行，常偏显存带宽受限。量化、剪枝、蒸馏、投机解码、fused kernel 和连续批处理，分别从模型大小、候选生成和调度层面降低成本。

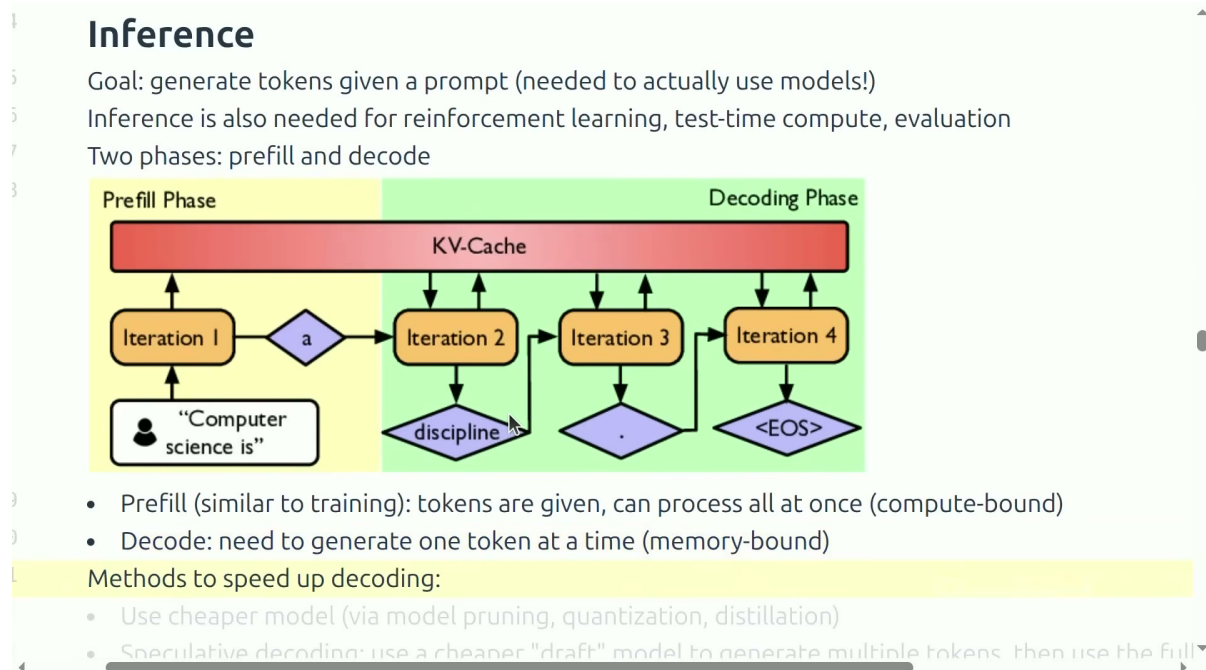


图 16: 自回归推理的两阶段：Prefill 可并行，Decode 逐 token 串行并逐步扩大 KV cache。¹⁶

训练吞吐不能代替推理指标

一个模型可以在训练时充分利用大矩阵，却在 Decode 时被 KV cache 和参数读取拖慢。部署评估至少要分别报告首 token 延迟、逐 token 延迟、并发容量和缓存字节 / token。

5.5 本章小结

系统章节把“效率”具体化为算术强度、数据搬运和通信。Kernel、并行与推理优化虽然处在不同层次，却共享一个问题：如何让昂贵数据在被搬运后得到更多有效复用。

6 课程地图三：缩放定律与可预测训练

前沿训练无法在目标规模上网格搜索超参数。缩放定律的关键转变，是从“某个固定规模的最优点”转向一条 **Scaling Recipe**：给定 FLOPs，自动产生模型宽深、batch、学习率、训练 token 等超参数。

¹⁵ 视频画面时间区间：00:41:45–00:42:00。

¹⁶ 视频画面时间区间：00:43:00–00:43:15。

6.1 用小实验优化大训练配方

流程包含三步：在多个较小算力预算上运行同一配方；拟合损失随算力的规律；把规律外推到目标规模。这样既能优化目标配方，也能在真正训练之前预注册预测损失。可预测性本身就是工程价值，因为它降低昂贵训练的决策风险。

```

3 def scaling_laws():
4     Setting: if you had 1e25 FLOPs of compute, what hyperparameters would you use to train a good m
5     Too expensive to do hyperparameter tuning at full scale!
6
7     Key conceptual shift: instead of a single scale, think of a scaling recipe (FLOPs → hyperparameters)
8     For a scaling recipe:
9     • Run experiments to compute the loss at various smaller scales (e.g., up to 1e24 FLOPs)
10    • Fit a scaling law to predict the loss of the scaling recipe at the target scale (e.g., 1e25 FLOPs)
11
12    Now you can:
13    1. Optimize the scaling recipe targeting a larger scale using smaller scale experiments
14    2. Predict the loss at the target scale before actually running the experiment!
15
16    Scaling laws don't happen automatically, they require careful construction of a scaling recipe.
17    Parameterize the model in a way to get hyperparameter transfer [Yang+ 2022]
18    Predictability is at least as important as optimality!
  
```

图 17: Scaling Recipe: 用多个小规模实验拟合规律，优化并预测目标规模的训练结果。¹⁷

6.2 等算力曲线：更大模型还是更多 token

固定 $C \approx 6ND$ 后，增大 N 就必须减少 D ，反之亦然。等算力实验在多个小预算下扫描模型大小，寻找每条曲线的最优点，再拟合最优 N 随 C 的变化。经典 Chinchilla 经验常写成 $D \approx 20N$ ，但它没有纳入推理成本：需要大量服务时，更小、训练更久的模型可能更经济。

¹⁷视频画面时间区间：00:47:15–00:47:30。

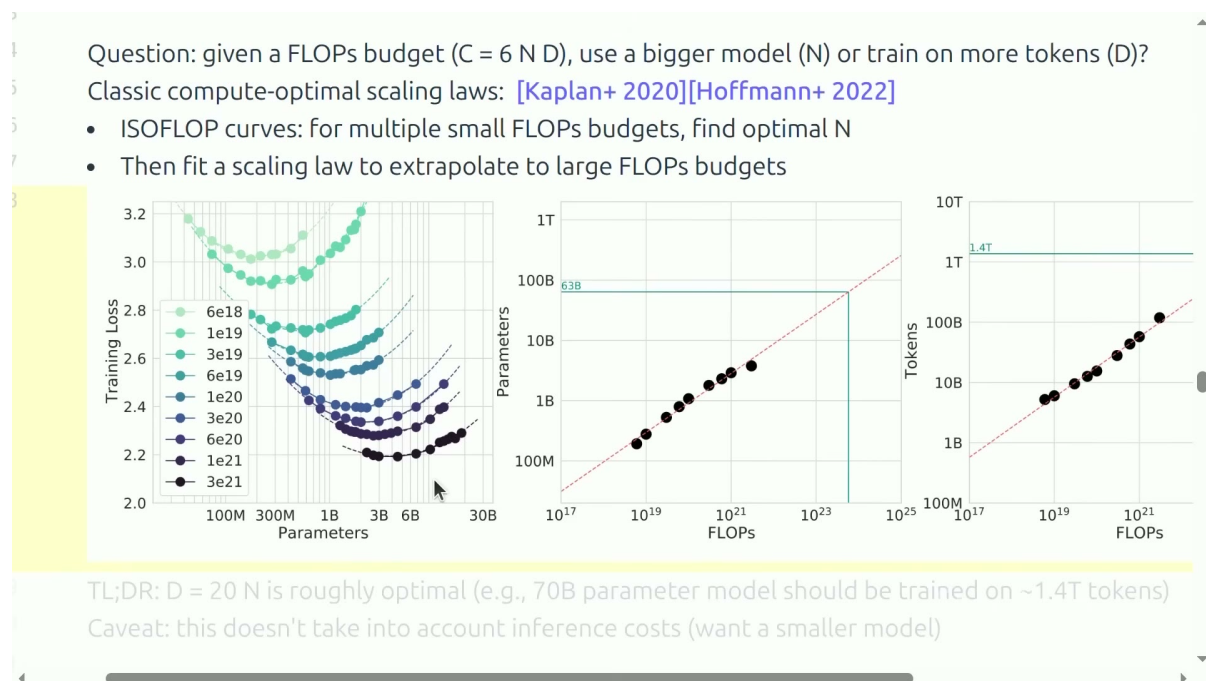


图 18: 等算力曲线: 在固定训练 FLOPs 下寻找模型参数与训练 token 的最佳分配。¹⁸

缩放规律不会自动出现

如果小实验改变了数据质量、参数化、优化稳定性或硬件利用率, 拟合到的曲线可能只是混合效应。可靠外推要求一条定义清楚、跨规模一致的 recipe, 并对超参数迁移做专门设计。

6.3 本章小结

缩放定律是决策工具, 不是自然常数。它通过小规模、受控、可比较的实验, 为大训练选择配方并量化风险; 推理成本和数据约束仍可能改变“计算最优”的答案。

7 课程地图四: 评测、数据与对齐

7.1 评测服务于两类不同目的

内部评测用于指导开发, 重视跨规模平滑和相对差异; 外部评测用于衡量真实用例, 重视生态有效性和绝对质量。困惑度适合连续监控, 但应尽量使用未被网络污染的私有文本; GPQA、HLE、SWE-Bench、Terminal-Bench 等高级任务则覆盖知识、推理和智能体能力。

7.2 数据不是“从天上掉下来”的

预训练数据来自网页、书籍、论文、代码等异构来源, 原始形态可能是 HTML、PDF 或目录而非干净文本。数据权利、许可和公平使用也是管线的一部分。多来源混合时, 体量最大的来源并不一定应该拥有最大的训练权重。

¹⁸ 视频画面时间区间: 00:50:00–00:50:15。

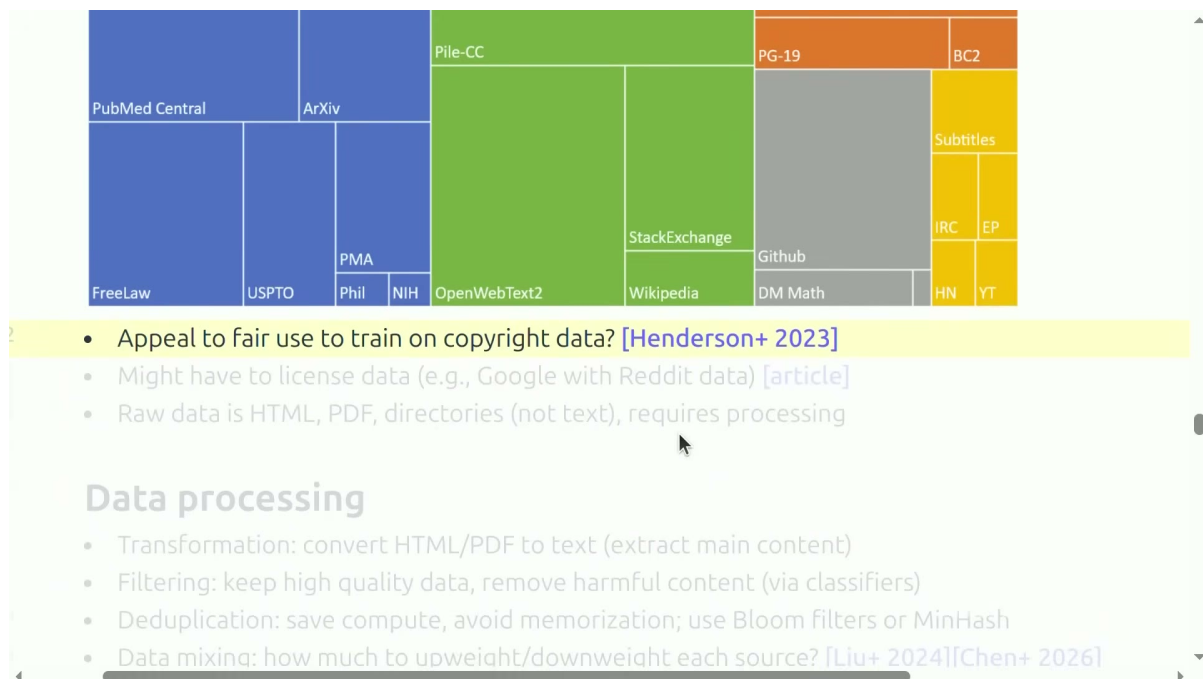


图 19: 典型预训练语料的来源构成：网页、论文、书籍、代码与其他领域数据共同组成混合分布。¹⁹

7.3 数据处理是一条有损决策链

转换把 HTML / PDF 变成正文；过滤移除低质量或有害内容；去重节省计算并降低记忆；混合调节不同来源权重；重写和合成使原始数据更接近目标任务。每一步都可能删除有价值的长尾，也可能把分类器偏见放大，因此必须用下游评测和数据审计闭环验证。

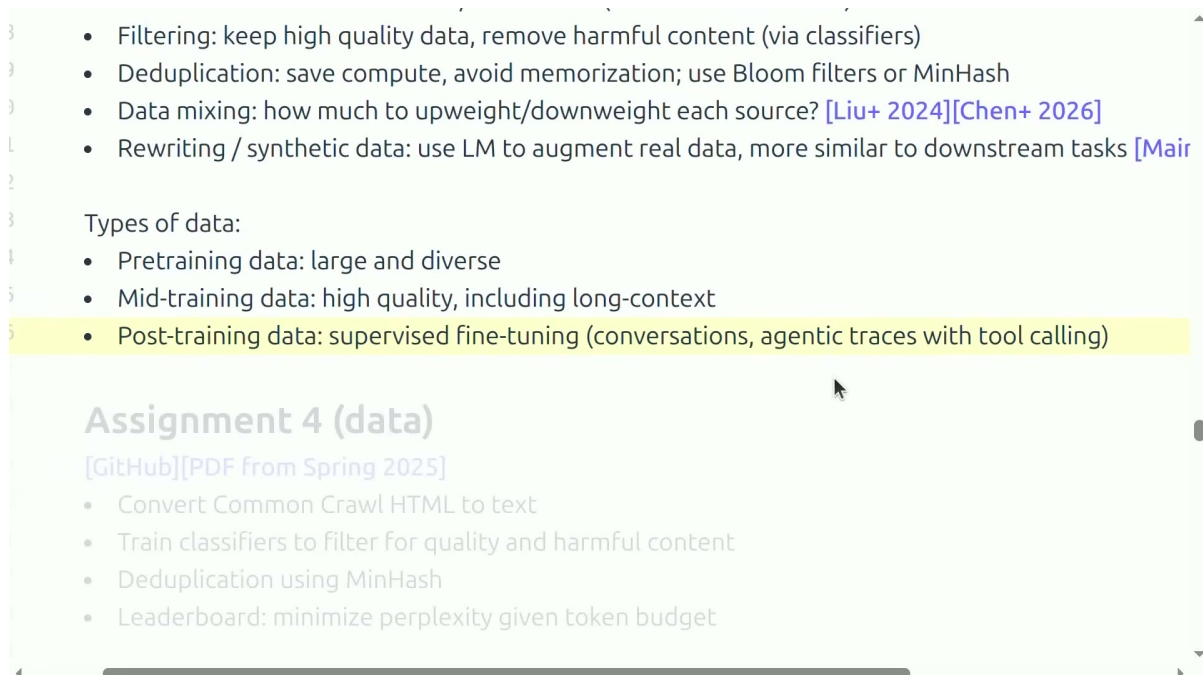


图 20: 数据处理主流程：转换、过滤、去重、混合与合成；不同阶段的数据承担不同训练目标。²⁰

¹⁹视频画面时间区间：00:57:30–00:57:45。

7.4 对齐：当评价比生成更容易

预训练使用完整监督——每个下一个 token 都有标签；对齐则利用弱监督：模型生成候选，由人类、验证器或 LM judge 打分，再更新模型偏好。DPO 直接使用偏好对，PPO 引入强化学习，GRPO 去掉价值函数。难点是奖励投机、训练不稳定，以及大规模 rollout 对推理系统的压力。

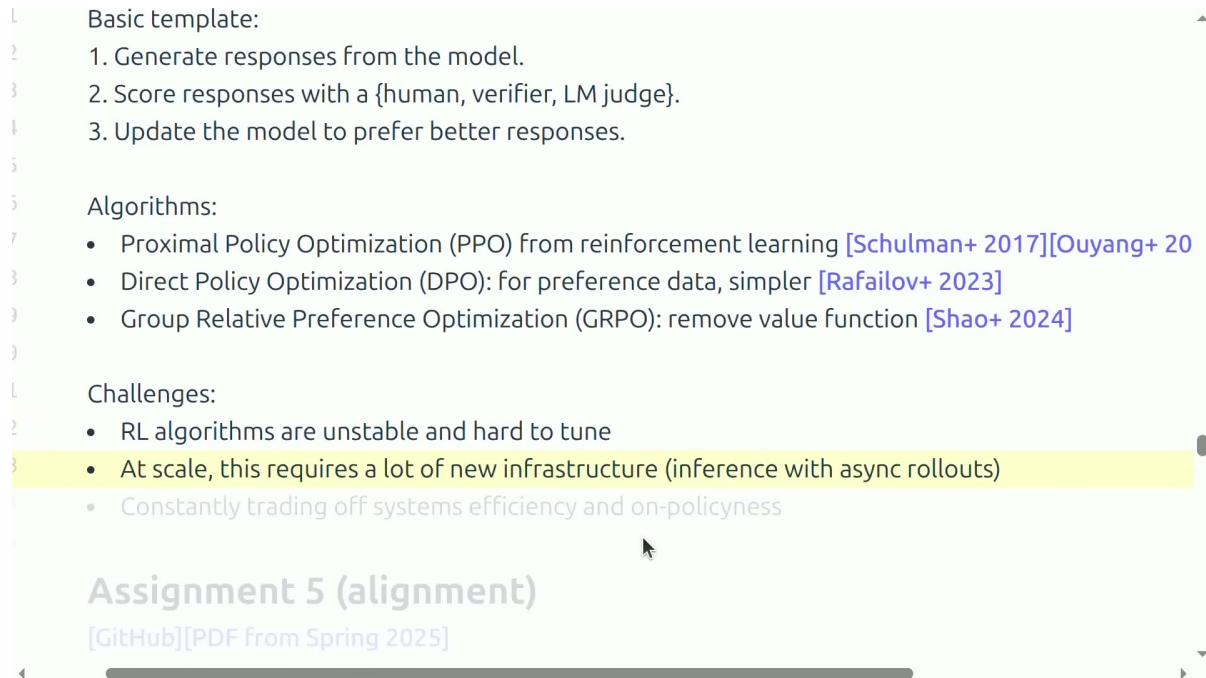


图 21：对齐基本循环与算法家族：生成、评分、更新，并在监督强度与在线性之间权衡。²¹

7.5 一条贯穿五个作业的主线

课程最后再次强调：当前训练往往受计算限制，因此 tokenizer、架构、数据过滤和缩放实验都围绕“不要浪费计算”展开；未来若高质量数据成为主要瓶颈，最优选择可能改变。资源约束不是背景条件，而是决定架构和算法价值的自变量。

²⁰ 视频画面时间区间：00:59:30–00:59:45。

²¹ 视频画面时间区间：01:01:45–01:02:00。

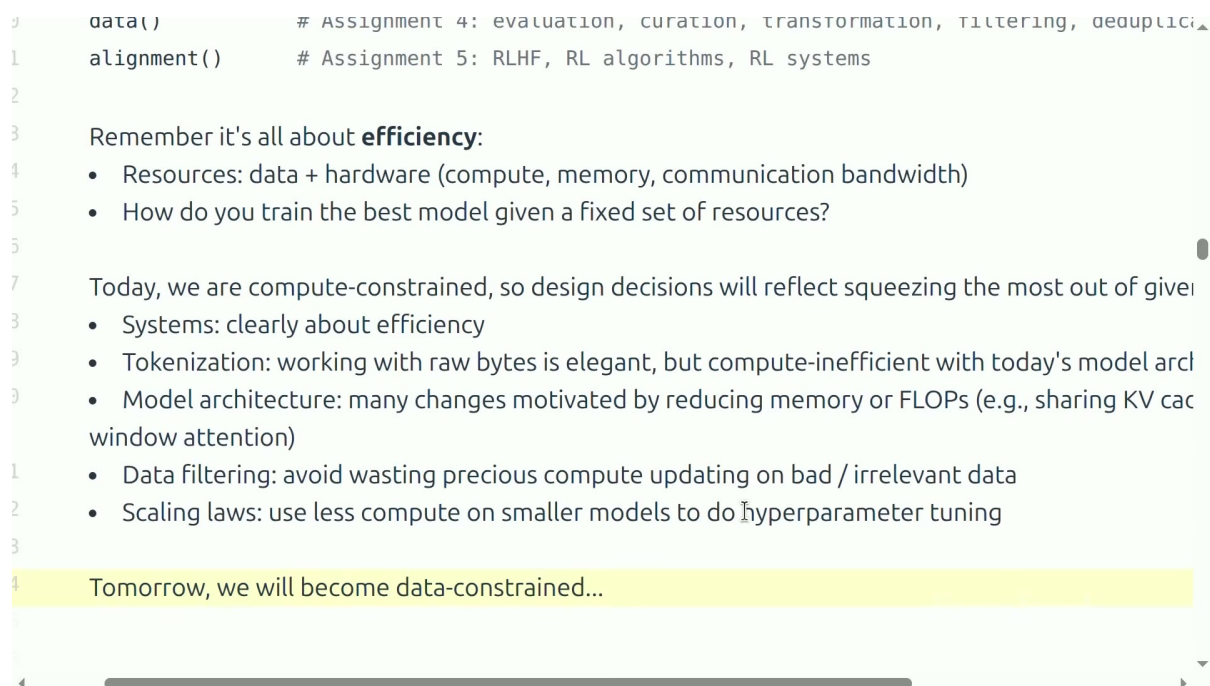


图 22: 全课程效率主线: 数据与硬件给出资源边界, 系统、Tokenizer、架构、数据过滤和缩放共同分配预算。²²

7.6 本章小结

评测定义目标, 数据决定学习分布, 对齐利用弱监督修正行为。三者必须闭环: 没有可靠评测就无法判断数据处理与对齐是否真的改善模型; 没有资源核算, 改进也可能无法扩展。

8 Tokenization 基础: 可逆表示与计算预算

语言模型对整数序列定义概率分布, 而用户输入通常是 Unicode 字符串。Tokenizer 必须在两种表示之间建立可逆映射:

$$E: \mathcal{S} \rightarrow \mathbb{N}^*, \quad D: \mathbb{N}^* \rightarrow \mathcal{S}, \quad D(E(s)) = s.$$

- $\mathcal{S}$: 合法输入字符串集合;
- E : encode, 把字符串映射为有限整数序列;
- D : decode, 把整数序列还原为字符串;
- $D(E(s)) = s$: round-trip 条件, 任何合法文本都不能在编码后丢失信息。

²² 视频画面时间区间: 01:04:30–01:04:45。

Listing 1: Tokenizer 的最小接口：编码与解码必须互为逆过程

```

1 class Tokenizer:
2     def encode(self, text: str) -> list[int]:
3         raise NotImplementedError
4
5     def decode(self, ids: list[int]) -> str:
6         raise NotImplementedError

```

8.1 现实 Tokenizer 的几个反直觉现象

常见 BPE Tokenizer 会把前导空格和单词合成一个 token，因此句首的“hello”与句中的“hello”可能不同；数字常按固定长度片段拆分；中英文和 emoji 的 token 数差异也很大。这些不是语言学真理，而是训练语料、预切分正则表达式和合并表共同产生的工程结果。

• A word and its preceding space are part of the same token (e.g., " world").
 • A word at the beginning and in the middle are represented differently (e.g., "hello hello").
 • Numbers are tokenized into a fixed number of digits.

string	= "Hello, 🌍! 你好!"
indices	= [13225, 11, 130321, 235, 0, 220, 177519, 0]
reconstructed_string	= "Hello, 🌍! 你好!"

```

Here's the GPT-5 tokenizer from OpenAI (tiktoken) in action
tokenizer = get_gpt5_tokenizer()
string = "Hello, 🌍! 你好!"

Check that encode() and decode() roundtrip:
indices = tokenizer.encode(string)
reconstructed_string = tokenizer.decode(indices)
assert string == reconstructed_string

Compression ratio: number of bytes per token
compression_ratio = get_compression_ratio(string, indices)

The larger the compression ratio, the shorter the sequence (good since attention is quadratic in seq length).

```

图 23: GPT 系 Tokenizer 的 round-trip 示例：Unicode 字符串被映射为整数序列后可无损恢复。²³

8.2 压缩率与词表大小

讲者把压缩率定义为每个 token 覆盖的 UTF-8 字节数：

$$r = \frac{|s_{\text{UTF8}}|}{|E(s)|}.$$

- r ：平均每个 token 表示的字节数，越大表示序列越短；
- $|s_{\text{UTF8}}|$ ：字符串编码后的字节数；

²³ 视频画面时间区间：01:07:15–01:07:30。

- $|E(s)|$: Tokenizer 输出的 token 数。

短序列能降低 attention 的二次成本,但提升压缩率通常需要更大词表;大词表会扩大 embedding 和输出 softmax, 且让低频 token 更稀疏。Tokenizer 因此是在**序列长度、词表参数和统计复用**之间分配预算。

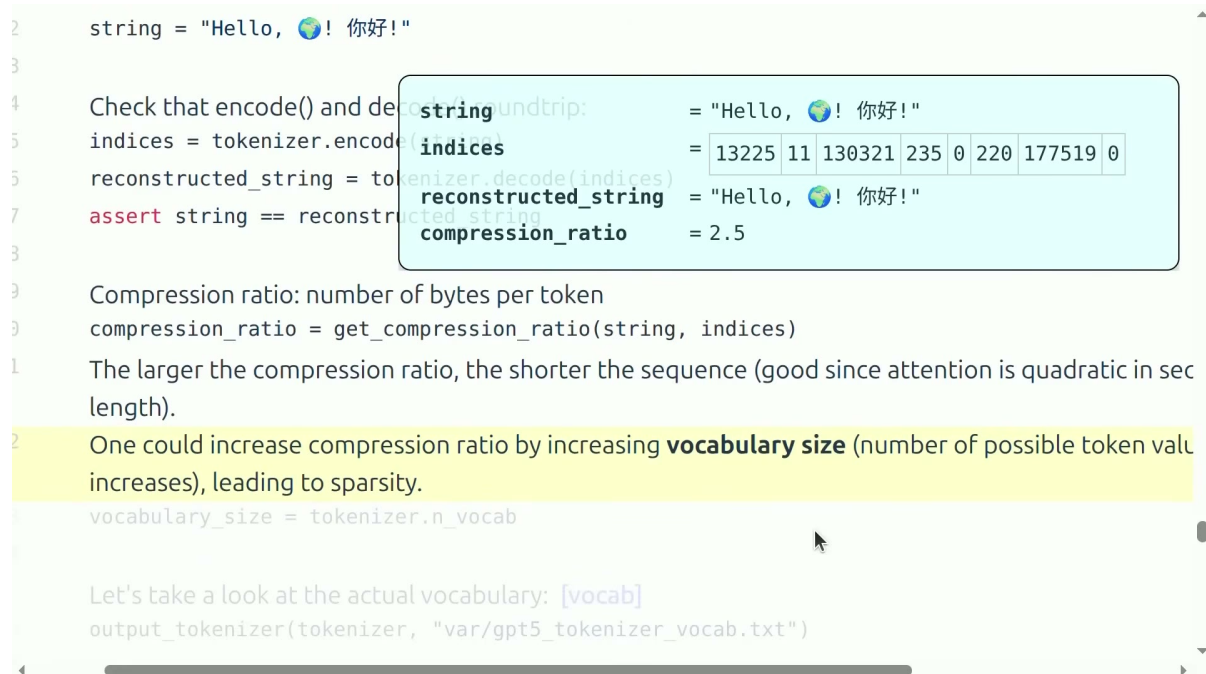


图 24: 压缩率与词表大小的权衡: 更大词表可缩短序列, 却会扩大稀疏表示和输出层。²⁴

8.3 本章小结

Tokenizer 首先必须可逆, 其次才比较效率。压缩率高能缩短上下文, 但词表过大又会浪费参数和样本; 任何方案都应同时报告 round-trip、token 数、词表规模和下游成本。

9 三种直接方案为什么都不理想

9.1 字符级: 语义直观, 但词表大且长尾严重

字符级 Tokenizer 把每个 Unicode code point 当作 token。实现简单、round-trip 清晰, 但 Unicode 字符约有十万量级, 许多字符极少出现; 模型要为罕见字符保留独立 embedding, 却很难获得足够训练信号。与此同时, 一个字符不一定对应一个字节, 跨语言压缩率差异明显。

²⁴ 视频画面时间区间: 01:08:00–01:08:15。

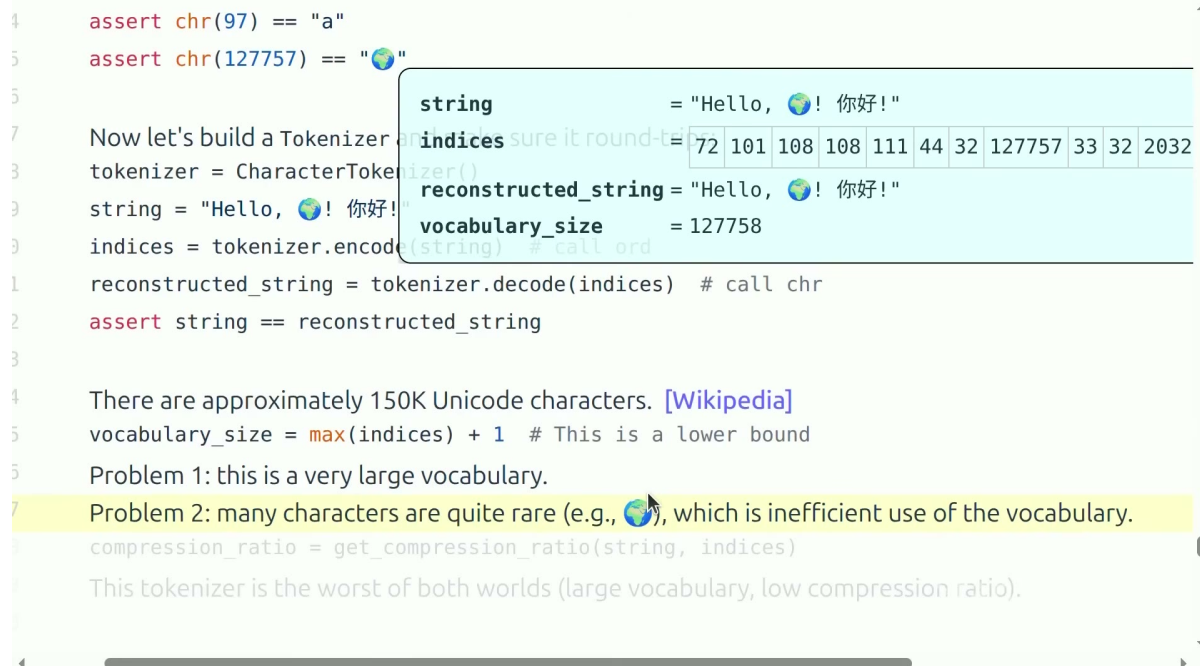


图 25: 字符级 Tokenizer: 直接使用 Unicode code point, 词表很大且大量字符低频。²⁵

9.2 字节级: 词表固定, 却把序列拉得过长

UTF-8 把任何字符串编码为 0–255 的字节序列, 因此字节级方案只需 256 个 token, 不存在未知字符。然而每个 token 只承载一个字节, 压缩率恒为 1; 中文和 emoji 需要多个 token, 序列很长, 而标准 attention 成本随长度平方增长。

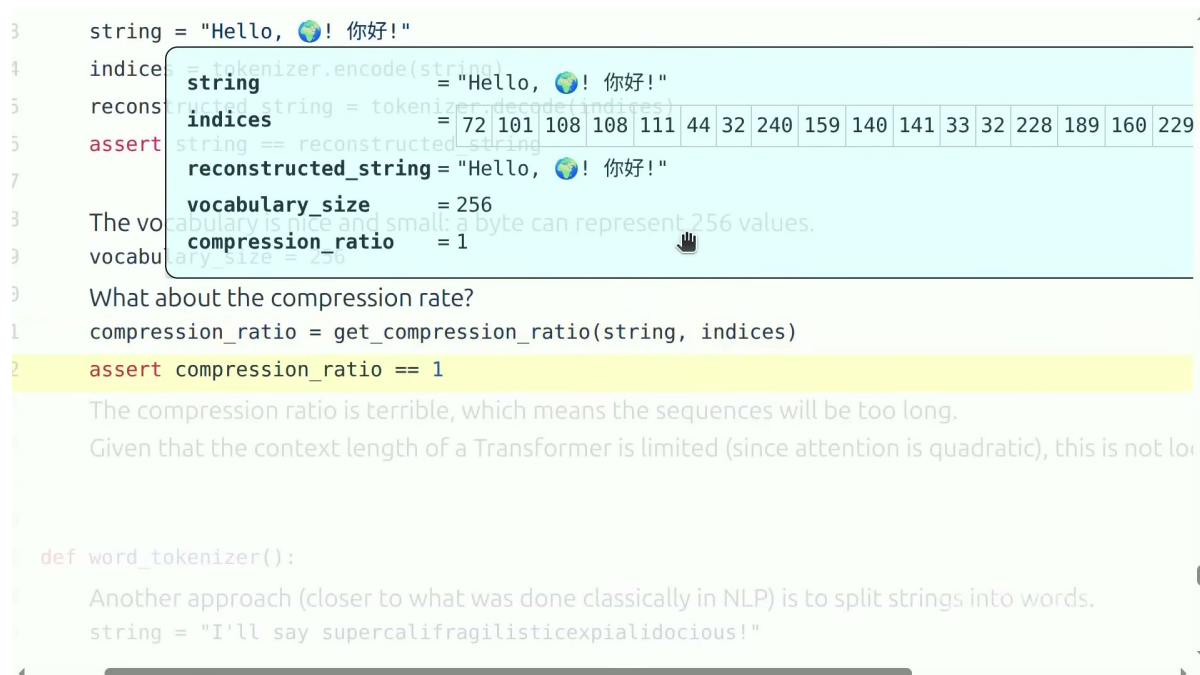


图 26: 字节级 Tokenizer: 词表仅 256, 但压缩率为 1, 长序列会放大 attention 成本。²⁶

²⁵ 视频画面时间区间: 01:09:15–01:09:30。

9.3 词级：压缩好，却无法封闭词表

按正则表达式切词能得到人类可解释的单元，压缩率也较高；但专有名词、拼写变化、复合词、代码标识符和新词不断增长，词表没有自然上限。测试时未见词只能映射成 UNK，这会丢失原始信息并污染困惑度计算。

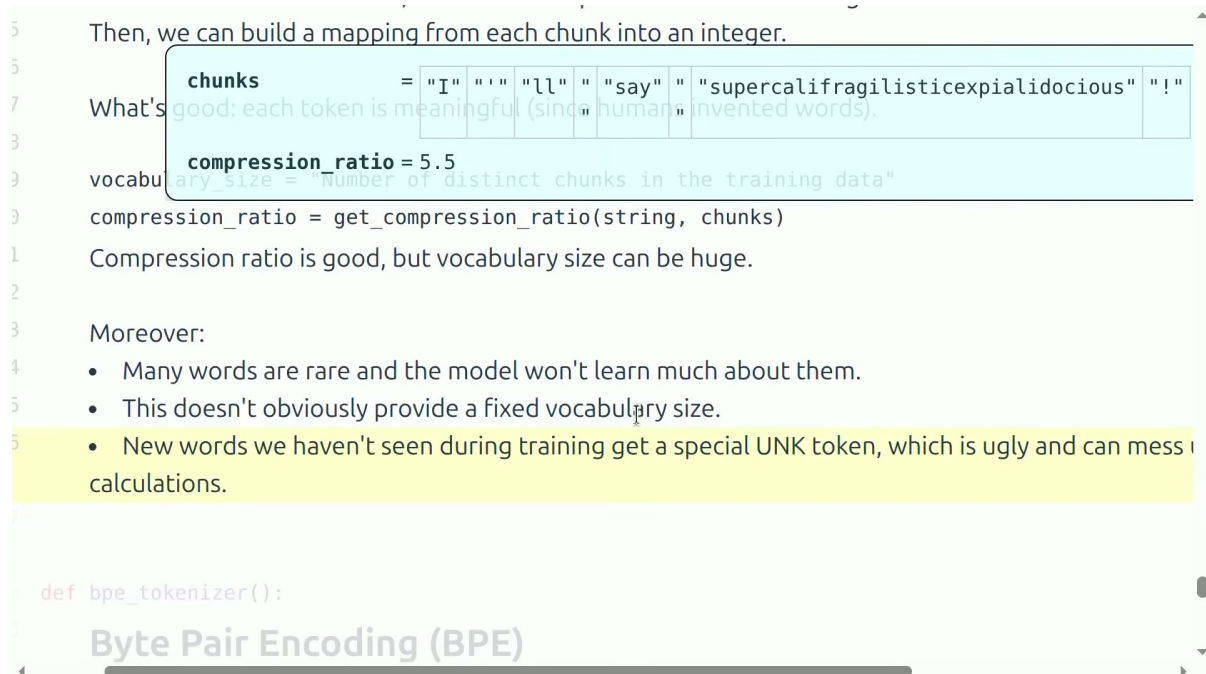


图 27: 词级 Tokenizer: 语义单元清晰，但长尾词和未登录词使固定词表难以成立。²⁷

三种方案的共同矛盾

字符级浪费词表，字节级浪费序列位置，词级无法覆盖长尾。理想方案要以小而封闭的基础字节集保证可逆，同时让高频片段共享更大的 token 以提高压缩率。

9.4 本章小结

字符、字节和词分别优化了可解释性、封闭词表和压缩率中的一部分，却无法同时满足三者。BPE 的价值正是用数据驱动的多尺度单元连接这三个极端。

10 Byte Pair Encoding: 从字节学习可变粒度词表

10.1 基本直觉

BPE 最初是压缩算法，后来用于神经机器翻译并被 GPT-2 普及。训练从 256 个字节 token 开始，反复找出当前序列中最常见的相邻 token 对，把它合并为新 token。高频序列很快变成单个 token，罕见序列仍可退化为原始字节，所以不存在 UNK。

²⁶ 视频画面时间区间: 01:10:30–01:10:45。

²⁷ 视频画面时间区间: 01:11:45–01:12:00。

```
def bpe_tokenizer():
```

Byte Pair Encoding (BPE)

The BPE algorithm was introduced by Philip Gage in 1994 for data compression. [\[article\]](#)

It was adapted to NLP for neural machine translation. [\[Sennrich+ 2015\]](#)

(Previously, papers had been using word-based tokenization.)

BPE was then used by GPT-2. [\[Radford+ 2019\]](#)

Basic idea: *train* the tokenizer on raw text to construct a vocabulary tailored to the data.

Intuition: common sequences of bytes are represented by a single token, rare sequences are represented by many tokens.

Sketch: start with each byte as a token, and successively merge the most common pair of adjacent tokens.

Training the tokenizer

```
string = "the cat in the hat"
```

```
params = train_bpe(string, num_merges=3)
```

图 28: BPE 的核心: 从字节出发, 按语料频率逐步构建可变粒度词表。²⁸

设当前 token 序列为 z , 相邻对计数为 $c_z(a, b)$, 第 t 次合并选择:

$$(a_t, b_t) = \arg \max_{(a, b)} c_z(a, b), \quad v_t = 256 + t.$$

- $c_z(a, b)$: token 对 (a, b) 在当前序列中的相邻出现次数;
- (a_t, b_t) : 第 t 轮最常见的相邻对;
- v_t : 为该合并片段分配的新 token ID;
- 每轮替换后要重新统计, 因为合并会改变邻接关系。

²⁸视频画面时间区间: 01:12:45–01:13:00。

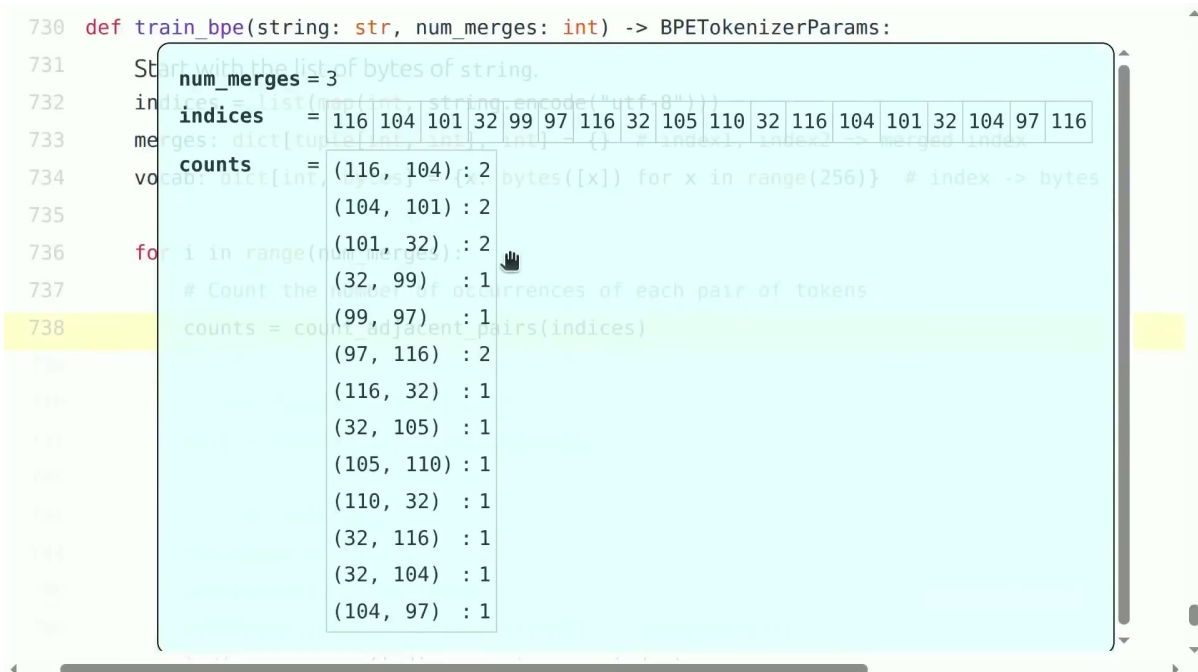


图 29: BPE 训练过程：统计相邻对、选择最高频组合、创建新 ID 并替换全部匹配。²⁹

10.2 最小实现与复杂度

Listing 2: 教学版 BPE 训练循环，生产实现需使用增量计数和高效索引

```

1 tokens = list(text.encode("utf-8"))
2 merges = {}
3 vocab = {i: bytes([i]) for i in range(256)}
4
5 for step in range(num_merges):
6     counts = count_adjacent_pairs(tokens)
7     pair = max(counts, key=counts.get)
8     new_id = 256 + step
9     merges[pair] = new_id
10    vocab[new_id] = vocab[pair[0]] + vocab[pair[1]]
11    tokens = merge_all(tokens, pair, new_id)

```

朴素实现每轮扫描全序列并重算全部 pair，训练和编码都很慢。Assignment 1 要求只更新受合并影响的计数、只检查相关 merge，并正确保护特殊 token。真正困难的地方通常不是十行算法，而是大语料上的数据结构、并行 I/O、预切分和确定性。

10.3 用合并表编码新文本

训练结果由基础词表和有序 merge 规则组成。编码新字符串时先转为字节，再按训练顺序应用相关合并；解码则把每个 token ID 映射回字节串并拼接，最后用 UTF-8 还原字符串。

²⁹ 视频画面时间区间：01:13:30–01:13:45。

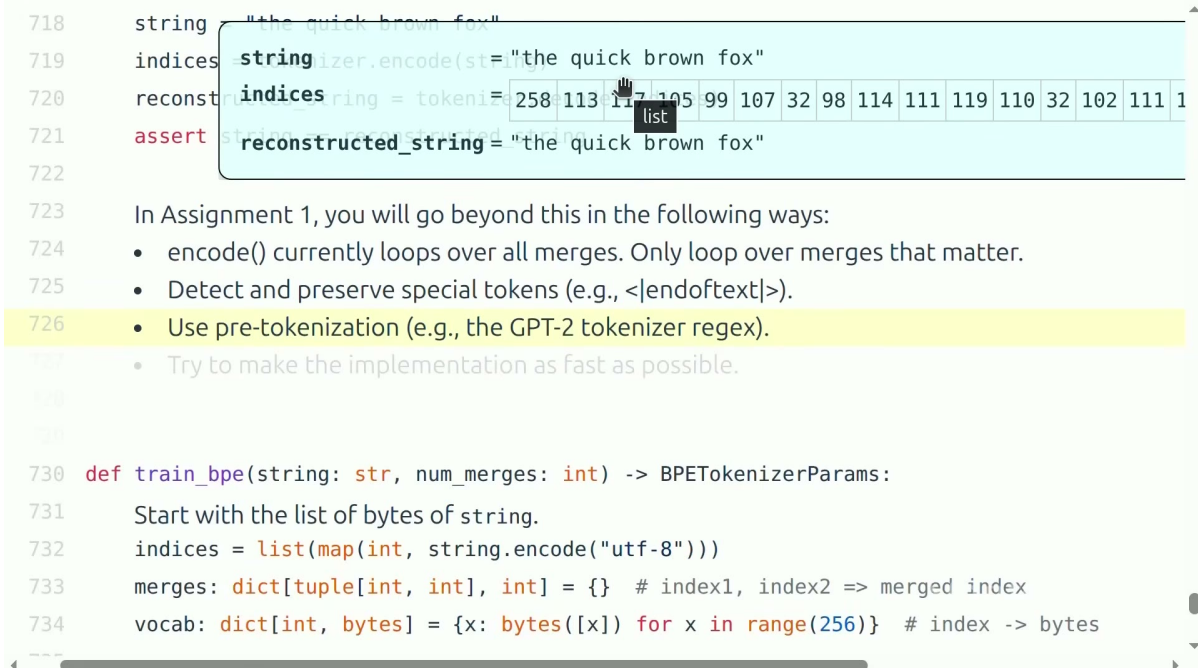


图 30: 使用训练后的 BPE: 对新文本应用合并规则, 编码结果仍能通过字节词表无损解码。³⁰

Tokenizer 是模型协议的一部分

训练、推理和评测必须使用完全相同的词表、merge 顺序、预切分规则与特殊 token 定义。只替换 Tokenizer 而不重训 embedding, 会让所有 token ID 的语义失效; 评测时忽略不同 Tokenizer 的 token 数, 也会误比较困惑度和成本。

10.4 Tokenizer-free 的长期方向

BPE 仍是独立于模型的预处理启发式。更理想的系统可能直接从字节学习分块, 使块边界与模型目标共同训练; 但现有 Transformer 对长字节序列的成本太高, 这类方案尚未在前沿规模普遍验证。无论未来采用何种机制, 都要满足两个条件: 模型在有意义的块上计算, 且块大小能随信息密度变化。

³⁰ 视频画面时间区间: 01:17:00–01:17:15。


```

tokenizer()

try:
    tokenizer: strings ↔ tokens (indices)
    character-based, byte-based, word-based tokenization are highly suboptimal
    BPE is an effective heuristic that is data-driven
    tokenization is a separate step, maybe one day do it end-to-end from bytes...

    whatever solution needs to satisfy:
    model (e.g., Transformer) should operate on chunks (abstractions) of the sequence (text, video, DNA, etc.)
    chunk sizes should be variable (allocate more model capacity to interesting chunks)

    characterTokenizer(Tokenizer):
    """Represent a string as a sequence of Unicode code points."""
    @code(self, string: str) -> list[int]:
    def code(self, string: str) -> list[int]:
    return list(map(ord, string))

```

图 31: Tokenization 总结: BPE 是有效的数据驱动启发式, 未来方向是端到端学习可变粒度分块。³¹

10.5 本章小结

BPE 用字节保证可逆, 用频繁合并提高压缩率, 用固定 merge 数控制词表规模。它解决了字符、字节和词级方案的主要矛盾, 但训练效率、特殊 token、预切分和跨语言公平性仍需工程化处理。

11 总结与延伸

11.1 讲者总结

Lecture 1 建立了整门课的坐标系: 从零构建不是为了复刻闭源前沿模型, 而是为了掌握可迁移的机制和效率思维; 课程的每个模块都在固定资源下平衡表达力、稳定性与效率。分词作为第一个具体模块, 展示了这种方法: 把抽象的“文本输入”拆成可逆性、词表、序列长度和计算成本, 再用代码验证权衡。

11.2 结构化提炼

1. **抽象必须可下钻。**高层 API 提升生产率, 但研究创新需要能追到数据、数值和系统边界。
2. **效率是乘数。**规模越大, 算法、数据与硬件利用率的一点改进价值越高。
3. **预测性与最优性同样重要。**稳定的缩放 recipe 能在投入大训练前量化结果与风险。
4. **Tokenizer 是资源分配器。**它决定每种文本消耗多少序列位置, 也决定 embedding 和 softmax 的词表成本。

³¹ 视频画面时间区间: 01:18:30–01:18:45。

5. 实现暴露隐含假设。Round-trip、特殊 token、merge 顺序和数据结构都不是公式里自动包含的细节。

Lecture 1 的一句话结论

真正的 “from scratch” 不是拒绝现成工具，而是让每个抽象都能被解释、测量和替换；从 Tokenizer 开始，把语言模型重新看成一套受资源约束的可执行系统。

11.3 建议练习

1. 为同一份中英代码混合文本计算字符、字节和 BPE 的 token 数、压缩率与 round-trip。
2. 实现朴素 BPE 后，用 profile 找到每轮全量计数的瓶颈，再设计增量更新结构。
3. 固定模型和训练 token 数，只改变词表大小，比较序列长度、embedding 参数、softmax 时间与验证损失。
4. 选一个架构改动，分别写出它对 FLOPs、显存容量、数据搬运、训练稳定性和推理缓存的影响。

11.4 拓展阅读

- CS336 Spring 2026: <https://cs336.stanford.edu/>
- Lecture 1 可执行课件：
https://github.com/stanford-cs336/lectures/blob/main/lecture_01.py
- Assignment 1 (Basics): <https://github.com/stanford-cs336/assignment1-basics>
- Neural Machine Translation of Rare Words with Subword Units: <https://arxiv.org/abs/1508.07909>
- The Bitter Lesson: <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>
- Let's build the GPT Tokenizer (Karpathy): <https://www.youtube.com/watch?v=zduSFxRajkE>

11.5 本章小结

课程全景和 Tokenization 共同展示了 CS336 的方法论：先明确资源与不变量，再实现、测量、比较。后续架构、系统、数据与对齐章节都将重复这套闭环，只是约束和瓶颈不同。

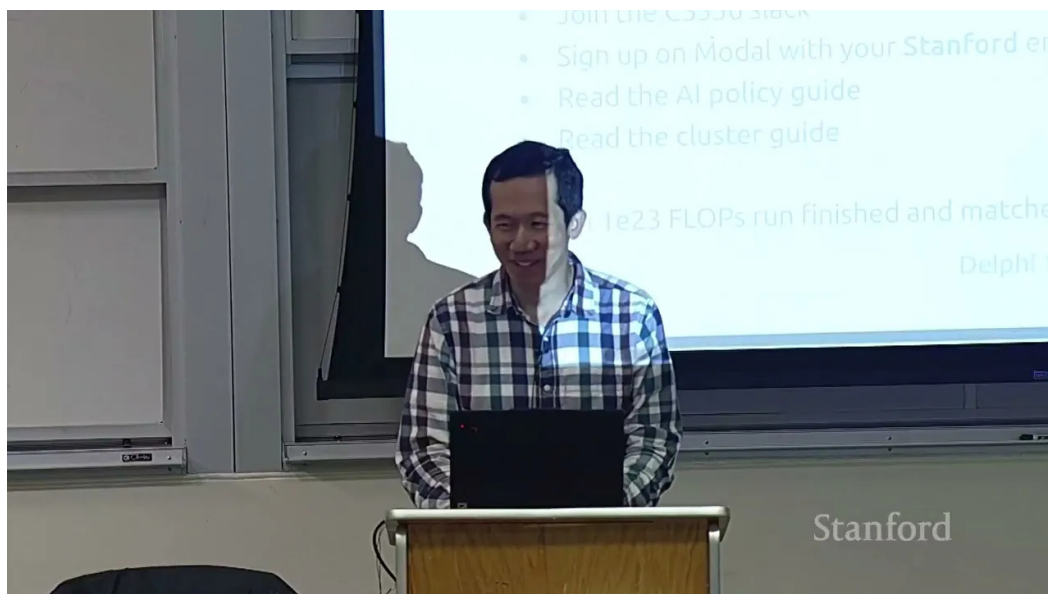
课程笔记

PyTorch、资源核算与算术强度

Stanford CS336 Spring 2026 · Lecture 2 高密度中文讲义

Percy Liang (讲授) / lecture-to-notes (整理)

2026 年 7 月 11 日



视频作者 / 频道: Stanford Online

发布日期: 2026-04-14

视频时长: 01:17:24

视频链接: <https://www.youtube.com/watch?v=kuYAsz7zspQ>

课程主页: <https://cs336.stanford.edu/>

官方可执行课件: `lecture_02.py`

目录

1 资源核算：先把训练问题变成数量级	3
1.1 70B 参数、15T token、1024 张 H100 需要多久	3
1.2 8 张 80GB H100 最多能装多少参数	4
1.3 本章小结	5
2 张量是资源核算的统一对象	5
2.1 秩、形状与语义维度	5
2.2 显存等于元素数乘元素宽度	5
2.3 本章小结	6
3 数值格式：容量、范围与分辨率的三角折中	6
3.1 FP16：省一半显存，但小数可能直接归零	7
3.2 BF16：保留 FP32 的范围，牺牲分辨率	7
3.3 混合精度：不同状态承担不同数值责任	8
3.4 FP8 与 NVFP4：位宽继续下降后必须引入尺度管理	9
3.5 本章小结	10
4 设备放置与具名张量运算	10
4.1 CPU 与 GPU 之间的移动也是成本	10
4.2 Einsum：把维度语义写进运算	11
4.3 Reduce 与 Rearrange：把形状变换变成类型检查	12
4.4 本章小结	13
5 FLOPs：从矩阵形状推导计算量	13
5.1 矩阵乘法为何是 2BDK	14
5.2 其他算子通常不是主导 FLOPs，却可能主导时间	15
5.3 本章小结	15
6 正确基准测试与 MFU	15
6.1 GPU 异步执行为何会让计时失真	15
6.2 MFU：理论峰值到底兑现了多少	16
6.3 本章小结	17
7 算术强度：计算与数据搬运谁更慢	17
7.1 ReLU：FLOPs 很少，时间却由带宽决定	18

7.2	工作负载强度与硬件临界强度	19
7.3	本章小结	21
8	Roofline: 矩阵乘法为何适合训练, 矩阵向量为何拖慢推理	21
8.1	几类操作的强度对比	21
8.2	训练与自回归推理的结构差异	22
8.3	本章小结	22
9	Autograd 与反向传播的计算账本	22
9.1	Autograd 记录什么	23
9.2	单层反向为何是前向的两倍	24
9.3	全网络的 6BP 近似	25
9.4	本章小结	26
10	训练显存: 静态状态与随 batch 增长的激活	26
10.1	生命周期决定峰值, 而不是简单总和	27
10.2	本章小结	27
11	用计算换显存: 梯度累积与激活检查点	28
11.1	梯度累积: 把一个大 batch 切成多个微批	28
11.2	激活检查点: 前向少存, 反向重算	28
11.3	检查点频率的复杂度折中	29
11.4	本章小结	30
12	总结与延伸	31
12.1	讲者总结	31
12.2	结构化提炼: 四张表贯穿后续课程	31
12.3	建议练习	32
12.4	拓展阅读	32
12.5	本章小结	32

1 资源核算：先把训练问题变成数量级

本讲的主题不是某个神奇的新模型，而是一种工程习惯：在开始实现之前，先把计算量、显存容量、数据搬运和墙钟时间写成可检查的数量级。固定资源下的目标是训练出最好的模型；要讨论“最好”，就必须先知道一个计算会消耗多少资源。

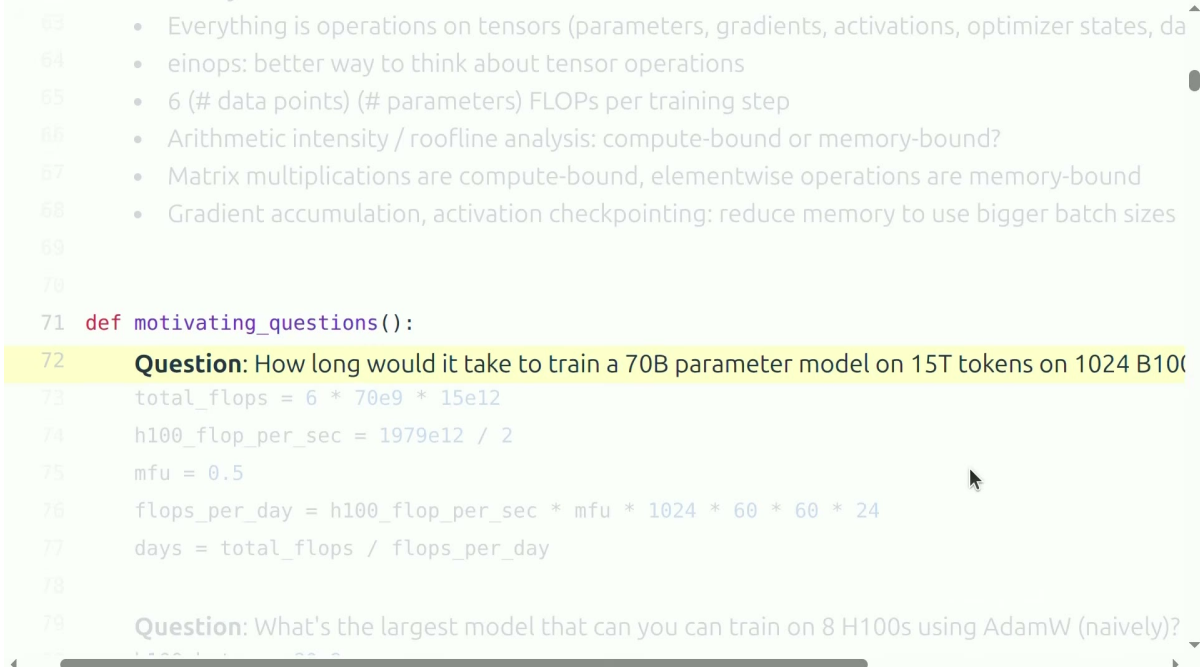


图 1：本讲的出发点：给定固定计算与显存预算，先回答训练需要多久、模型能装多大。¹

1.1 70B 参数、15T token、1024 张 H100 需要多久

对以稠密矩阵乘法为主、上下文不太长的 Transformer，训练总计算量常用

$$C_{\text{train}} \approx 6PT$$

估算。若 $P = 70 \times 10^9$ 、 $T = 15 \times 10^{12}$ ，则

$$C_{\text{train}} \approx 6.3 \times 10^{24} \text{ FLOPs.}$$

其中：

- C_{train} ：完整训练所执行的浮点运算总数；
- P ：模型参数量；
- T ：训练中处理的 token 数；
- 系数 6：前向约 $2PT$ 、反向约 $4PT$ 的合计近似。

课件取 H100 稠密 BF16 峰值 $F_{\text{peak}} = 1979/2 \text{ TFLOP/s}$ 、模型 FLOPs 利用率 $u = 0.5$ 、GPU 数 $G = 1024$ ，于是每天的有效计算量为

$$C_{\text{day}} = F_{\text{peak}} uG (86400) \approx 4.377 \times 10^{22} \text{ FLOPs/day,}$$

¹视频画面时间区间：00:02:00–00:02:15。

训练时间约为

$$D_{\text{train}} = \frac{C_{\text{train}}}{C_{\text{day}}} \approx 143.9 \text{ days.}$$

这里 F_{peak} 是单卡理论峰值, u 是实际有效吞吐与峰值之比, G 是卡数, D_{train} 是理想化训练天数。通信、故障恢复、评估和数据停顿若未反映在 u 中, 实际时间还会更长。

1.2 8 张 80GB H100 最多能装多少参数

若参数与梯度用 BF16, 每项各 2 bytes; AdamW 的一阶矩和二阶矩用 FP32, 各 4 bytes, 则不计激活时每个参数需要

$$b_p = 2 + 2 + 4 + 4 = 12 \text{ bytes.}$$

八张 80GB GPU 的理想参数上限为

$$P_{\text{max}} = \frac{8 \times 80 \times 10^9}{12} \approx 53.3 \times 10^9.$$

其中 b_p 是参数、梯度和优化器状态的合计字节数, P_{max} 是忽略激活、临时缓冲、碎片和通信副本后的理论上界。

```

71 def motivating_questions():
72     total_flops: How long would it take to train a 70B parameter model on 15T tokens on 1024 B100s?
73     flops_per_day = 6 * 70e9 * 15e12 * 24 * 60 * 60 = 4.377e+22
74     days = total_flops / flops_per_day = 1979e12 / 4.377e+22 = 143.9266
75     h100_bytes = 80,000,000,000
76     bytes_per_parameter = 12 # 2 for parameters, 2 for gradients, 4 for AdamW m1, 4 for AdamW m2
77     num_parameters = total_flops / (flops_per_day * bytes_per_parameter) = 53,333,333,333.333
78
79     Question: What's the largest model that you can train on 8 H100s using AdamW (naively)?
80     h100_bytes = 80e9
81     bytes_per_parameter = 2 + 2 + (4 + 4) # parameters, gradients, optimizer state
82     num_parameters = (h100_bytes * 8) / bytes_per_parameter
83     Caveat: activations are not accounted for (depends on batch size and sequence length).
84
85     This is a rough back-of-the-envelope calculation.
86
87

```

图 2: 草算结果: 约 144 天与约 53.3B 参数; 后者尚未计入激活, 因此只能视为上界。²

数量级估算的边界

草算的价值不是给出精确采购数字, 而是暴露主导项。把激活、序列长度、张量并行副本、主权重、通信缓冲和检查点加入后, 显存结论会改变; 把数据加载、评估与故障加入后, 墙钟结论也会改变。先有可解释的上界, 再逐层补项。

²视频画面时间区间: 00:03:45–00:04:00。

1.3 本章小结

训练预算可以先压缩成两条公式：计算近似 $6PT$ ，静态训练状态约 $12P$ bytes。它们不是最终答案，却能在几分钟内排除明显不可行的规模，并指出下一步必须细算的激活、通信和利用率。

2 张量是资源核算的统一对象

参数、数据、梯度、优化器状态和中间激活看似用途不同，在 PyTorch 中都由张量表示。一个张量至少要从四个角度理解：**形状**决定有多少元素，**dtype** 决定每个元素多少字节，**device** 决定存放位置，**运算图**决定它是否需要为反向传播保留信息。

```

80 h100_bytes = 80e9
81 bytes_per_parameter = 2 + 2 + (4 + 4) # parameters, gradients, optimizer state
82 num_parameters = (h100_bytes * 8) / bytes_per_parameter
83 Caveat: activations are not accounted for (depends on batch size and sequence length).
84
85 This is a rough back-of-the-envelope calculation.
86
87
88 def tensors_basics():
89     Tensors are the basic building block for storing everything:
90     • parameters, gradients, optimizer state, data, activations.
91
92     Example: parameters of the DeepSeek v3.2 model [DeepSeek-AI+ 2025]
93     [DeepSeek v3.2 model on Hugging Face]
94
95     x = torch.zeros(4) # vector
96     x = torch.zeros(4, 8) # matrix

```

图 3：张量承载训练的全部核心状态：数据、参数、梯度、优化器状态和激活。³

2.1 秩、形状与语义维度

向量是 rank-1 张量，矩阵是 rank-2 张量；Transformer 中常见 rank-4 形状

$$X \in \mathbb{R}^{B \times S \times H \times D_h}.$$

其中 B 是 batch size， S 是序列长度， H 是注意力头数， D_h 是每头维度。维度不仅是整数位置，还承担语义；若只记“-2”、“-1”，代码很容易在重排、广播或收缩时悄悄出错。

2.2 显存等于元素数乘元素宽度

连续张量的基础显存核算为

$$M(X) = \text{numel}(X) \times \text{element_size}(X).$$

³视频画面时间区间：00:05:15–00:05:30。

其中 $M(X)$ 是字节数， $\text{numel}(X)$ 是元素总数， $\text{element_size}(X)$ 是 dtype 的单元元素字节数。形状 (4,8) 的 FP32 张量有 32 个元素，每元素 4 bytes，共 128 bytes。

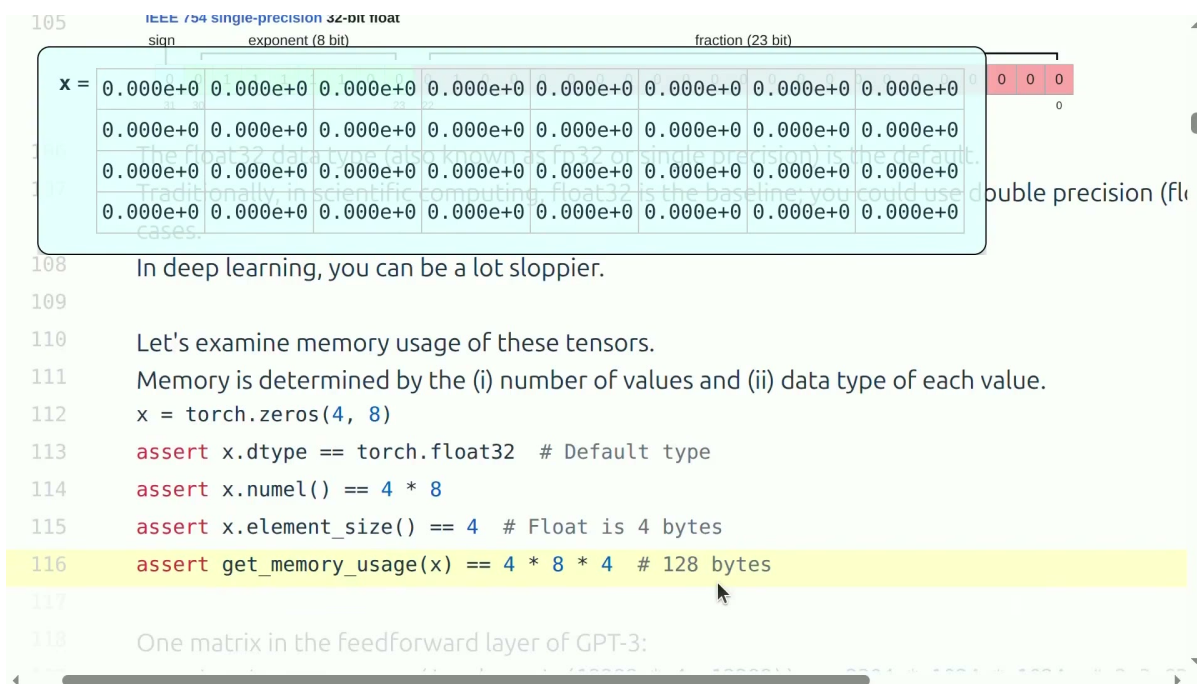


图 4: FP32 示例：显存由元素数与单元元素字节数共同决定。⁴

以 GPT-3 前馈层的一块权重为例，若矩阵形状是 $12288 \times (4 \cdot 12288)$ ，FP32 存储量约为

$$12288 \times 49152 \times 4 \approx 2.42 \times 10^9 \text{ bytes} \approx 2.25 \text{ GiB}.$$

这只是一个矩阵；大模型中大量权重、梯度与状态会同时驻留。因此，dtype 不是代码细节，而是模型能否放入设备的一级设计变量。

容量核算还要留出非张量开销

真实峰值通常高于“所有张量字节数之和”：算子会申请 workspace，分配器会保留缓存，张量布局可能产生临时 contiguous 副本，分布式库还需要通信桶。工程预算应保留余量，并用峰值显存测量验证静态表格。

2.3 本章小结

资源核算的统一入口是张量清单。先为每个张量写出语义形状、dtype、device 和生命周期，再计算元素数与字节数；这样显存问题才能从“感觉不够”变成可审计的账本。

3 数值格式：容量、范围与分辨率的三角折中

浮点数通常由符号位、指数位和尾数组成。指数位主要控制动态范围，尾数位主要控制相邻可表示数的分辨率。减少位宽能降低显存与带宽，并可能提高 Tensor Core 吞吐，但会增加下溢、上溢和舍入误差。

⁴视频画面时间区间：00:07:30–00:07:45。

格式	符号位	指数位	尾数位	典型字节数
FP32	1	8	23	4
FP16	1	5	10	2
BF16	1	8	7	2
FP8 E4M3	1	4	3	1
FP8 E5M2	1	5	2	1
NVFP4	约 4 bits/value, 另配分块尺度			0.5 (未计尺度)

3.1 FP16: 省一半显存，但小数可能直接归零

FP16 只有 5 个指数位，动态范围明显小于 FP32。课件将 10^{-8} 转为 FP16 后得到 0；若梯度或更新量在训练中发生这种下溢，优化器可能失去有效信号。

IEEE half-precision 16-bit float

sign (1 bit) | exponent (5 bits) | fraction (10 bits)

x = 0.000e+0

The float16 data type (also known as fp16 or half precision) cuts down the memory.

```
x = torch.zeros(4, 8, dtype=torch.float16)
assert x.element_size() == 2
```

However, the dynamic range (especially for small numbers) isn't great.

```
x = torch.tensor([1e-8], dtype=torch.float16)
assert x == 0 # Underflow!
```

If this happens when you train, you can get instability.

bfloat16

[Wikipedia]

bfloat16

sign (1 bit) | exponent (8 bits) | fraction (7 bits)

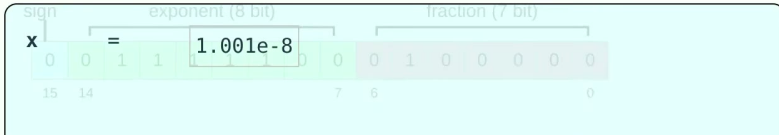
图 5: FP16 下溢示例: 10^{-8} 被量化为 0, 低精度会直接改变训练动力学。⁵

3.2 BF16: 保留 FP32 的范围，牺牲分辨率

BF16 与 FP32 都有 8 个指数位，因此动态范围接近；但 BF16 只有 7 个尾数位，数值间隔更粗。深度学习的随机性和冗余常能容忍较低分辨率，而上溢、下溢往往更危险，所以 BF16 成为训练中的常用折中。

⁵视频画面时间区间: 00:09:15–00:09:30。

bfloat16



Google Brain developed bfloat (brain floating point) in 2018 to address this issue.

bfloat16 uses the same memory as float16 but has the same dynamic range as float32!

The only catch is that the resolution is worse, but this matters less for deep learning.

```
x = torch.tensor([1e-8], dtype=torch.bfloat16)
```

```
assert x != 0 # No underflow!
```

Let's compare the dynamic ranges and memory usage of the different data types:

```
float32_info = torch.finfo(torch.float32)
float16_info = torch.finfo(torch.float16)
bfloat16_info = torch.finfo(torch.bfloat16)
```

Implications on training:

- Training with float32 works, but requires lots of memory.

图 6: BF16 用 8 位指数保留动态范围；代价是更少尾数位和更低分辨率。⁶

3.3 混合精度：不同状态承担不同数值责任

课程给出的常见配置是：参数、激活和梯度使用 BF16，优化器累计状态使用 FP32。前者占据大部分容量与带宽，适合降精度；后者跨很多训练步累积，舍入误差更敏感。PyTorch AMP 会在安全的算子（如矩阵乘法）中自动降精度，而对指数等敏感算子保留更高精度。

```
142 • Training with float32 works, but requires lots of memory.
143 • Training with fp8, float16 and even bfloat16 is risky, and you can get instability.
144
145 Mixed precision training [Micikevicius+ 2017]
146 • Use bf16 for parameters, activations, and gradients
147 • Use fp32 for optimizer states
148
149 Pytorch has an automatic mixed precision (AMP) library. [docs]
150 Tries to cast things into bf16 when safe (matmuls, not exp).
```

fp8

In 2022, fp8 was standardized, motivated by machine learning workloads [primer](#).

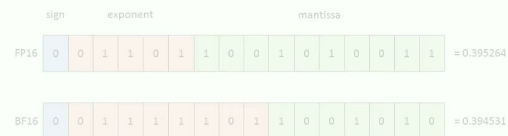


图 7: 混合精度分工：大体量训练张量用 BF16，长期累计的优化器状态用 FP32。⁷

⁶视频画面时间区间：00:10:45–00:11:00。

3.4 FP8 与 NVFP4：位宽继续下降后必须引入尺度管理

H100 支持 FP8 E4M3 与 E5M2：前者尾数更多，后者范围更大。它们的选择取决于张量分布与算子位置，不能简单把所有 BF16 张量统一转换为 FP8。

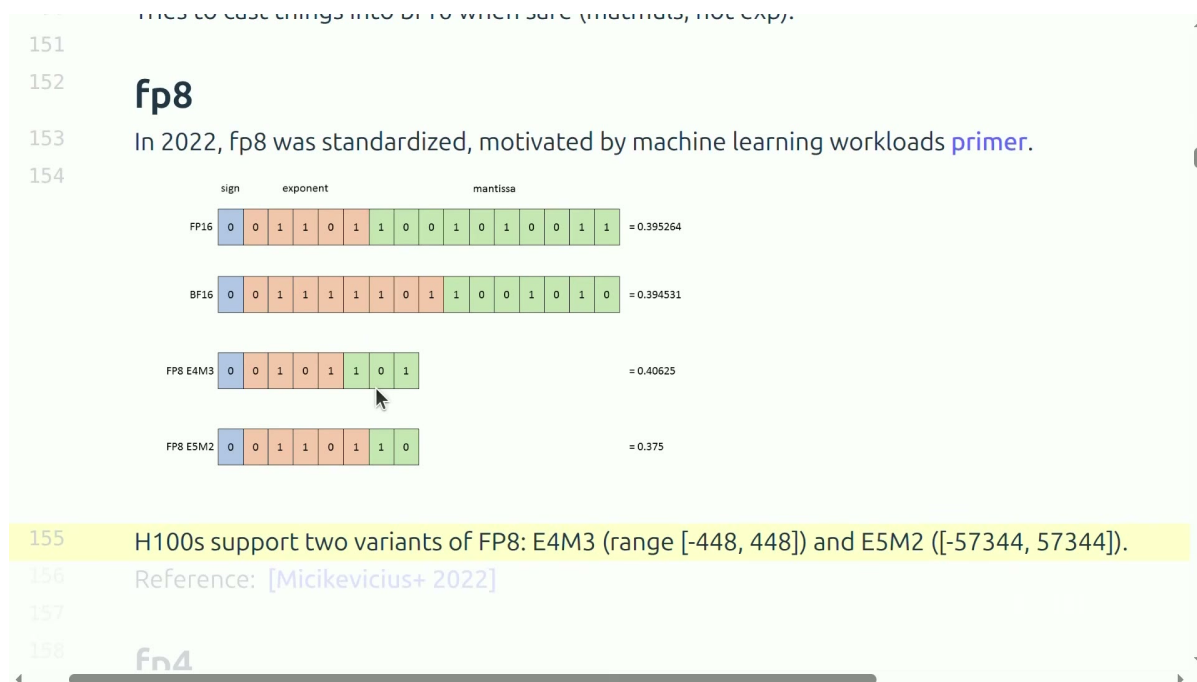


图 8：FP8 两种格式在指数位和尾数位之间重新分配预算。⁸

NVFP4 每个值只有 4 bits，可表示的基础离散值很少，因此按 block 额外存储 scale factor。块内数共享尺度：扩大了有效范围，却也让相邻值的缩放耦合。低位宽的系统收益依赖硬件内核、分块策略、校准和误差控制，而不仅是文件变小。

⁷视频画面时间区间：00:12:45–00:13:00。

⁸视频画面时间区间：00:13:30–00:13:45。

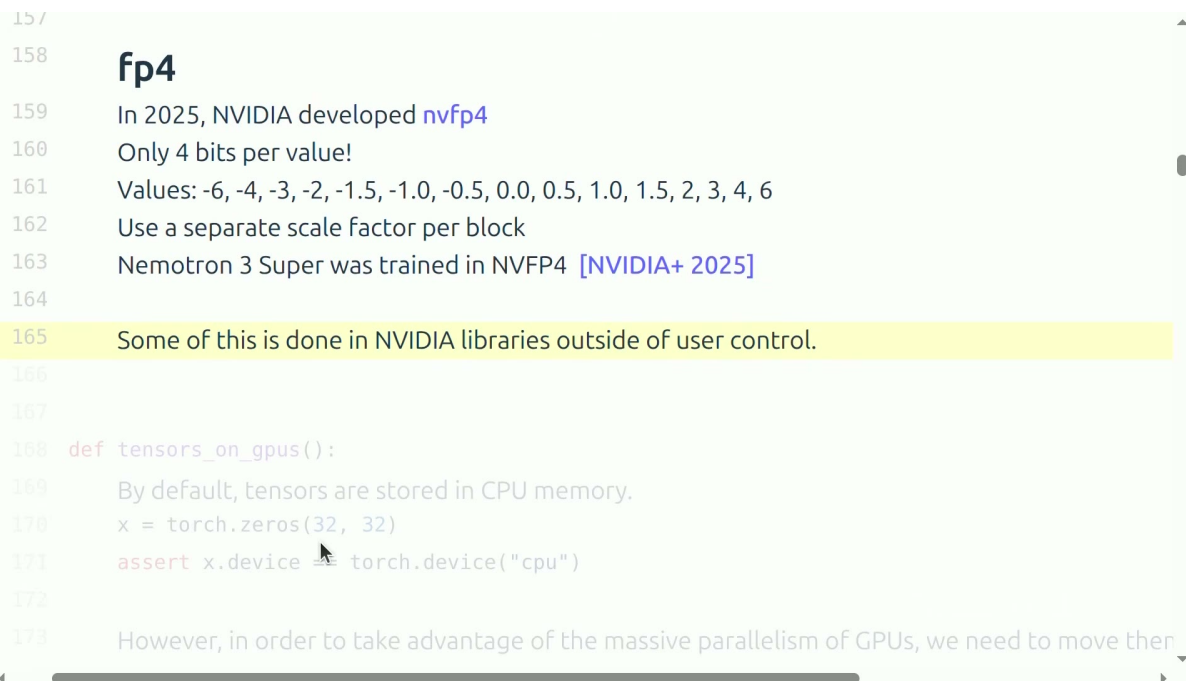


图 9: NVFP4 通过离散 4-bit 值与分块尺度共同表示更宽范围。⁹

位宽选择必须同时验收三项

一是质量与稳定性，二是峰值显存和数据搬运，三是实际 kernel 吞吐。若硬件没有对应低精度路径，或者频繁量化、反量化与尺度归约占据时间，理论上的位宽优势不会自动变成墙钟优势。

3.5 本章小结

FP16、BF16、FP8 与 FP4 不是一条单调“越低越好”的阶梯。指数位控制范围、尾数位控制分辨率、分块尺度补偿超低位宽；混合精度的本质是让每类状态使用刚好足够的数值预算。

4 设备放置与具名张量运算

4.1 CPU 与 GPU 之间的移动也是成本

PyTorch 默认把张量创建在 CPU 内存。要利用 GPU 的并行计算，输入必须进入 GPU DRAM；CPU RAM 与 GPU DRAM 之间通常通过 PCIe 或更快互连搬运。若每个小算子都来回移动数据，即使 GPU 算得很快，端到端性能仍会被传输吞没。

⁹视频画面时间区间：00:14:45–00:15:00。

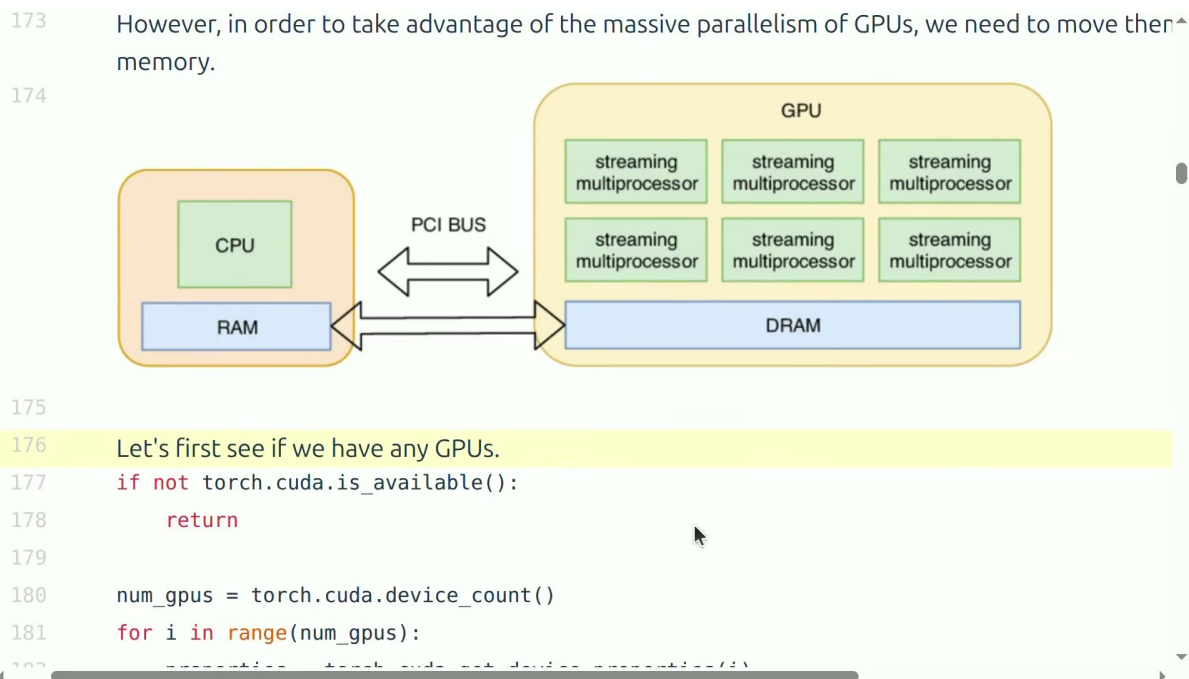


图 10: CPU 与 GPU 拥有独立内存; 跨 PCIe 搬运是显式资源成本。¹⁰

设备核算至少要回答: 张量在哪里创建、在哪个设备执行、何时复制、复制是否能与计算重叠。理想路径是尽早把批数据送入目标设备, 并让中间状态留在设备上, 直到必须输出。

4.2 Einsum: 把维度语义写进运算

传统写法常依赖位置索引:

```
1 z = x @ y.transpose(-2, -1)           # batch seq1 seq2
```

用 einops 的具名维度写法可明确表达收缩关系:

```
1 z = einsum(x, y,
2           "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2")
```

输出中未出现的 `hidden` 维会被求和; 共同保留的 `batch` 维则参与批量广播。省略号还可代表任意数量的前导批维。

¹⁰视频画面时间区间: 00:17:30–00:17:45。

```

232 Let's try a more complex example...
233
234 x = torch.ones(2, 3, 4) # batch seq1 hidden
235 y = torch.ones(2, 3, 4) # batch seq2 hidden
236
237 # Old way
238 z = x @ y.transpose(-2, -1) # batch seq1 seq2
239
240 # New (einops) way
241 z = einsum(x, y, "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2")
242 Dimensions that are not named in the output are summed over.
243
244 # Or can use `...` to represent broadcasting over any number of dimensions
245 z = einsum(x, y, "... seq1 hidden, ... seq2 hidden -> ... seq1 seq2")
246
247
248 def einops_reduce():
249     You can reduce a single tensor via some operation (e.g., sum, mean, max, min).

```

图 11: 具名 Einsum 明确写出批维、序列维和被收缩的隐藏维。¹¹

4.3 Reduce 与 Rearrange: 把形状变换变成类型检查

`reduce` 用名字指定被聚合的维度; `rearrange` 可以拆开或合并复合维度。例如把总隐藏维 d_{total} 拆成头数 H 与每头宽度 d_h , 逐头做线性变换, 再合并回去:

```

1 x = rearrange(x, "... (heads hidden1) -> ... heads hidden1", heads=H)
2 x = einsum(x, w, "... hidden1, hidden1 hidden2 -> ... hidden2")
3 x = rearrange(x, "... heads hidden2 -> ... (heads hidden2)")

```

¹¹ 视频画面时间区间: 00:22:30-00:22:45。

```

258
259 def einops_rearrange():
260     Sometimes, a dimension represents two dimensions
261     ...and you want to operate on one of them.
262
263     x = torch.ones(3, 8) # seq total_hidden
264     ...where total_hidden is a flattened representation of heads * hidden1
265     w = torch.ones(4, 4) # hidden1 hidden2
266
267     # Break up `total_hidden` into two dimensions (`heads` and `hidden1`
268     x = rearrange(x, "... (heads hidden1) -> ... heads hidden1", heads=2)
269
270     # Perform the transformation by `w`
271     x = einsum(x, w, "... hidden1, hidden1 hidden2 -> ... hidden2")
272
273     # Combine `heads` and `hidden2` back together
274     x = rearrange(x, "... heads hidden2 -> ... (heads hidden2)")

```

图 12: Rearrange 将扁平隐藏维拆成多头结构，运算后再合并。¹²

具名维度不是语法糖而是局部规范

当形状复杂时，维度名相当于轻量类型系统：它降低轴顺序错误，也让 FLOPs 和显存公式能直接对应代码。推荐在实现旁写出每个张量的语义形状，并在边界处用断言验证。

4.4 本章小结

张量不仅有形状，还驻留在具体设备并经历具体搬运。Einsum、Reduce 和 Rearrange 把维度语义暴露在代码中，使正确性检查、FLOPs 计数和通信分析共享同一套符号。

5 FLOPs: 从矩阵形状推导计算量

FLOP 表示一次浮点加法或乘法；FLOP/s 表示硬件每秒能完成多少浮点运算。二者读音相同却含义不同：前者是工作量，后者是速度。硬件表中的 FLOP/s 还依赖 dtype、稀疏假设和矩阵形状，不能脱离条件比较。

¹²视频画面时间区间：00:26:00–00:26:15。

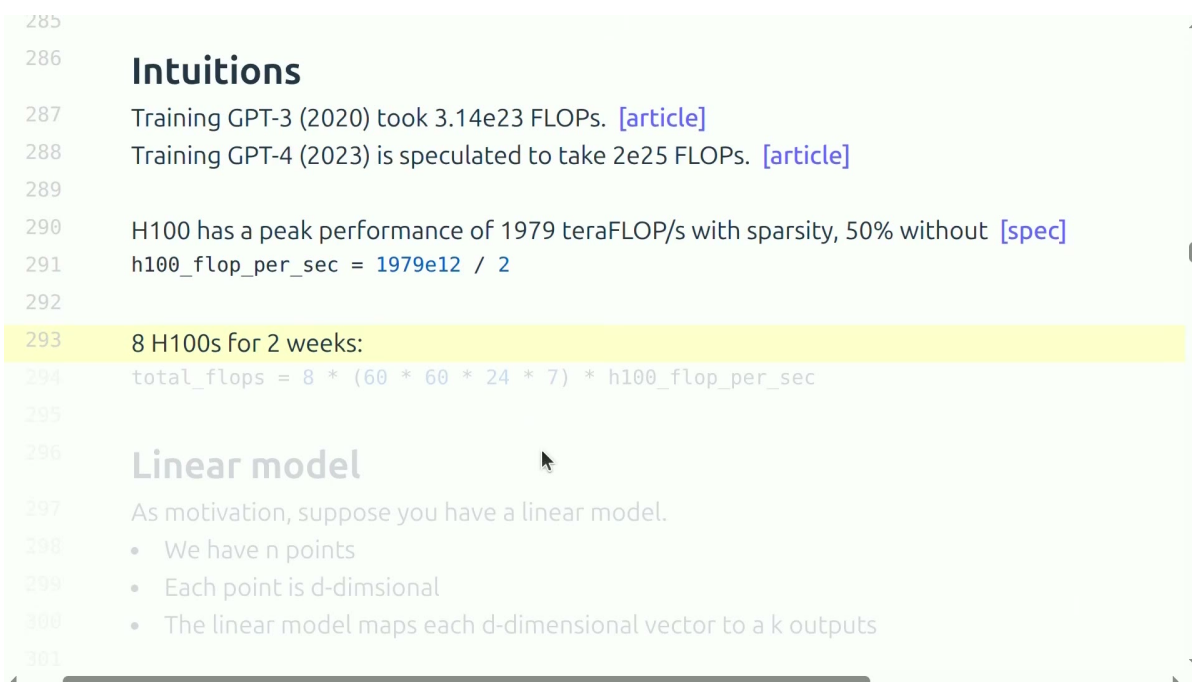


图 13: 训练总 FLOPs 与硬件峰值 FLOP/s 是两个量; H100 稠密值需去掉稀疏加速假设。¹³

5.1 矩阵乘法为何是 2BDK

令

$$X \in \mathbb{R}^{B \times D}, \quad W \in \mathbb{R}^{D \times K}, \quad Y = XW.$$

每个输出 Y_{bk} 是长度 D 的点积, 包含约 D 次乘法与 D 次加法; 共有 BK 个输出, 因此

$$C_{\text{matmul}} \approx 2BDK.$$

其中 B 是数据点或 token 数, D 是输入维度, K 是输出维度, C_{matmul} 是浮点运算数。严格来说每个点积有 $D - 1$ 次加法, 但大形状下用 $2D$ 近似足够。

¹³视频画面时间区间: 00:30:00–00:30:15。


```

02     if torch.cuda.is_available():
03         B = 16384 # Number of points
04         D = 32768 # Dimension
05         K = 8192  # Number of outputs
06     else:
07         B = 1024
08         D = 256
09         K = 64
10
11     x = torch.ones(B, D, device=cuda_if_available())
12     w = torch.randn(D, K, device=cuda_if_available())
13     y = x @ w
14     We have one multiplication (x[i][j] * w[j][k]) and one addition per (i, j, k) triple.
15     actual_num_flops = 2 * B * D * K
16
17     FLOPs of other operations
18     - Elementwise operation on a m x n matrix requires O(m * n) FLOPs

```

图 14: 矩阵乘法中每个 (b, d, k) 三元组对应一次乘法和一次加法。¹⁴

5.2 其他算子通常不是主导 FLOPs，却可能主导时间

对 $B \times K$ 张量做逐元素激活通常是 $O(BK)$ ；两个同形矩阵相加也是 $O(BK)$ 。与 $O(BDK)$ 的大矩阵乘法相比，它们的 FLOPs 很少。但逐元素算子对每个字节只做少量工作，常受内存带宽限制。由此得到重要区分：**少 FLOPs 不代表少时间**。

统一计数口径

不同资料有时把 fused multiply-add 计作一次指令，有时计作两个 FLOPs。训练核算通常按一乘一加等于 2 FLOPs。比较论文、硬件表和 profiler 时，先确认口径、稀疏性和 dtype。

5.3 本章小结

计算量可以从张量形状机械推导：矩阵乘法约为 $2BDK$ ，逐元素运算约为元素数。FLOPs 是必要账本，但不能单独预测墙钟时间，因为访存和硬件利用率尚未进入公式。

6 正确基准测试与 MFU

6.1 GPU 异步执行为何会让计时失真

CPU 发起 CUDA kernel 后通常立即返回，GPU 在后台执行。如果只用普通墙钟包围一次调用，测到的可能只是 launch 时间。正确的微基准要在计时区间前后同步 GPU，先预热，再重复多次并报告稳定统计量。

¹⁴视频画面时间区间：00:32:00–00:32:15。

```

353
354 def time_matmul(a: torch.Tensor, b: torch.Tensor) -> float:
355     """Return the number of seconds required to perform `a @ b`."""
356
357     # Wait until previous CUDA threads are done
358     if torch.cuda.is_available():
359         torch.cuda.synchronize()
360
361     def run():
362         # Perform the operation
363         a @ b
364
365     # Wait until CUDA threads are done
366     if torch.cuda.is_available():
367         torch.cuda.synchronize()
368
369     # Time the operation `num_trials` times
370     num_trials = 5

```

图 15: CUDA 微基准在执行前后同步，避免把异步 launch 误当成完整执行时间。¹⁵

若已知操作量 C 与实测时间 t ，有效吞吐为

$$F_{\text{actual}} = \frac{C}{t}.$$

其中 F_{actual} 是实测 FLOP/s， C 是已执行 FLOPs， t 是同步后的秒数。

6.2 MFU：理论峰值到底兑现了多少

Model FLOPs Utilization 定义为

$$\text{MFU} = \frac{F_{\text{actual}}}{F_{\text{promised}}}.$$

其中 F_{promised} 是给定设备、dtype 和稠密性条件下的理论峰值。课程把 $\text{MFU} \geq 0.5$ 视为通常已经不错的结果；但它忽略通信与其他开销的细分，因此是总体现象指标，不是根因诊断。

¹⁵视频画面时间区间：00:35:30–00:35:45。

```

329 Let us time it!
330 actual_time = time_matmul(x, w)
331 actual_flop_per_sec = actual_num_flops / actual_time
332
333 Each GPU has a specification sheet that reports the peak performance.
334 • Example: [H100 spec]
335 Note that the FLOP/s depends heavily on the data type!
336 promised_flop_per_sec = get_promised_flop_per_sec(x.dtype)
337
338 Model FLOPs utilization (MFU)
339
340 Definition: (actual FLOP/s) / (promised FLOP/s) [ignore communication/overhead]
341 mfu = actual_flop_per_sec / promised_flop_per_sec
342 Usually, MFU of  $\geq 0.5$  is quite good (and will be higher if matmuls dominate)
343
344 Summary

```

图 16: MFU 将实测 FLOP/s 与对应 dtype 的理论峰值相除。¹⁶

MFU 低可能来自矩阵太小、形状不友好、访存受限、kernel launch 过多、同步、通信、数据加载或负载不均。定位时要把握端到端 MFU 逐层拆成算子时间、数据移动和空闲时间。

一个可复用的基准流程

固定形状与 dtype；预热直到缓存和编译稳定；在 GPU 前后同步；重复测量并报告中位数与分位数；同时记录 FLOPs、峰值显存和硬件型号；最后用 profiler 确认没有隐式复制或不期望的同步。

6.3 本章小结

GPU 基准测试首先要处理异步性。MFU 把计算量和墙钟时间连接起来，但它只是利用率结果；要解释低 MFU，必须继续分析计算与内存之间的瓶颈。

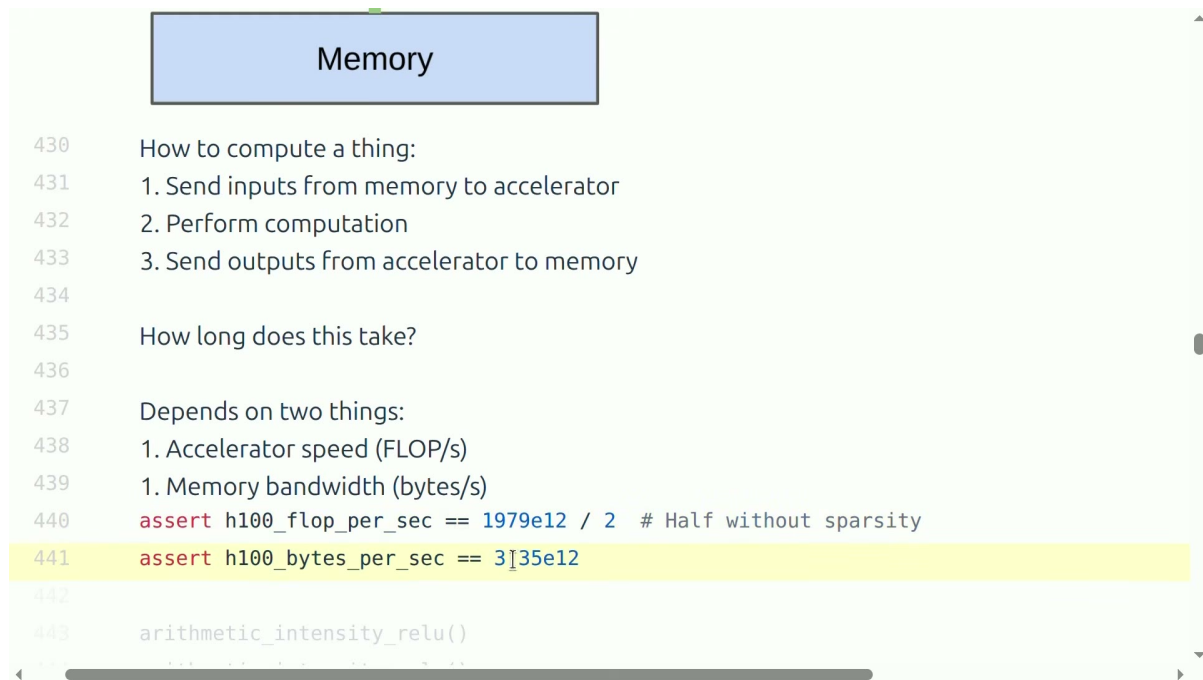
7 算术强度：计算与数据搬运谁更慢

一个 GPU 算子的抽象流程是：从显存读取输入，在计算单元执行操作，再把输出写回显存。理想情况下计算和搬运可以重叠，则时间下界为

$$t_{\min} = \max \left(\frac{Q}{B_w}, \frac{C}{F_{\text{peak}}} \right).$$

其中 Q 是搬运字节数， B_w 是显存带宽 (bytes/s)， C 是 FLOPs， F_{peak} 是峰值 FLOP/s。

¹⁶ 视频画面时间区间：00:37:15–00:37:30。



Memory

```

430 How to compute a thing:
431 1. Send inputs from memory to accelerator
432 2. Perform computation
433 3. Send outputs from accelerator to memory
434
435 How long does this take?
436
437 Depends on two things:
438 1. Accelerator speed (FLOP/s)
439 1. Memory bandwidth (bytes/s)
440 assert h100_flop_per_sec == 1979e12 / 2 # Half without sparsity
441 assert h100_bytes_per_sec == 3135e12
442
443 arithmetic_intensity_relu()

```

图 17: 算子时间由显存搬运与计算两条路径中更慢的一条决定。¹⁷

7.1 ReLU: FLOPs 很少，时间却由带宽决定

对 n 个 BF16 元素做 ReLU，读输入 $2n$ bytes、写输出 $2n$ bytes，若每元素近似一次比较，则

$$Q_{\text{ReLU}} = 4n, \text{quad}C_{\text{ReLU}} \approx n.$$

在课件的 H100 数值下，通信时间约 1.252×10^{-6} 秒，而计算时间约 1.060×10^{-9} 秒，前者大约高三个数量级。

¹⁷ 视频画面时间区间：00:41:45–00:42:00。

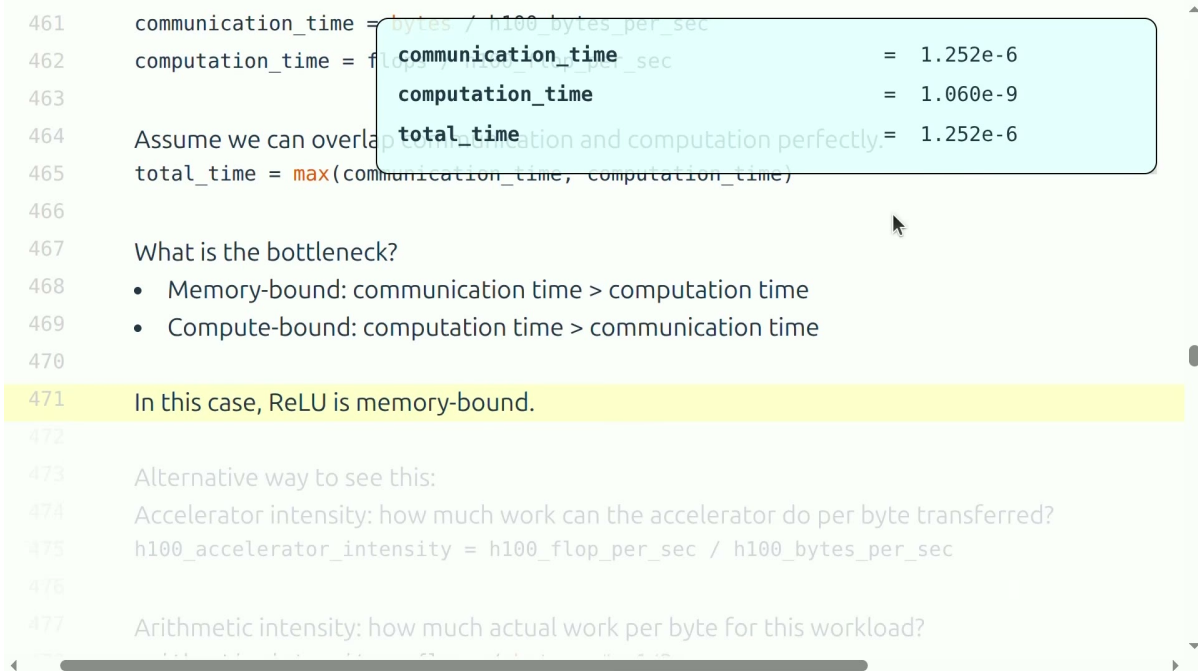


图 18: ReLU 示例中显存通信时间远大于计算时间，因此是 memory-bound。¹⁸

7.2 工作负载强度与硬件临界强度

定义工作负载的算术强度

$$I = \frac{C}{Q} \quad (\text{FLOPs/byte}),$$

以及硬件临界强度

$$I_* = \frac{F_{\text{peak}}}{B_w}.$$

其中 I 表示每搬一个字节做多少工作， I_* 表示硬件要吃满计算单元所需的最低工作密度。若 $I < I_*$ ，算子受带宽限制；若 $I > I_*$ ，算子受计算限制。

课件使用 H100 稠密 BF16 峰值 989.5 TFLOP/s 和显存带宽 3.35 TB/s，得到

$$I_* \approx 295.37 \text{ FLOPs/byte}.$$

ReLU 的 $I \approx n/(4n) = 0.25$ ，远低于临界值。

¹⁸ 视频画面时间区间：00:45:45–00:46:00。

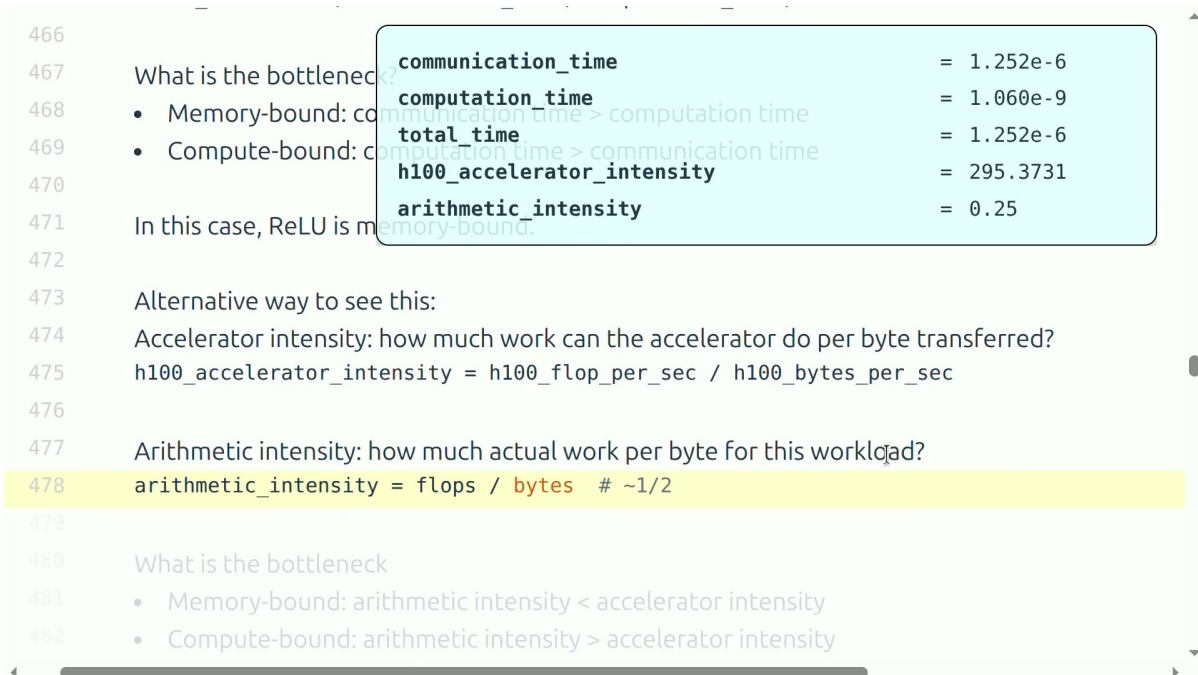


图 19: ReLU 的工作负载强度约 0.25 FLOPs/byte, 而 H100 临界强度约 295。¹⁹

GeLU 虽包含多项式和双曲函数近似, 课件按每元素约 20 FLOPs 估算, 因此 $I \approx 20n/(4n) = 5$ 。它比 ReLU 做更多计算, 却仍远低于 I_* ; 孤立执行时, 两者都主要在等显存。

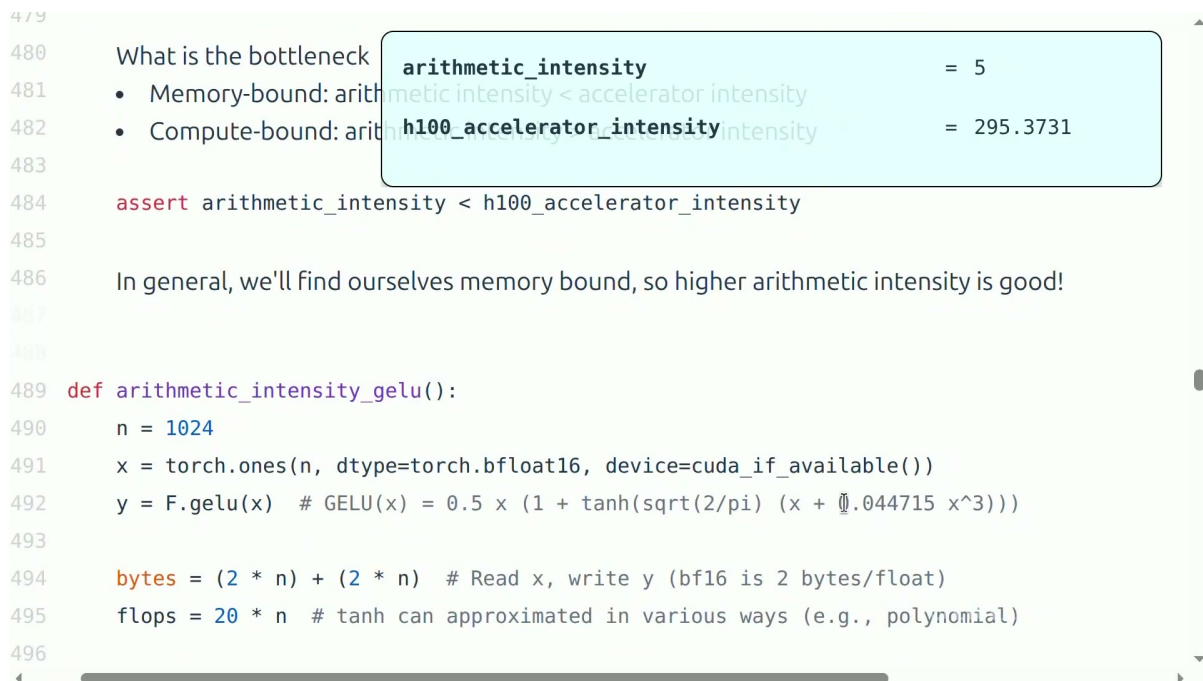


图 20: GeLU 的算术强度约为 5, 明显高于 ReLU, 但仍属于 memory-bound。²⁰

¹⁹视频画面时间区间: 00:47:00–00:47:15。

²⁰视频画面时间区间: 00:49:30–00:49:45。

融合为什么有效

算子融合不一定显著减少理论 FLOPs，却能让中间值留在寄存器或片上存储，减少 HBM 的读写字节 Q 。由于 $I = C/Q$ ，降低 Q 会提高算术强度，也可能减少 kernel launch 和同步开销。

7.3 本章小结

算术强度把 FLOPs 与字节数放进同一坐标。逐元素算子的计算便宜但搬运昂贵，往往 memory-bound；优化重点应是减少中间读写、融合操作并增加数据复用。

8 Roofline：矩阵乘法为何适合训练，矩阵向量为何拖慢推理

Roofline 模型给出的可实现吞吐上界是

$$F_{\text{realized}} \leq \min(F_{\text{peak}}, IB_w).$$

其中水平屋顶 F_{peak} 表示计算上限，斜线 IB_w 表示带宽上限，两者交点的横坐标就是 I_* 。

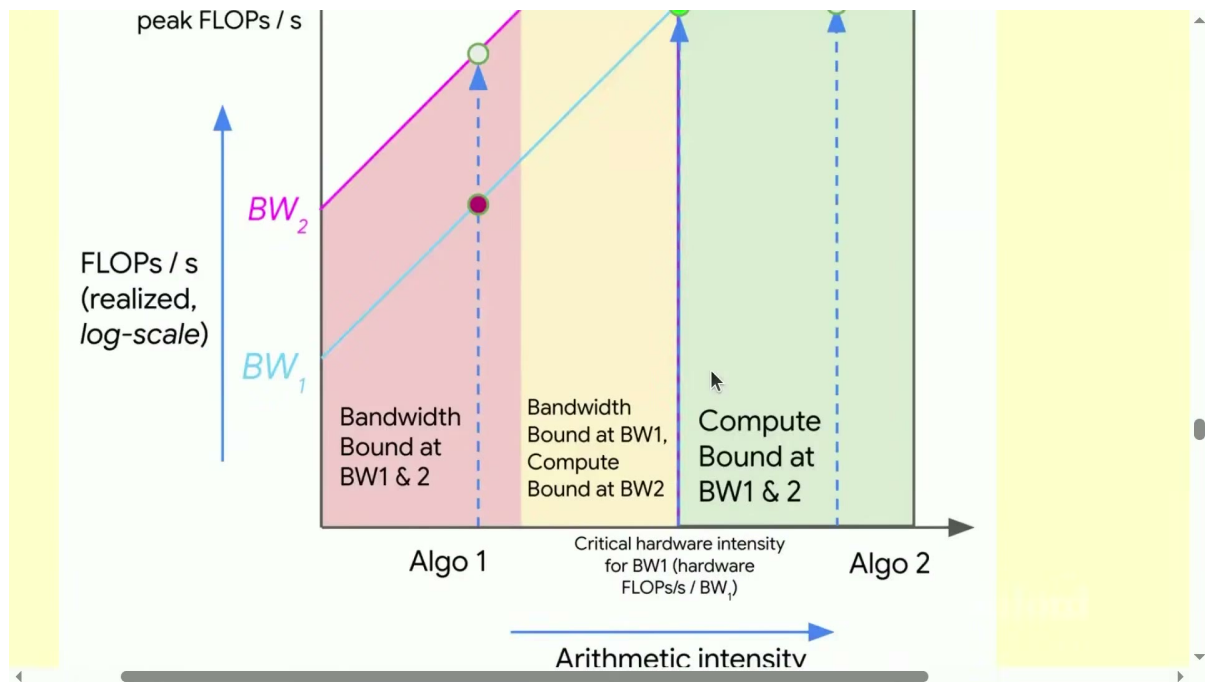


图 21：Roofline 将算术强度映射到可实现 FLOP/s，并划分带宽受限与计算受限区域。²¹

8.1 几类操作的强度对比

对 BF16 大形状忽略低阶项：

- 点积读取两个长度 n 的向量， $I \approx 1/2$ ；
- $n \times n$ 矩阵乘向量，权重只复用一次， $I \approx 1$ ；

²¹视频画面时间区间：00:55:45–00:56:00。

- $n \times n$ 矩阵乘矩阵，输入被反复复用，

$$I_{\text{matmul}} \approx \frac{2n^3}{6n^2} = \frac{n}{3}.$$

这里 n 是方阵边长。若 $n = 1024$ ，矩阵乘法强度约 341 FLOPs/byte，超过课件 H100 的临界值 295，因而有机会成为 compute-bound。

8.2 训练与自回归推理的结构差异

训练把许多 token 组成大矩阵，同一权重被 batch 和序列位置复用，容易形成高强度 GEMM。自回归 decode 每一步常接近矩阵向量乘法，每生成一个 token 都要读取大量权重，复用不足，所以更容易受显存带宽限制。增加 batch 可提高权重复用，但会牺牲单请求延迟并增加 KV cache 压力。

Roofline 还给出 MFU 的理想上界：

$$\text{MFU} \lesssim \min\left(1, \frac{I}{I_*}\right).$$

若算法位于斜坡区，再精巧的计算调度也无法越过带宽屋顶；应先提高复用或减少字节数。

低精度同时移动两条线

降低 dtype 往往减少字节数并提高峰值 FLOP/s，因此既改变工作负载强度 I ，也改变硬件临界强度 I_* 。不能只用“字节减半”预测最终加速，必须重新计算两者并测量对应 kernel。

8.3 本章小结

Roofline 把硬件与算法连接起来。大矩阵乘法通过数据复用提高算术强度，适合吃满训练算力；矩阵向量乘法复用弱，解释了 decode 常被带宽限制。

9 Autograd 与反向传播的计算账本

课程先用由线性层和 ReLU 组成的深网络建立统一例子：每层权重为 $D \times D$ ，输入和激活为 $B \times D$ ，共有 L 层，参数量近似 $P = LD^2$ 。

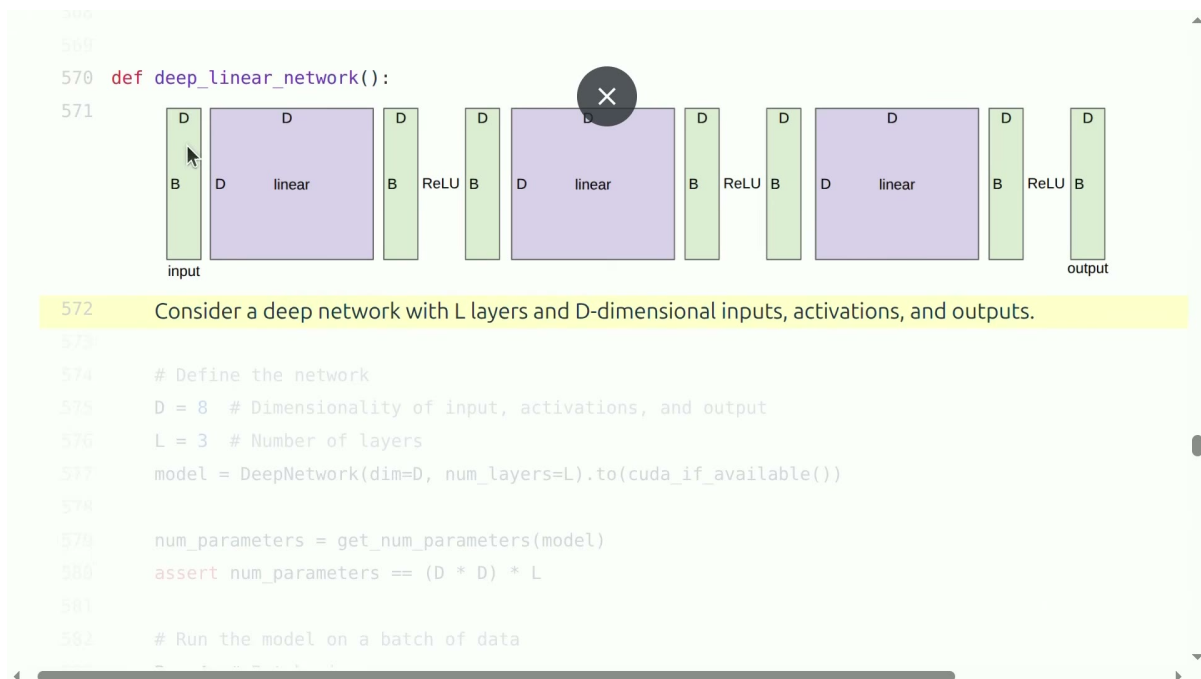


图 22: 运行示例: L 层 $D \times D$ 线性变换与 ReLU 串联。²²

9.1 Autograd 记录什么

若张量设置 `requires_grad=True`, PyTorch 在前向时记录产生结果的操作和依赖, 反向时根据链式法则累积梯度。对

$$\ell = \frac{1}{2}(x^\top w - 5)^2,$$

梯度为

$$\nabla_w \ell = (x^\top w - 5)x.$$

其中 x 是输入, w 是可训练权重, ℓ 是标量损失。课件取 $x = (1, 2, 3)$ 、 $w = (1, 1, 1)$, 残差为 1, 因此 $\nabla_w \ell = (1, 2, 3)$ 。

²²视频画面时间区间: 00:57:45–00:58:00。

```

347 • FLOP/s depends on hardware (B200 >> H100) and data type (bfloat16 >> float32)
348 • Model FLOPs utilization (MFU): (actual FLOP/s) / (promised FLOP/s)
349
350 def gradients_basics():
351     So far, we've constructed tensors (which correspond to either parameters or data) and passed them through
352     operations (forward).
353     Now, we're going to compute the gradient (backward).
354     As a simple example, let's consider the simple linear model:
355     y = 0.5 (x * w - 5)^2
356
357     Forward pass: compute loss
358     x = torch.tensor([1., 2, 3])
359     w = torch.tensor([1., 1, 1], requires_grad=True) # Want gradient
360     pred_y = x @ w
361     loss = 0.5 * (pred_y - 5).pow(2)
362
363     Backward pass: compute gradients
364     loss.backward()
365     """

```

图 23: 简单标量损失展示前向构图与 `loss.backward()` 的梯度计算。²³

9.2 单层反向为何是前向的两倍

对第二层

$$H_2 = H_1 W_2,$$

其中 $H_1, H_2 \in \mathbb{R}^{B \times D}$, $W_2 \in \mathbb{R}^{D \times D}$ 。前向需要

$$C_f \approx 2BD^2.$$

反向要计算两次同量级矩阵乘法:

$$\nabla_{H_1} \ell = \nabla_{H_2} \ell W_2^\top, \quad \nabla_{W_2} \ell = H_1^\top \nabla_{H_2} \ell,$$

所以

$$C_b \approx 4BD^2 \approx 2C_f.$$

这里 C_f 与 C_b 分别是单层前向和反向 FLOPs。

²³ 视频画面时间区间: 00:59:15–00:59:30。

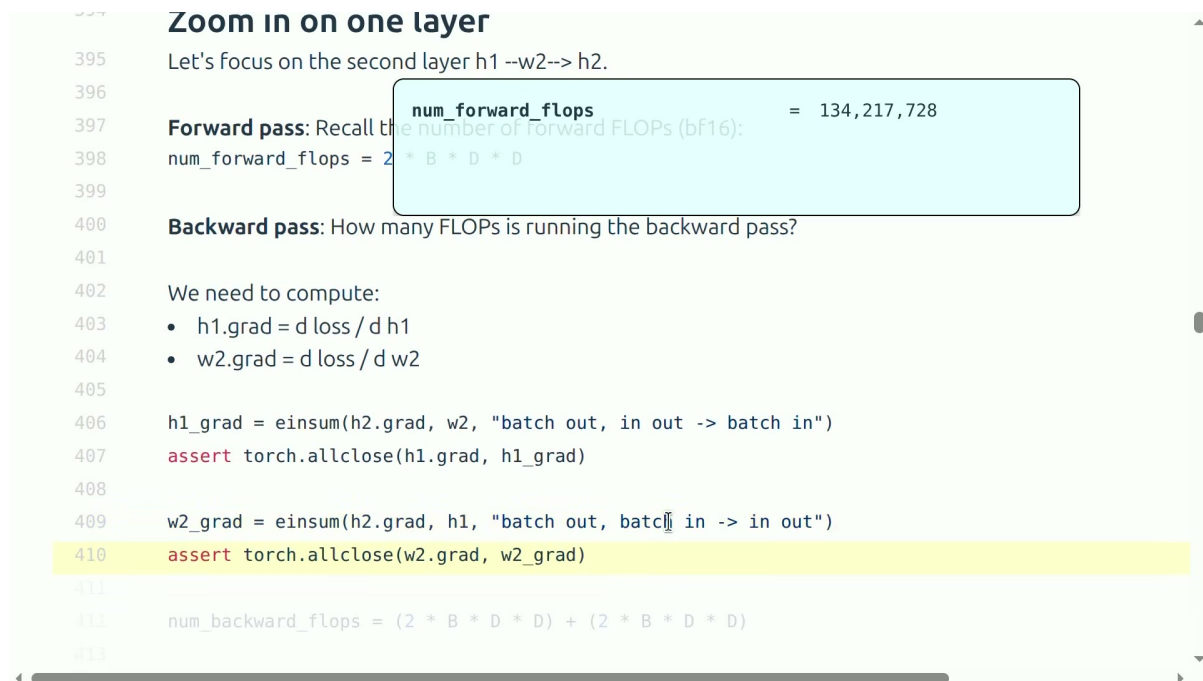


图 24: 单层反向需要计算输入梯度和权重梯度，两者都是矩阵乘法。²⁴

9.3 全网络的 6BP 近似

对所有层求和，若 $P \approx LD^2$ ，则

$$C_{\text{step}} \approx 2BP + 4BP = 6BP.$$

其中 B 表示该步处理的数据点或 token 数， P 是参数量。这一推导对 MLP 精确到主导项，对短上下文 Transformer 也是常用近似；长上下文下注意力的序列二次项会变得重要，需单独加入。

²⁴ 视频画面时间区间：01:04:30–01:04:45。

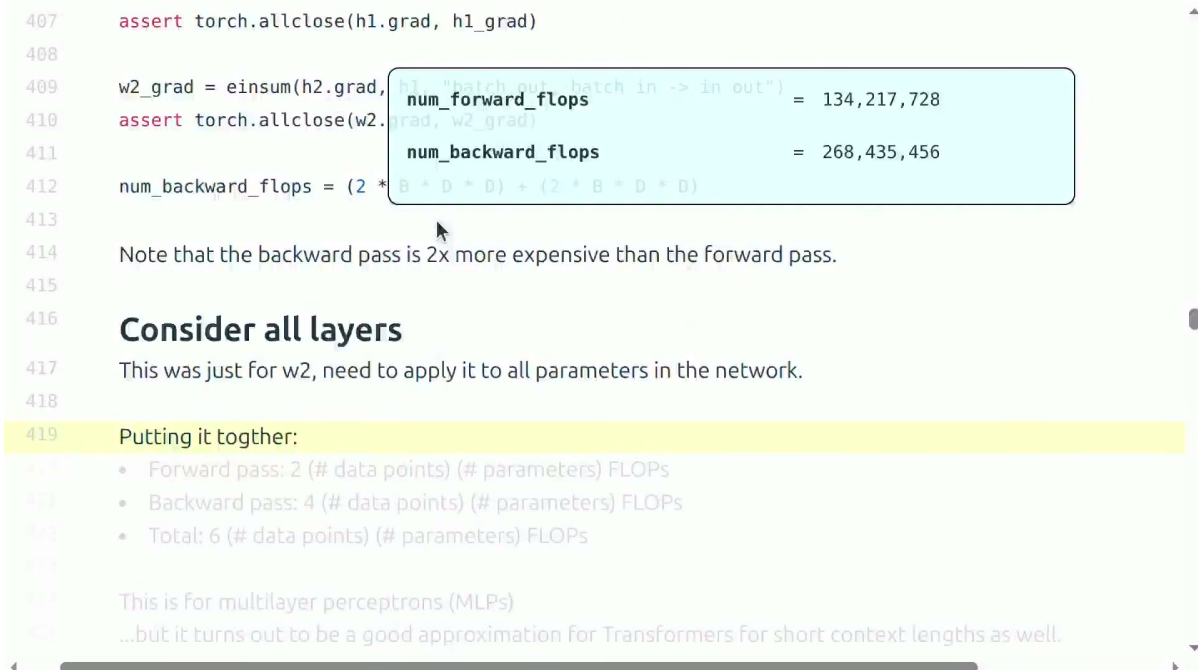


图 25: 前向约 2BP, 反向约 4BP, 训练一步合计约 6BP。²⁵

不要把 6PT 用到所有情形

稀疏 MoE 只有部分参数被激活; 权重共享、嵌入、长上下文注意力和非标准算子会改变比例。6PT 是先验, 不是测量。最终应从真实模型形状逐层计数, 并与 profiler 或训练日志交叉验证。

9.4 本章小结

Autograd 的计算并非黑箱: 线性层反向由输入梯度和权重梯度两次矩阵乘法组成, 因此约为前向两倍。累加到全网络后得到训练规模估算的核心 6BP。

10 训练显存: 静态状态与随 batch 增长的激活

在 BF16 参数与梯度、FP32 Adam 一阶/二阶矩的简化配置中, 显存主项为

$$M_{\text{train}} \approx 2P + 2P + 8P + M_{\text{act}} = 12P + M_{\text{act}}.$$

其中 $2P$ 是参数, 另一个 $2P$ 是梯度, $8P$ 是两个 FP32 优化器状态, M_{act} 是反向所需激活。若还保留 FP32 master weights, 需再加 $4P$; 不同优化器和分片策略也会改变系数。

²⁵ 视频画面时间区间: 01:05:45–01:06:00。


```

643
644 Memory
645
646 # Parameters
647 parameter_memory = 2 * (D * D * L) # (2 bytes for bf16)
648
649 # Activations
650 activation_memory = 2 * B * D * L # (2 bytes for bf16)
651
652 # Gradients
653 gradient_memory = 2 * parameter_memory # (2 bytes for bf16)
654
655 # Optimizer states
656 optimizer_state_memory = 4 * parameter_memory # (4 bytes for fp32)
657 Customary to use fp32 for stability (accumulating averages over powers over many steps)
658 4 bytes/parameter for storing second moments, Adam requires 8 bytes/parameter for storing first and second
659 moments
660
661 # Putting it all together

```

图 26: 训练显存账本: 参数、激活、梯度和优化器状态具有不同 dtype 与生命周期。²⁶

对课堂的 L 层、宽度 D 的简化网络, 若每层保存一个 $B \times D$ BF16 激活, 则

$$M_{\text{act}} \approx 2BDL \text{ bytes.}$$

其中 B 是 batch size, D 是激活宽度, L 是层数。真实 Transformer 还会保存 Q/K/V、注意力概率、归一化统计或算子专用中间量, 系数通常更大。

10.1 生命周期决定峰值, 而不是简单总和

参数和优化器状态长期驻留; 梯度在反向逐步产生; 激活在前向积累、反向逐层释放; 临时 workspace 只在算子执行期间存在。峰值显存必须按时间线计算, 不能把从未同时存在的临时量全部相加, 也不能漏掉在峰值时重叠的通信缓冲。

显存优化的三个方向

减少每元素字节数 (低精度), 减少同时驻留的元素数 (分片、微批、检查点), 或缩短生命周期 (及时释放、原地或融合)。每一种都可能以计算、通信、精度或实现复杂度为代价。

10.2 本章小结

训练显存至少包含静态的参数、梯度和优化器状态, 以及随 batch、序列长度和层数增长的激活。准确预算要同时考虑 dtype、张量形状和生命周期。

²⁶ 视频画面时间区间: 01:10:45–01:11:00。

11 用计算换显存：梯度累积与激活检查点

11.1 梯度累积：把一个大 batch 切成多个微批

若目标 batch 为 B ，将其分成 K 个 micro-batch，每个大小 $b = B/K$ 。依次前向和反向，梯度不清零；处理完 K 个微批后再更新参数。于是活跃激活大致从与 B 成正比降为与 b 成正比：

$$M_{\text{act, live}} \propto bDL = \frac{B}{K}DL.$$

其中 K 是累积步数。若损失原本按 batch 取平均，应保证每个微批的梯度缩放与整批等价。

```

728
729     # Update parameters
730     optimizer.step()
731     optimizer.zero_grad(set_to_none=True)
732
733
734 def gradient_accumulation():
735     Large batch sizes: improve training stability
736     However, activation memory scales with batch size, so might run out.
737     B = 1024
738     L = 16
739     D = 1024
740     activation_memory = B * L * D
741     Gradient accumulation:
742     • Compute gradient on micro batches
743     • Accumulate the gradients (don't zero it out)
744     • Every batch_size / micro_batch_size steps, update the parameters and zero out the gradients
745     micro_batch_size = 256
746     activation_memory = micro_batch_size * L * D
747

```

图 27：梯度累积在 micro-batch 上逐次反向，只在达到目标 batch 后更新并清零。²⁷

梯度累积不会减少整个大 batch 的主要 FLOPs，但会增加 kernel launch、数据循环与某些归一化差异；微批过小还会降低 GEMM 尺寸和 MFU。因此它解决的是容量约束，不保证吞吐不变。

11.2 激活检查点：前向少存，反向重算

普通训练为反向保存每层激活。Activation checkpointing（也称 gradient checkpointing 或 re-materialization）只保留部分边界激活；反向到某个区间时，从最近的检查点重新执行前向以恢复缺失中间量。

²⁷ 视频画面时间区间：01:13:00–01:13:15。

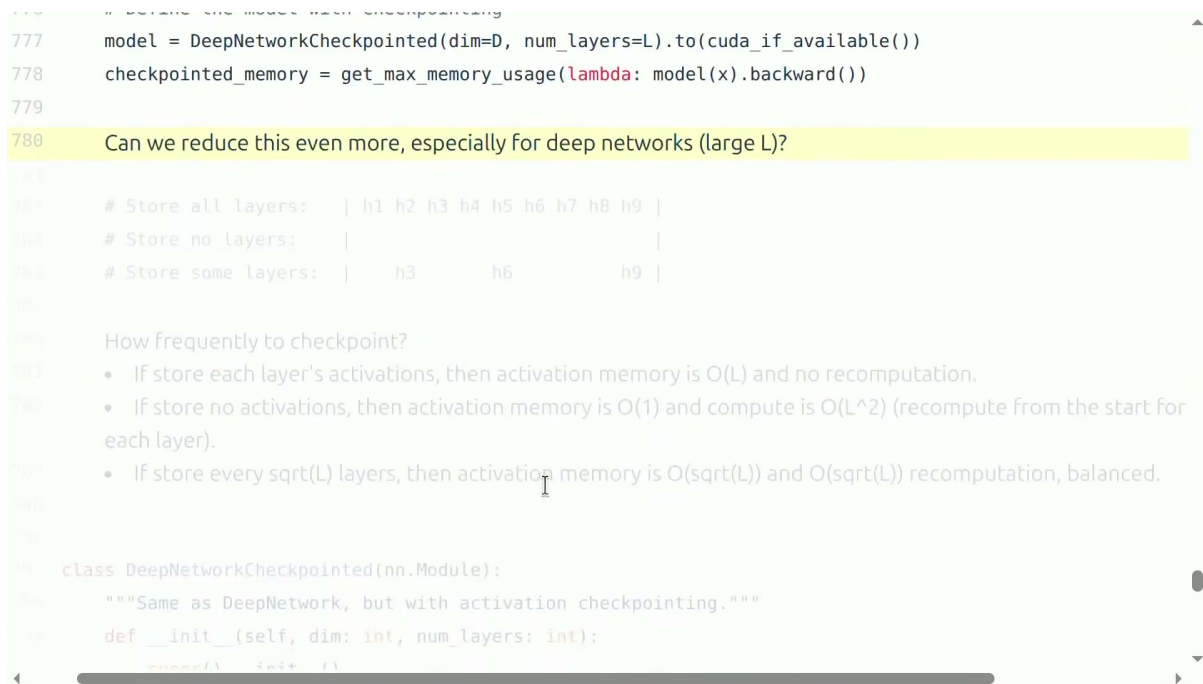


图 28: 检查点策略只保存部分层的激活, 并在反向阶段重算其余激活。²⁸

11.3 检查点频率的复杂度折中

把网络抽象成 L 层:

- 保存每层: 激活内存 $O(L)$, 无额外重算;
- 不保存中间层: 激活内存可到 $O(1)$, 但若每层都从头恢复, 计算可到 $O(L^2)$;
- 每隔约 $\sqrt{L}$ 层保存: 激活内存约 $O(\sqrt{L})$, 额外重算保持 $O(L)$ 量级。

²⁸ 视频画面时间区间: 01:14:30–01:14:45。

```

778 checkpointed_memory = get_max_memory_usage(lambda: model(x).backward())
779
780 Can we reduce this even more, especially for deep networks (large L)?
781
782 # Store all layers: | h1 h2 h3 h4 h5 h6 h7 h8 h9 |
783 # Store no layers: |                               |
784 # Store some layers: |   h3       h6       h9   |
785
786 How frequently to checkpoint?
787 • If store each layer's activations, then activation memory is  $O(L)$  and no recomputation.
788 • If store no activations, then activation memory is  $O(1)$  and compute is  $O(L^2)$  (recompute from the start for each layer).
789 • If store every  $\sqrt{L}$  layers, then activation memory is  $O(\sqrt{L})$  and  $O(\sqrt{L})$  recomputation, balanced.
790
791
792 class DeepNetworkCheckpointed(nn.Module):
793     """Same as DeepNetwork, but with activation checkpointing."""
794     def __init__(self, dim: int, num_layers: int):
795         super().__init__()
796         self.layers = nn.ModuleList([Block(dim) for i in range(num_layers)])

```

图 29: 从全存到全不存, 检查点频率决定激活显存与重计算开销。²⁹

实践中还要选择检查点边界: 把昂贵、低内存的算子放进重算区可能不划算; 把内存大、重算便宜的区段检查点化通常收益更好。应测量峰值显存、每步时间和数值一致性, 而不是只看理论复杂度。

显存不足时的排查顺序

先测量峰值发生在哪个阶段; 确认是否存在意外 FP32、隐式复制或过大临时量; 再调 micro-batch 与梯度累积; 随后对高激活区启用检查点; 最后才扩大并行或采用更复杂分片。每一步同时记录吞吐和收敛行为。

11.4 本章小结

梯度累积把大 batch 的激活分时处理, 检查点把部分激活从“存储”改为“重算”。两者都用额外执行开销换容量, 因此必须与 MFU、数值等价性和端到端吞吐一起验收。

²⁹ 视频画面时间区间: 01:15:45–01:16:00。

12 总结与延伸

```

59     gradient_accumulation()
60     activation_checkpointing()
61
62     Summary:
63     • Everything is operations on tensors (parameters, gradients, activations, optimizer states, data)
64     • einops: better way to think about tensor operations
65     • 6 (# data points) (# parameters) FLOPs per training step
66     • Arithmetic intensity / roofline analysis: compute-bound or memory-bound?
67     • Matrix multiplications are compute-bound, elementwise operations are memory-bound
68     • Gradient accumulation, activation checkpointing: reduce memory to use bigger batch sizes
69
70
71 def motivating_questions():
72     Question: How long would it take to train a 70B parameter model on 15T tokens on 1024 B100s?
73     total_flops = 6 * 70e9 * 15e12
74     h100_flop_per_sec = 1979e12 / 2
75     mfu = 0.5
76     flops_per_day = h100_flop_per_sec * mfu * 1024 * 60 * 60 * 24
77     days = total_flops / flops_per_day
78

```

图 30: 本讲总结：张量统一表示训练状态，6BP 估算计算，Roofline 判断瓶颈，累积与检查点降低显存。³⁰

12.1 讲者总结

本讲把语言模型训练还原成一组可核算的张量操作：

1. 所有核心状态都落在具有形状、dtype、device 和生命周期的张量上；
2. 具名维度让运算语义、FLOPs 公式和代码实现相互对应；
3. 稠密短上下文训练可用 6PT 估算总计算；
4. 算术强度与 Roofline 判断算子是 compute-bound 还是 memory-bound；
5. 大矩阵乘法通常计算受限，逐元素与矩阵向量操作通常带宽受限；
6. 梯度累积和激活检查点以时间或重计算换取更低峰值显存。

12.2 结构化提炼：四张表贯穿后续课程

对任何新模型或 kernel，可以维护四张最小账表：

1. **张量表**：名字、语义形状、dtype、device、生命周期、字节数；
2. **计算表**：每个算子的 FLOPs、调用次数和总计算量；
3. **搬运表**：HBM、片上存储和互连之间的字节数与复用；
4. **时间表**：理论下界、实测时间、MFU、瓶颈与异常同步。

³⁰ 视频画面时间区间：01:17:00–01:17:15。

这四张表让架构、系统和训练优化使用同一语言：改变注意力形状时能看到 FLOPs 和激活变化，改变 dtype 时能看到容量、带宽和峰值变化，改变并行策略时能看到通信和驻留副本变化。

12.3 建议练习

1. 复算 70B/15T/1024 H100 的 144 天结论，并分别将 MFU 改为 0.35 和 0.6，比较墙钟敏感性。
2. 为一个含 embedding、attention、MLP 和 AdamW 的小 Transformer 建立完整张量表，区分十进制 GB 与二进制 GiB。
3. 分别推导点积、矩阵向量和矩阵乘法的算术强度，并在不同 batch 下画 Roofline 位置。
4. 用 PyTorch 写同步正确的 matmul 基准，比较 FP32、BF16 与不同形状的 MFU；同时记录峰值显存。
5. 验证梯度累积与整批训练的梯度是否等价，并比较不同 micro-batch 对吞吐的影响。
6. 对深层网络尝试不同检查点间隔，画出峰值显存与每步时间的 Pareto 曲线。

12.4 拓展阅读

- Stanford CS336 Spring 2026: <https://cs336.stanford.edu/>
- Lecture 2 可执行课件: [lecture_02.py](#)
- Einops 教程: <https://einops.rocks/1-einops-basics/>
- Mixed Precision Training: <https://arxiv.org/abs/1710.03740>
- JAX Scaling Book Roofline: <https://jax-ml.github.io/scaling-book/roofline/>
- Transformer 显存核算: <https://erees.dev/transformer-memory/>

12.5 本章小结

资源核算的最终目的不是得到一串静态数字，而是建立可迭代的因果模型：形状决定元素与 FLOPs，dtype 决定字节和峰值，复用决定算术强度，硬件屋顶决定可实现吞吐，生命周期决定峰值显存。后续所有架构与系统优化，都可以回到这套账本验证。

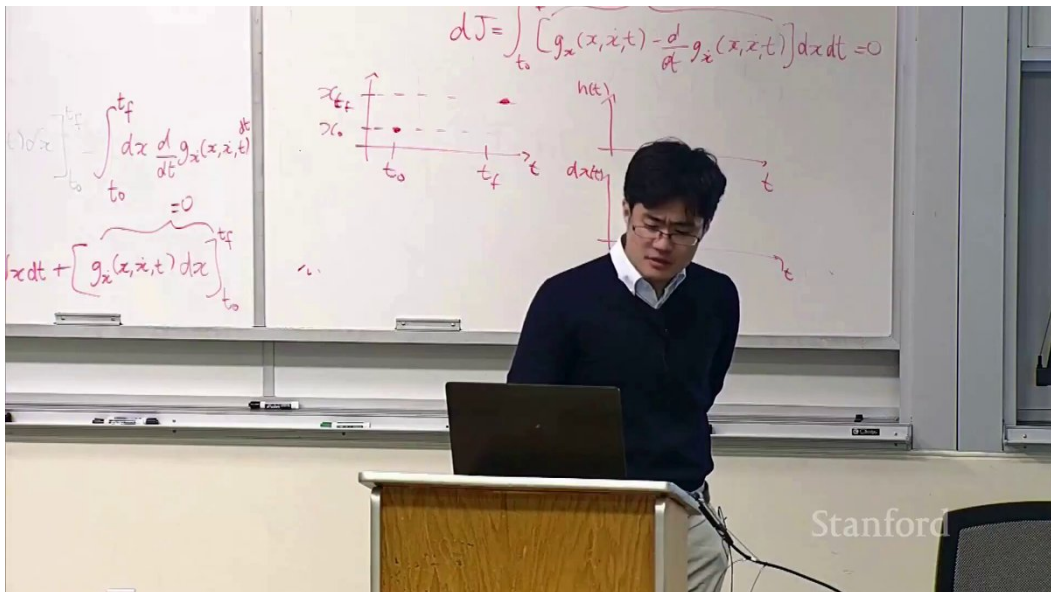
课程笔记

现代大语言模型架构与超参数

从共识到例外——Stanford CS336 2026 Lecture 3 高密度中文讲义

Tatsunori Hashimoto (讲授) / lecture-to-notes (整理)

2026 年 7 月 11 日



视频作者 / 频道: Stanford Online (原课程) / Vikas Gupta @vicky_grok (X 转发)

发布日期: 原课程视频: 2026-04-15; X 转发: 2026-07-10

视频时长: 01:29:03

X 视频链接: https://x.com/vicky_grok/status/2075594420163092606/video/1

课程主页: <https://cs336.stanford.edu/>

官方课件: Lecture 3: Architectures

目录

1 阅读指南与课程定位	2
1.1 如何使用这份讲义	2
1.2 本章小结	2
2 现代 Transformer 基线：哪些选择已接近共识	2
2.1 跨模型调查能告诉我们什么	3
2.2 一个实用的基线策略	4
2.3 本章小结	4
3 残差通路和归一化：让深网络可训练	4
3.1 Pre-Norm 与 Post-Norm 的根本差别	4
3.2 LayerNorm 与 RMSNorm	5
3.3 为什么 FLOPs 不能代表运行时间	6
3.4 本章小结	7
4 前馈网络、门控激活与并行块	7
4.1 从 ReLU / GELU 到 GLU 家族	7
4.2 串行块与并行块	8
4.3 本章小结	9
5 位置表示与 RoPE：把顺序写入注意力	9
5.1 RoPE 的核心目标：内积只感知相对距离	10
5.2 实现与外推时的常见陷阱	12
5.3 本章小结	12
6 尺寸超参数：宽度、深度、头数与词表	12
6.1 前馈层维度	13
6.2 注意力头：总维度比头数更关键	13
6.3 宽深比与扩展路径	14
6.4 词表大小是一项系统设计	15
6.5 本章小结	15
7 正则化与优化动力学：旧概念的新解释	16
7.1 Dropout 消退，权重衰减保留	16
7.2 权重衰减不只是“防过拟合”	16

7.3 本章小结	17
8 训练稳定性：控制 Softmax 的危险尺度	17
8.1 语言模型中有两处关键 Softmax	18
8.2 z-loss：约束输出归一化常数	18
8.3 QK-Norm：在注意力 Softmax 之前控幅	19
8.4 稳定性实验该怎样设计	20
8.5 本章小结	20
9 推理系统：Prefill、Decode 与 KV 缓存	20
9.1 Prefill：矩阵大、复用高、较偏计算受限	21
9.2 Decode：顺序依赖强、算术强度低	21
9.3 MQA：所有查询头共享一组 K/V	22
9.4 GQA：系统效率与表达能力的折中	23
9.5 本章小结	24
10 长上下文：稀疏、滑动窗口与混合注意力	24
10.1 从稀疏模式到滑动窗口	24
10.2 交错全局层与局部层	25
10.3 混合架构仍在快速演化	26
10.4 本章小结	27
11 总结与延伸	27
11.1 讲者总结：共识、例外与变化最快的边界	27
11.2 一张可执行的设计清单	27
11.3 进一步推导：从组件思维转向约束思维	28
11.4 建议练习	28
11.5 拓展阅读	29
11.6 本章小结	29

1 阅读指南与课程定位

这节课不是一份“照着抄就能得到最强模型”的配方，而是一张架构决策地图。讲者 Tatsunori Hashimoto 以当前主流稠密语言模型为样本，追问三个问题：哪些设计已经成为可靠默认值？哪些选择主要服务于 GPU 与推理系统？哪些新改动是在昂贵训练中为稳定性购买保险？

全课主线

架构选择同时受三类约束支配：**学习与泛化能力、训练 / 推理系统效率、数值稳定性**。只比较参数量或 FLOPs，往往无法预测真实吞吐、长上下文成本与训练失败风险。

本讲的经验来源有明确层级。最可靠的是亲手实现、训练并做受控消融；其次是跨模型调查表；最后才是从单个成功模型反推因果。表格能揭示“共识”，却不能自动证明某一行就是性能来源：成功模型常常一次改变多个因素，而且训练数据、优化器、预算与硬件也不同。

材料对齐说明

本讲义以用户提供的 X 视频为时间轴。为修复 X 自动字幕后半段的明显错乱，制作时将其与 Stanford Online 发布的同一场 2026 年 Lecture 3 官方录像进行画面、音频和时长核验，并把官方人工字幕整体对齐到 X 时间轴；所有插图仍取自 X 视频文件。图注中的时间可直接用于回看原链接。

1.1 如何使用这份讲义

建议先看每节的“判断框架”和“小结”，再回到公式与图表。模型设计时，不要把“现代模型常用”误读为“在任何规模、硬件和数据上都最优”；应从稳定默认值出发，只在存在明确瓶颈和可验证收益时偏离。

1.2 本章小结

本课讨论的不是组件清单，而是约束下的资源配置。读表格时要区分相关性与因果；做设计时要同时记录质量、吞吐、显存、训练稳定性和实现复杂度。

2 现代 Transformer 基线：哪些选择已接近共识

原始 Transformer 奠定了自注意力、前馈网络、残差连接与归一化的基本骨架，但现代自回归语言模型通常使用一套更接近 LLaMA 的基线：Pre-Norm、RMSNorm、旋转位置编码（RoPE）、SwiGLU、无偏置线性层，以及面向因果语言建模的掩码注意力。这些变化并非都来自同一种动机：有的改善梯度传播，有的减少内存访问，有的提升参数利用率，有的服务长上下文。

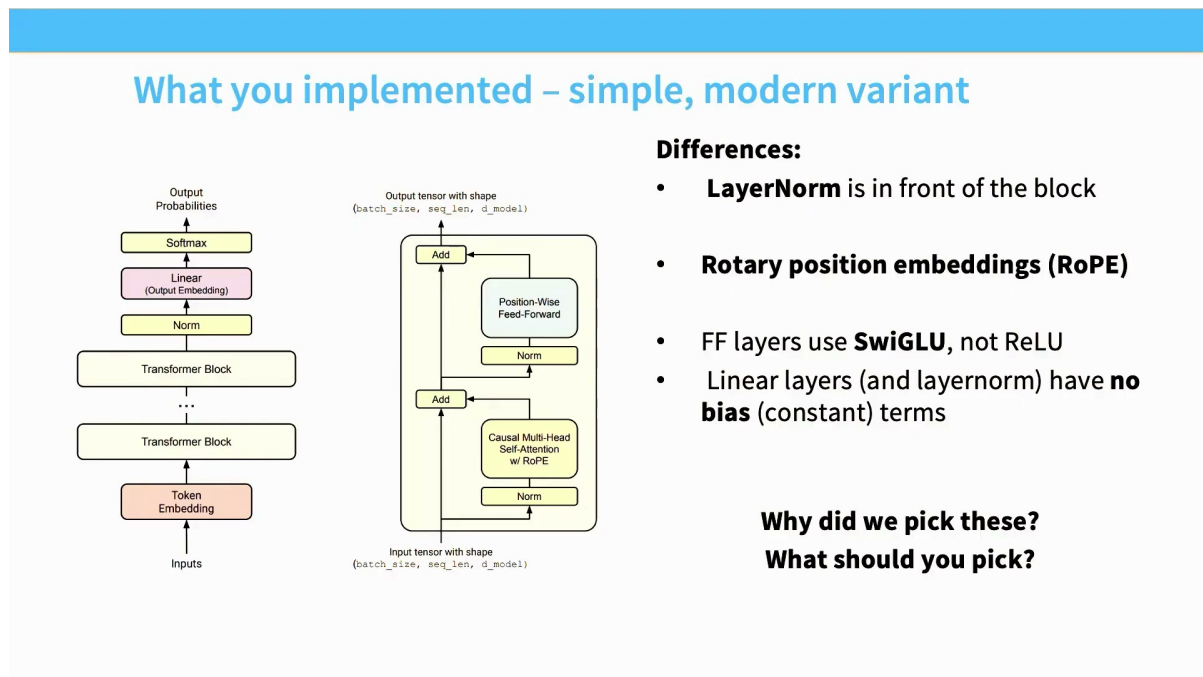


图 1: 课程作业采用的现代 Transformer 基线: Pre-Norm、RoPE、SwiGLU 与无偏置线性层。¹

2.1 跨模型调查能告诉我们什么

把 GPT 系列、PaLM、LLaMA、Gemma、Qwen 等模型放在同一张表里，可以快速识别高频组合：Pre-Norm 几乎一统主流稠密模型；RMSNorm 与 SwiGLU 在 LLaMA 之后十分常见；RoPE 成为位置处理的强默认值；偏置项和 dropout 则常被移除。然而，表格中的例外同样重要：它们说明设计空间仍未收敛为唯一答案。

¹视频画面时间区间：00:02:15–00:02:30。

Let's look at the data (on dense architectures)

Learn from the many other models (and papers) out there

We will talk through many major architecture and hyperparameter variants.

- What do all these models have in common?
- What parts vary?
- What can we learn from this?

图 2: 主流自回归语言模型的架构调查表: 共识清晰, 但每一列都仍存在成功例外。²

调查表不是因果实验

“所有强模型都用了某组件”只能说明它是可行且流行的工程选择。要判断组件是否必要, 需要在相同数据、规模、优化器与训练预算下做消融; 否则收益可能来自同时变化的训练配方。

2.2 一个实用的基线策略

新项目宜先复现一套充分验证的现代基线, 建立可重复的损失、吞吐与稳定性曲线。之后每次只改变一个关键因素, 并至少记录: 验证损失、每 token 计算量、tokens/s、峰值显存、通信量、训练尖峰和恢复行为。这样, 架构讨论才从“模型品牌比较”转化为可复核的设计证据。

2.3 本章小结

现代基线已形成, 但尚不存在唯一架构。最稳妥的工作方式是: 先站在成熟默认值上, 再用受控实验解释每一次偏离; 既观察平均性能, 也观察尾部失败和系统代价。

3 残差通路与归一化: 让深网络可训练

3.1 Pre-Norm 与 Post-Norm 的根本差别

残差流可以理解为贯穿全部层的信息高速公路。Post-Norm 在残差相加之后做归一化; Pre-Norm 则先归一化输入, 再把子层产生的增量写回未经归一化的残差流。对第 ℓ 层, 二者可概括为:

²视频画面时间区间: 00:04:30–00:04:45。

$$\text{Post-Norm: } x_{\ell+1} = \text{Norm}(x_{\ell} + F_{\ell}(x_{\ell})),$$

$$\text{Pre-Norm: } x_{\ell+1} = x_{\ell} + F_{\ell}(\text{Norm}(x_{\ell})).$$

- x_{ℓ} : 第 ℓ 层进入残差通路的表示;
- F_{ℓ} : 注意力或前馈子层;
- Norm: LayerNorm、RMSNorm 或其变体。

Pre-Norm 的关键不是“多了一个归一化”，而是保留了从浅层到深层的恒等路径。梯度可以沿残差通路更直接地传播，因此深层网络通常更容易优化。代价是各层对残差流持续累加更新，最终常需额外的输出归一化。

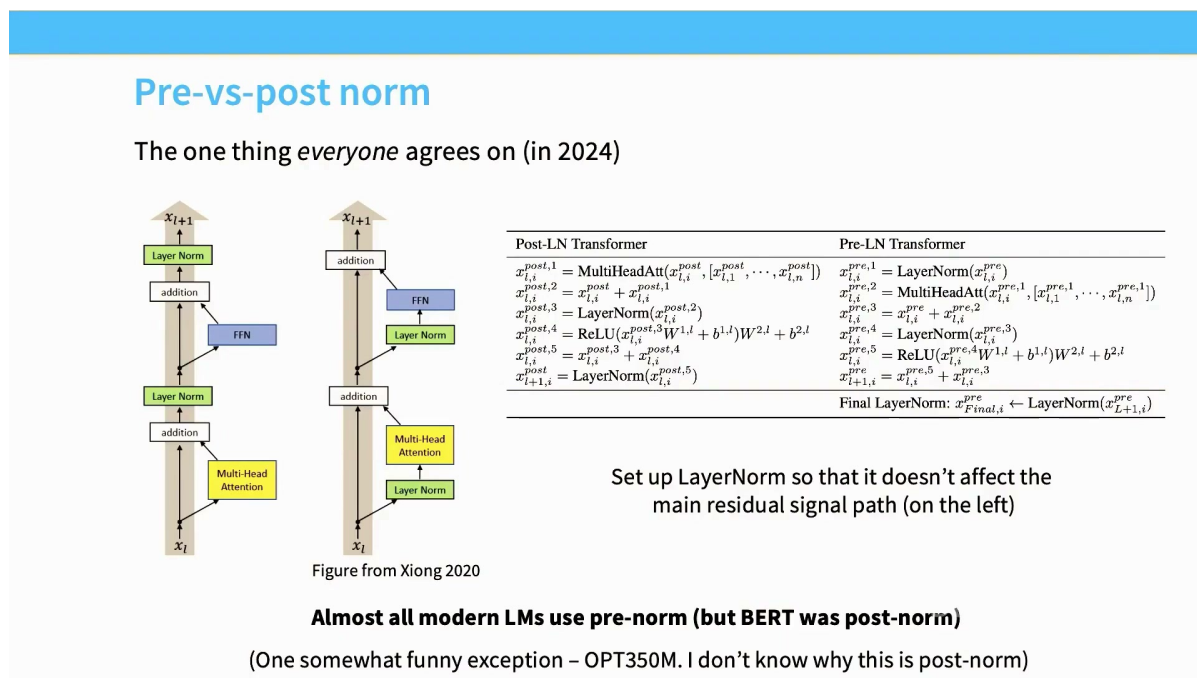


图 3: Post-Norm 与 Pre-Norm: 归一化位置决定了残差高速公路是否保持“干净”。³

判断要点

Pre-Norm 的主要价值是优化稳定性，而非直接增加表达能力。近年来也出现“无残差 Post-Norm”“双重归一化”等成功方案，但这些应视为重新设计信号与梯度尺度的完整系统，而不是随意挪动一层 Norm。

3.2 LayerNorm 与 RMSNorm

LayerNorm 同时去除均值并按标准差缩放；RMSNorm 不做均值中心化，只按均方根缩放。忽略可学习偏置时，它们可写为：

³ 视频画面时间区间：00:08:00–00:08:15。

$$\begin{aligned}\text{LN}(x)_i &= \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \\ \text{RMSNorm}(x)_i &= \gamma_i \frac{x_i}{\sqrt{\frac{1}{d} \sum_{j=1}^d x_j^2 + \varepsilon}}.\end{aligned}$$

- x_i : 一个 token 隐状态的第 i 个通道; d : 隐藏维度;
- μ, σ^2 : 同一 token 跨通道的均值与方差;
- γ_i : 可学习缩放; ε : 防止除零的稳定常数。

RMSNorm 省去了均值计算与减法，结构更简单，也更便于高效内核融合。它不是“近似错误的 LayerNorm”，而是只保留尺度不变性、放弃平移不变性的另一种归一化假设。大量现代语言模型表明，这个假设通常足够。

LayerNorm vs RMSNorm

Original transformer: **LayerNorm** – normalizes the mean and variance across d_{model}

$$y = \frac{x - \mathbf{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

Many modern LMs: **RMSNorm** – does not subtract mean or add a bias term

$$y = \frac{x}{\sqrt{\|x\|_2^2 + \epsilon}} * \gamma$$

Notable models:

GPT3/2/1, OPT, GPT-J, BLOOM

Notable models:

LLaMA-family, PaLM, Chinchilla, T5

图 4: LayerNorm 与 RMSNorm 的运算差异, 以及主流模型中的采用情况。⁴

3.3 为什么 FLOPs 不能代表运行时间

归一化操作的 FLOPs 占比很小，却可能消耗显著运行时间。原因是它通常受显存带宽、kernel launch、同步和中间张量读写支配。讲者给出的例子中，统计归一化约占 0.17% 的 FLOPs，却可能在某些负载中占到约四分之一的运行时间。

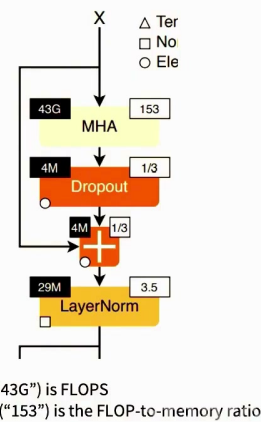
⁴视频画面时间区间：00:14:00—00:14:15。

Why RMSNorm (2)

Important lesson: FLOPS are not runtime! (we will discuss this in far more detail later)

Operator class	% flop	% Runtime
△ Tensor contraction	99.80	61.0
□ Stat. normalization	0.17	25.5
○ Element-wise	0.03	13.5

RMSNorm can still matter due to the importance of *data movement*



[Ivanov et al 2023]

图 5: FLOPs 与墙钟时间并不等价: 低算术强度算子可能被数据搬运主导。⁵

系统侧检查清单

比较归一化方案时, 应在目标硬件和真实序列长度上测量融合后的端到端吞吐; 同时检查是否多次读写残差流、是否能与相邻线性层融合、是否引入额外同步。只看理论 FLOPs 会系统性低估逐元素算子的代价。

3.4 本章小结

Pre-Norm 通过保留恒等残差路径改善深层优化; RMSNorm 用更少操作实现有效的尺度控制。两者的流行同时体现了数值与系统动机, 而真实效率必须用端到端运行时间验证。

4 前馈网络、门控激活与并行块

4.1 从 ReLU / GELU 到 GLU 家族

Transformer 的前馈网络对每个 token 独立执行通道混合。经典形式先升维、施加非线性、再降维。GLU 家族把升维结果拆成“内容分支”和“门分支”, 通过逐元素乘法让一个分支调节另一个分支:

$$\text{SwiGLU}(x) = [\text{SiLU}(xW_g) \odot (xW_u)] W_d.$$

- W_g 、 W_u : 门分支与内容分支的上投影矩阵;
- W_d : 将中间表示投回模型维度的下投影矩阵;

⁵视频画面时间区间: 00:16:00–00:16:15。

- $\odot$: 逐元素乘法; $\text{SiLU}(z) = z \sigma(z)$ 。

GeGLU 用 GELU 作为门激活, SwiGLU 用 SiLU / Swish。Google 系模型常见 GeGLU, LLaMA 及其后继常见 SwiGLU。门控通常能带来稳定的小幅收益, 但 GPT-3 等非 GLU 模型的成功提醒我们: 它是高质量默认值, 不是语言建模成立的必要条件。

Gated variants of standard FF layers	
GeGLU $\text{FFN}_{\text{GeGLU}}(x, W, V, W_2) = (\text{GELU}(xW) \otimes xV)W_2$	Notable models: T5 v1.1, mT5, LaMDA, Phi3, Gemma 2, Gemma 3, Gemma 4
SwiGLU (swish is $x * \text{sigmoid}(x)$) $\text{FFN}_{\text{SwiGLU}}(x, W, V, W_2) = (\text{Swish}_1(xW) \otimes xV)W_2$	Notable models: LLaMa 1/2/3, PaLM, Mistral, OLMo, <i>most models post 2023</i>
Note: Gated models use smaller dimensions for the d_{ff} by 2/3	

图 6: GeGLU 与 SwiGLU: 门控分支使用不同的平滑激活函数。⁶

为什么 GLU 的中间维度常小于 $4d$

门控 FFN 有两套上投影, 若仍令每套中间维度等于 $4d$, 参数与计算都会显著增加。实践中常把门控中间维度设为约 $\frac{8}{3}d$, 使三块大矩阵的总参数量接近传统 $4d$ FFN, 从而进行近似等预算比较。

4.2 串行块与并行块

标准 Transformer 块先做注意力, 再把结果交给 FFN; 并行块则让注意力和 FFN 读取同一归一化输入, 并把两者输出一次性加回残差流:

$$\begin{aligned}
 x' &= x + \text{Attn}(\text{Norm}(x)), \\
 x_{\text{serial}}^+ &= x' + \text{FFN}(\text{Norm}(x')), \\
 x_{\text{parallel}}^+ &= x + \text{Attn}(\text{Norm}(x)) + \text{FFN}(\text{Norm}(x)).
 \end{aligned}$$

- x' : 串行结构中注意力写回后的中间残差;
- x_{serial}^+ : 串行块输出, FFN 可以读取本层注意力结果;
- x_{parallel}^+ : 并行块输出, 两条分支之间没有本层内依赖。

⁶视频画面时间区间: 00:23:45–00:24:00。

并行块可以共享一次归一化，并把多个矩阵乘法调度得更紧凑，理论上更利于并行与融合；但它改变了层内信息依赖。大多数近期模型仍采用串行块，说明系统收益需要与表达能力、现有内核和训练经验一起评估。

Parallel layers

A few models (GPTJ, PaLM, GPT-NeoX) do parallel layers. Originally in GPT-J

Parallel Layers – We use a “parallel” formulation in each Transformer block (Wang & Komatsuzaki, 2021), rather than the standard “serialized” formulation. Specifically, the standard formulation can be written as:

$$y = x + \text{MLP}(\text{LayerNorm}(x + \text{Attention}(\text{LayerNorm}(x))))$$

Whereas the parallel formulation can be written as:

$$y = x + \text{MLP}(\text{LayerNorm}(x)) + \text{Attention}(\text{LayerNorm}(x))$$

The parallel formulation results in roughly 15% faster training speed at large scales, since the MLP and Attention input matrix multiplications can be fused. Ablation experiments showed a small quality degradation at 8B scale but no quality degradation at 62B scale, so we extrapolated that the effect of parallel layers should be quality neutral at the 540B scale.

If implemented right, LayerNorm can be shared, and matrix multiplies can be fused

Recent Models: Cohere Command A, Falcon 2 11B, Command R+

图 7: 串行与并行 Transformer 块：并行形式减少依赖链，但改变了层内计算图。⁷

4.3 本章小结

SwiGLU 是参数利用率较高的现代默认选择，但要按等预算调整中间维度。并行块展示了“架构即调度”的一面：减少归一化和依赖可能提升系统效率，却并不自动保证质量收益。

5 位置表示与 RoPE：把顺序写入注意力

自注意力本身对输入排列不敏感；如果不给位置信息，模型无法区分“猫追狗”和“狗追猫”的顺序。位置处理大致经历了四类方案：固定正弦编码、可学习绝对位置向量、相对位置偏置，以及把位置变换直接施加到查询和键上的 RoPE。

⁷视频画面时间区间：00:28:15–00:28:30。

Many variations in position embeddings

Sine embeddings: add sines and cosines that enable localization

$$\text{Embed}(x, i) = v_x + PE_{pos}$$

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Notable models:

Original transformer

Absolute embeddings: add a position vector to the embedding

$$\text{Embed}(x, i) = v_x + u_i$$

Notable models:

GPT1/2/3, OPT

Relative embeddings: add a vector to the *attention computation*

$$e_{ij} = \frac{x_i W^Q (x_j W^K + a_{ij}^K)^T}{\sqrt{d_z}}$$

Notable models:

T5, Gopher, Chinchilla

Rope embeddings (next slides..)

Notable models:

GPTJ, PaLM, LLaMA
Most 2024+ models

图 8: 位置表示家族: 从绝对向量、相对偏置到作用在查询与键上的旋转变换。⁸

5.1 RoPE 的核心目标: 内积只感知相对距离

注意力分数来自查询与键的内积。理想的位置变换 f 应让两个位置变换后的内积只依赖内容和相对位移, 而不是绝对坐标:

$$\langle f(q, i), f(k, j) \rangle = g(q, k, j - i).$$

- q, k : 某个注意力头中的查询和键向量;
- i, j : 对应 token 的绝对位置;
- g : 只通过 $j - i$ 感知相对距离的打分函数。

RoPE 将通道两两配对, 在每个二维子空间内按位置旋转。直观上, 每个 token 的向量像钟表指针一样随位置转动; 两个指针的夹角天然等于两次旋转角之差, 因此点积能编码相对位置。

⁸视频画面时间区间: 00:31:45–00:32:00。

RoPE: rotary position embeddings

How can we solve this problem?

- We want our embeddings to be invariant to absolute position
- We know that inner products are invariant to arbitrary rotation.

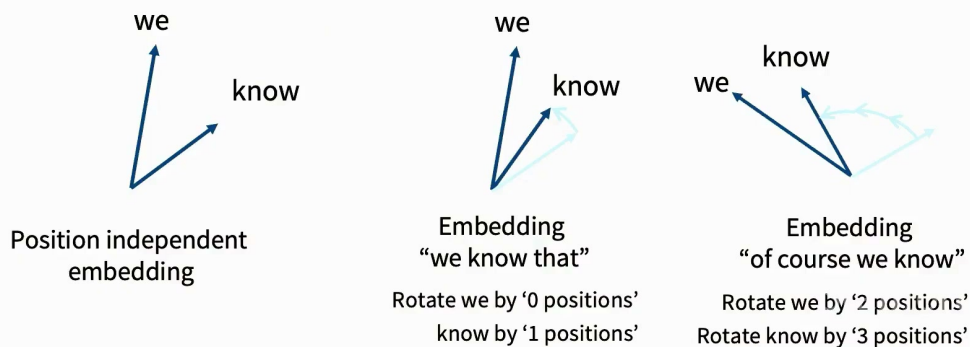


图 9: RoPE 的二维直觉: 对内容向量按位置旋转, 内积由相对旋转角决定。⁹

对第 m 个二维通道对, 可定义旋转矩阵:

$$R_m(i) = \begin{bmatrix} \cos(i\theta_m) & -\sin(i\theta_m) \\ \sin(i\theta_m) & \cos(i\theta_m) \end{bmatrix}, \quad (R_m(i)q)^\top R_m(j)k = q^\top R_m(j-i)k.$$

- θ_m : 第 m 个通道对的旋转频率, 通常按几何级数分布;
- $R_m(i)$: 位置 i 对该二维子空间施加的正交旋转;
- 等式使用 $R_m(i)^\top R_m(j) = R_m(j-i)$, 因此绝对位置相消。

⁹视频画面时间区间: 00:34:45-00:35:00。

The actual RoPE math

Multiply with sines and cosines

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}_{\Theta, m}^d \mathbf{W}_{\{q,k\}} \mathbf{x}_m \quad (14)$$

$$\mathbf{R}_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix} \quad (15)$$

Difference with sine embeddings – not additive, no cross terms

图 10: RoPE 的分块旋转矩阵：不同通道对使用不同频率，覆盖多尺度位置变化。¹⁰

5.2 实现与外推时的常见陷阱

RoPE 应施加在每次注意力计算的 Q 、 K 上，而不是把旋转永久写进 token embedding。缓存解码时，缓存中的键必须已经带有其原始位置旋转，新查询则使用当前时间步的位置。频率基数、缩放方法与训练长度共同决定长度外推行为；仅把推理上下文窗口调大，不能保证模型在新距离上仍能可靠利用信息。

RoPE 不是免费的无限上下文

相对位置结构缓解了绝对位置表的长度限制，但模型仍只在有限距离分布上训练。长上下文能力还取决于训练数据、频率缩放、注意力模式、KV 缓存预算和检索行为；必须用真实长序列任务验证。

5.3 本章小结

RoPE 把绝对位置旋转转化为注意力内积中的相对位移，是现代模型的强默认值。其优雅代数结构不等于无限外推；实现中需正确处理 Q/K 旋转、缓存位置和频率缩放。

6 尺寸超参数：宽度、深度、头数与词表

架构表中最醒目的往往是组件名称，但训练成本与容量分配更直接地由尺寸超参数决定。讲者强调，许多比例在成功模型中聚集到窄区间；这些比例是很好的起点，却不是物理定律。

¹⁰ 视频画面时间区间：00:37:45–00:38:00。

6.1 前馈层维度

非门控 FFN 的经典经验值是 $d_{\text{ff}} \approx 4d_{\text{model}}$ 。对含两条上投影的 GLU，为保持近似等参数与等计算，常用：

$$d_{\text{ff, GLU}} \approx \frac{8}{3}d_{\text{model}}.$$

- d_{model} ：残差流 / 隐藏状态宽度；
- d_{ff} ：FFN 的中间通道数；
- 系数 8/3：补偿 GLU 的两条上投影，使总矩阵参数接近传统 $4d$ FFN。

工程实现还会把 d_{ff} 向上取整到适合 Tensor Core、张量并行或量化分组的倍数，因此论文中的比例常有小幅偏离。

Surprising (?) consensus hyperparameter 1

Feedforward – model dimension ratio.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

There are two dimensions that are relevant – the feedforward dim (d_{ff}) and model dim (d_{model}). What should their relationship be?

$$\mathbf{d_{ff} = 4 d_{model}}$$

This is *almost always* true. There's just a few exceptions.

图 11：传统 FFN 的经验比例：中间维度通常约为模型维度的四倍。¹¹

6.2 注意力头：总维度比头数更关键

多头注意力通常满足 $n_{\text{heads}}d_{\text{head}} \approx d_{\text{model}}$ 。增加头数并不会凭空增加总容量，而是把查询空间切成更多子空间；固定模型宽度时，头数与每头维度互相制约。每头维度也受高效矩阵形状和 RoPE 配对约束，常见值集中在 64、128 等硬件友好尺寸。

¹¹ 视频画面时间区间：00:45:00–00:45:15。

How many heads, whats the model dim?

Some examples of this hyperparameter

	Num heads	Head dim	Model dim	Ratio
GPT3	96	128	12288	1
T5	128	128	1024	16
T5 v1.1	64	64	4096	1
LaMDA	128	128	8192	2
PaLM	48	258	18432	1.48
LLaMA2	64	128	8192	1
Qwen 3.5 (27B)	24	256	5120	1.2

Most models have ratios around 1 – notable exceptions by some google models.

图 12: 不同模型中的头数、每头维度与模型宽度: 多数模型保持总注意力维度接近隐藏维度。¹²

6.3 宽深比与扩展路径

模型宽度除以层数可视为一种“宽深比”。调查显示, 大量模型落在约 100–200 的量级, 但同参数量下的最佳宽深分配还受训练并行、流水线气泡、单 token 延迟和内存布局影响。更深模型增加顺序依赖和 kernel 次数; 更宽模型则放大单层矩阵并可能提高通信压力。

Aspect ratios

Should my model be deep or wide? *How deep and how wide?*

Most models are surprisingly consistent on this one too!

	Model	d_{model}/n_{layer}
Sweet spot?	BLOOM	205
	T5 v1.1	171
	PaLM (540B)	156
	GPT3/OPT/Mistral/Qwen /OLMo 3	128
	LLaMA / LLaMA2	102
	Gemma 3	87
	Gemma 4	61
	T5 (11B)	33

图 13: 模型宽深比调查: 经验区间集中, 但系统约束会推动不同的扩展路径。¹³

¹²视频画面时间区间: 00:50:45–00:51:00。

扩展模型时要同时做的三张表

一张表记录参数与理论 FLOPs；一张表记录训练吞吐、并行效率和显存；第三张表记录推理的首 token 延迟、逐 token 延迟与 KV 缓存。只沿“参数等量线”扩展，可能得到训练或部署上不可用的形状。

6.4 词表大小是一项系统设计

早期英文单语模型常使用 3 万到 5 万左右词表；现代多语与生产模型常扩大到 10 万至 25 万。大词表能减少多语文本的 token 数和序列长度，却会增大 embedding、输出投影与 softmax 的参数和计算，并改变稀有 token 的训练频率。

What are typical vocabulary sizes?	
Monolingual models – 30-50k vocab	
Model	Token count
Original transformer	37000
GPT	40257
GPT2/3	50257
T5/T5v1.1	32128
LLaMA	32000
Multilingual / production systems 100-250k	
Model	Token count
mT5	250000
PaLM	256000
GPT4	100276
Gemma 4	262144
DeepSeek	100000
Qwen 15B	152064
Yi	64000
Monolingual vocabs don't need to be huge, but multilingual ones do	

图 14：主流模型词表大小：从英文单语的数万，扩展到多语 / 代码模型的十万量级。¹⁴

词表的完整成本函数

评价词表时，应同时考虑每字符 token 数、序列长度、长尾覆盖、embedding / 输出层参数、softmax 成本、跨语言公平性以及工具 / 代码字符集的兼容性。词表越大并不自动越好。

6.5 本章小结

尺寸比例提供了可靠起点：传统 FFN 约 $4d$ 、门控 FFN 约 $8d/3$ 、总头维度接近模型宽度。最终取值必须结合硬件整除、并行策略、推理延迟和语料分布共同确定。

¹³视频画面时间区间：00:51:30–00:51:45。

¹⁴视频画面时间区间：00:55:30–00:55:45。

7 正则化与优化动力学：旧概念的新解释

7.1 Dropout 消退，权重衰减保留

许多早期 Transformer 同时使用 dropout 和权重衰减；在数据规模巨大、训练 token 远多于参数的现代语言模型中，dropout 常被完全移除。权重衰减却仍十分普遍，这个反差说明不能只用“小数据过拟合”解释正则化。

Dropout and weight decay in practice			
Model	Dropout*	Weight decay	
Original transformer	0.1	0	
GPT2	0.1	0.1	
T5	0.1	0	
GPT3	0.1	0.1	Many older models used dropout during pretraining
T5 v1.1	0	0	
PaLM	0	(variable)	Newer models (except Qwen) rely only on weight decay
OPT	0.1	0.1	
LLaMA	0	0.1	
Qwen 14B	0.1	0.1	

* Most of the times papers just don't discuss dropout. On open models, this closely matches not doing dropout. This may not be true of closed models.

图 15: 主流模型中的正则化实践：dropout 常为零，权重衰减仍是高频选择。¹⁵

7.2 权重衰减不只是“防过拟合”

以 AdamW 为例，解耦权重衰减可写成：

$$\theta_{t+1} = \theta_t - \eta_t \hat{u}_t - \eta_t \lambda \theta_t.$$

- θ_t ：第 t 步参数； $\hat{u}_t$ ：Adam 预条件后的更新方向；
- η_t ：学习率，因此衰减强度随学习率日程变化；
- λ ：权重衰减系数。

这个更新不仅控制参数范数，也会改变有效步长、尺度平衡和优化轨迹。学习率与权重衰减相乘意味着两者高度耦合：改变 warmup、峰值学习率或衰减日程后，原来的 λ 不一定仍合适。

¹⁵ 视频画面时间区间：01:01:00–01:01:15。

Why weight decay LLMs?

[Andriushchenko et al 2023] has interesting observations about LLM weight decay

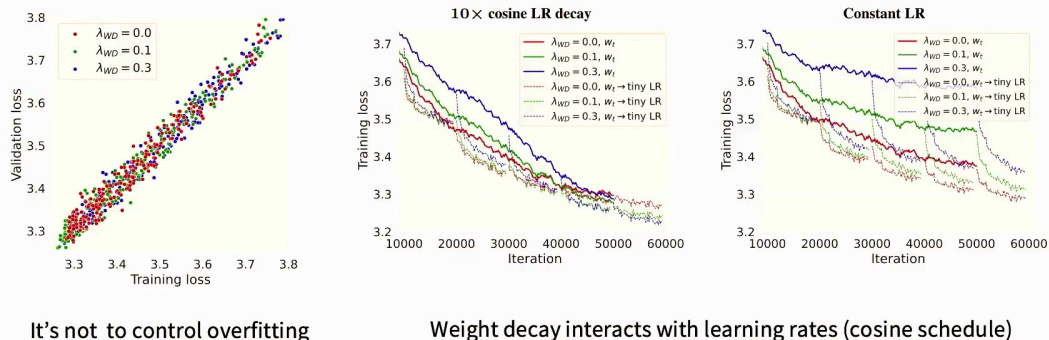


图 16: 权重衰减与学习率 / 训练动力学的耦合: 其作用不能简化为验证集正则化。¹⁶

迁移训练配方的常见错误

从另一规模复制权重衰减时, 若同时改变 batch size、学习率、训练长度或参数化方式, 就已经改变了每个参数累计受到的衰减。应比较整段训练中的积分效应, 而不是孤立比较一个 λ 数字。

7.3 本章小结

现代大规模训练减弱了 dropout 的必要性, 却没有消除权重衰减。后者更像优化动力学的一部分: 它与学习率、参数尺度和训练时长共同决定轨迹, 必须成组调节。

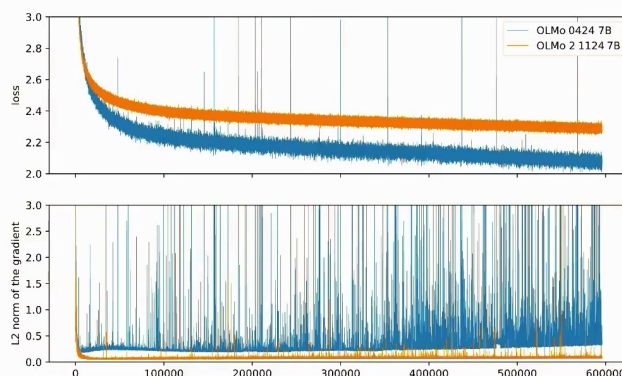
8 训练稳定性: 控制 Softmax 的危险尺度

模型规模越大、训练周期越长, 一次不可恢复的数值爆炸就越昂贵。现代架构研究因此不只追求平均验证损失, 还要降低训练尖峰、NaN 和发散的尾部风险。讲者指出, 许多稳定性干预都指向同一处: Softmax 中的指数与归一化除法。

¹⁶ 视频画面时间区间: 01:01:30–01:01:45。

Stability tricks

Recently, lots of attention on *stable training*



Don't train models that look like the blue curve!

图 17: 训练稳定性曲线: 短暂损失尖峰可能自行恢复, 也可能摧毁一次高成本训练。¹⁷

8.1 语言模型中有两处关键 Softmax

第一处是输出词表 Softmax, 用于把 logits 转为下一个 token 的概率; 第二处是注意力 Softmax, 用于把 $QK^\top$ 转为加权系数。数值实现通常会先减去最大 logit 避免上溢, 但若 logit 尺度持续增大, 仍会造成极端饱和、低精度误差、梯度异常和训练脆弱性。

稳定性设计的共同模式

不要等到最终损失变成 NaN 才诊断。应持续监控输出 logits、注意力 logits、隐藏状态 RMS、梯度范数、更新 / 参数范数比以及每层异常值。稳定性组件的价值在于提前限制危险内部量。

8.2 z-loss: 约束输出归一化常数

设输出 logits 为 $z_1, \dots, z_V$, Softmax 的对数配分函数为 $\log Z = \log \sum_j e^{z_j}$ 。z-loss 在交叉熵外增加一个小惩罚:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \alpha \left(\log \sum_{j=1}^V e^{z_j} \right)^2.$$

- V : 词表大小; z_j : 第 j 个词的输出 logit;
- $\mathcal{L}_{\text{CE}}$: 标准交叉熵损失;
- α : 通常很小的 z-loss 权重, 用于抑制整体 logit 尺度漂移。

¹⁷ 视频画面时间区间: 01:05:15–01:05:30。

Softmax 对所有 logits 同加常数不敏感，交叉熵因此没有动力控制这个共同偏移；z-loss 显式给该“自由方向”加上软约束。它主要是数值保险，而不是要改变正确类别与错误类别的相对排序。

Output softmax stability – the ‘z-loss’

Recall the softmax calculation

$$\begin{aligned}\log(P(x)) &= \log\left(\frac{e^{U_r(x)}}{Z(x)}\right) \\ &= U_r(x) - \log(Z(x)) \\ Z(x) &= \sum_{r'=1}^{|V|} e^{U_{r'}(x)}\end{aligned}$$

$$\begin{aligned}L &= \sum_i [\log(P(x_i)) - \alpha(\log(Z(x_i)) - 0)^2] \\ &= \sum_i [\log(P(x_i)) - \alpha \log^2(Z(x_i))]\end{aligned}$$

[From Devlin 2014]

This is useful for stability! PaLM used this ‘z loss’ trick.

We additionally use an auxiliary loss of $z_loss = 10^{-4} \cdot \log^2 Z$ to encourage the softmax normalizer $\log(Z)$ to be close to 0, which we found increases the stability of training.

Other examples: Baichuan 2 (2023), DCLM (2024), OLMo 2 (2025), OLMo 3 (2025)

图 18: z-loss: 惩罚输出 Softmax 的对数配分函数，限制无意义的整体 logit 漂移。¹⁸

8.3 QK-Norm: 在注意力 Softmax 之前控幅

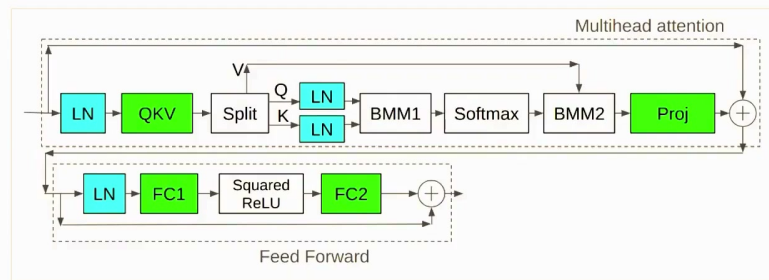
注意力 logits 通常为 $QK^\top/\sqrt{d_h}$ 。即使有 $1/\sqrt{d_h}$ 缩放，训练中的查询和键范数仍可能增长。QK-Norm 在内积之前分别归一化 Q 与 K ：

$$A = \text{softmax}\left(\frac{\text{Norm}(Q) \text{Norm}(K)^\top}{\sqrt{d_h}}\right) V.$$

- Q, K, V : 查询、键和值矩阵；
- d_h : 单个注意力头的维度；
- Norm: 常为沿 head dimension 的 RMSNorm 或 LayerNorm；
- A : 注意力加权后的输出。

¹⁸视频画面时间区间：01:07:15–01:07:30。

Attention softmax stability – the ‘QK norm’



The query and keys are Layer (RMS) normed before going into the softmax operation.

Other examples: DCLM, OLMo2, Gemma 2, Qwen3, OLMo 3, Gemma 4

Originally from vision and multimodal models [Dehgani 2023, Idefcs, Chameleon]



图 19: QK-Norm: 在查询与键形成内积前归一化, 直接限制注意力 logits 的尺度。¹⁹

另一类方法是 soft-capping, 例如把 logit z 映射为 $c \tanh(z/c)$ 。它给 logit 设置硬上界, 但也会压缩大间隔区域, 可能牺牲质量。 z -loss、QK-Norm 和 soft-cap 的共同目的都是控幅, 干预位置和质量代价却不同, 不能互相简单替代。

8.4 稳定性实验该怎样设计

平均损失不足以评价稳定性。至少应在多个随机种子上报告: 尖峰次数与幅度、是否自动恢复、最大 logit / 激活 / 梯度、恢复后最终质量, 以及干预带来的吞吐成本。对于超大训练, 可先在更小规模上人为提高学习率或降低精度, 构造“压力测试”, 但仍需确认干预在目标规模不过度约束表达能力。

8.5 本章小结

输出 Softmax 与注意力 Softmax 是两处核心危险区。 z -loss 约束输出配分函数, QK-Norm 约束查询 / 键尺度, soft-cap 则直接截住极端 logits。它们是面向昂贵训练的风险控制组件, 应以尾部失败概率而非只看平均损失来评估。

9 推理系统: Prefill、Decode 与 KV 缓存

训练完成后, 架构约束会从“如何高效并行训练”转向“如何低延迟服务”。自回归推理分为两个性质截然不同的阶段: Prefill 一次处理整段提示词; Decode 每步只生成一个新 token, 并依赖此前所有缓存状态。

¹⁹ 视频画面时间区间: 01:10:15–01:10:30。

9.1 Prefill: 矩阵大、复用高、较偏计算受限

对长度为 L 的提示词，注意力可并行生成全部查询、键和值，再进行大矩阵乘法。每次从显存读入权重后可服务许多 token，数据复用高；足够大的 batch 和序列常能接近 GPU 计算峰值。标准注意力的主要计算随 L^2 增长，因此长提示仍可能昂贵。

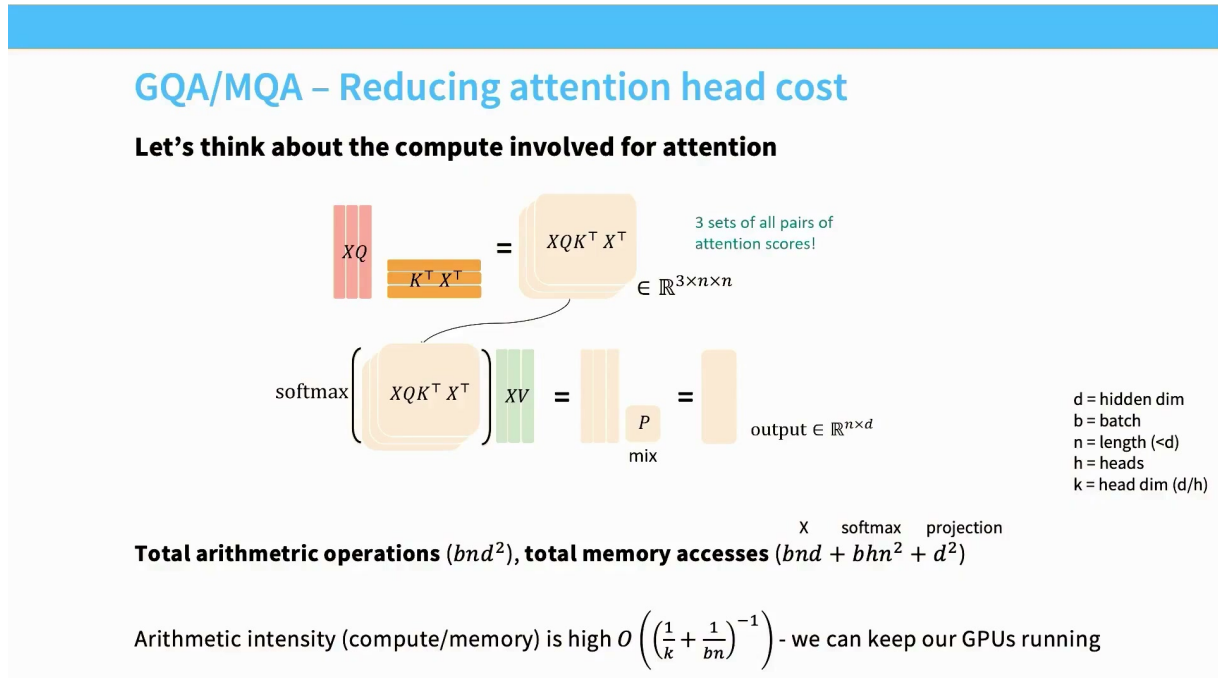


图 20: Prefill 阶段的注意力：大规模并行矩阵运算带来较高算术强度。²⁰

算术强度可粗略理解为单位显存传输产生的浮点运算：

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes moved}}.$$

- 分子：矩阵乘法等完成的浮点运算量；
- 分母：在显存与计算单元之间搬运的字节数；
- 比值高时更可能受计算吞吐限制，比值低时更可能受带宽限制。

9.2 Decode: 顺序依赖强、算术强度低

Decode 的第 t 步只有一个新查询，但必须读取过去 t 个 token 的键和值。KV 缓存避免了重算历史状态，却把瓶颈转成每步读取大量缓存。生成下一 token 后才能开始下一步，因此时间维无法像 Prefill 一样并行。

²⁰ 视频画面时间区间：01:15:15–01:15:30。

GQA/MQA – Reducing attention head cost

What's the incremental arithmetic intensity?

Total arithmetic operations (bnd^2), **total memory accesses** ($bn^2d + nd^2$)^{K, V projection}

Arithmetic intensity is not good $O\left(\left(\frac{n}{d} + \frac{1}{b}\right)^{-1}\right)$ - need large batches + short seq length (n) or big model dimensions (d)

Is there some way around this? The n/d term is difficult to reduce.



图 21：增量解码：每步仅产生一个查询，却要读取全部历史 KV，常表现为显存带宽受限。²¹

标准多头注意力的 KV 缓存元素量近似与下式成正比：

$$N_{KV} \propto 2BLn_{\text{layers}}n_{\text{kv}}d_h.$$

- B ：服务 batch size； L ：已缓存上下文长度；
- n_{layers} ：层数； n_{kv} ：每层 KV 头数；
- d_h ：每个 KV 头维度；系数 2 对应 K 与 V 两份缓存。

缓存字节数还需乘以数据类型字节宽度。这个关系解释了为什么减少 KV 头数、压缩缓存精度和限制局部窗口会直接提升服务容量。

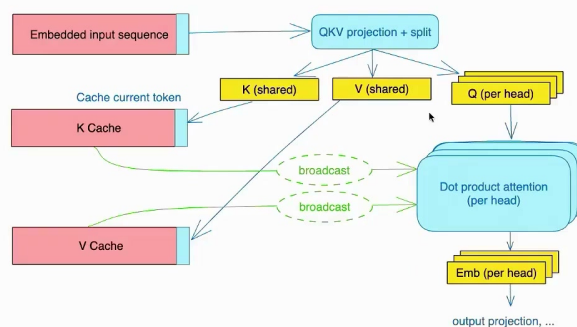
9.3 MQA：所有查询头共享一组 K/V

Multi-Query Attention (MQA) 保留多个查询头，却只使用一组键和值。与标准 MHA 相比，KV 缓存可近似按查询头数倍缩小，Decode 时的数据读取也随之下降。

²¹ 视频画面时间区间：01:18:15–01:18:30。

MQA – just have fewer key dimensions.

Key idea – have multiple queries, but just one dimension for keys and values



We have much fewer items to move in and out of memory (KV Cache)

Total memory access $(bnd + bn^2k + nd^2)$, **Arithmetic intensity** $O\left(\left(\frac{1}{d} + \frac{n}{dh} + \frac{1}{b}\right)^{-1}\right)$



[figure from <https://blog.fireworks.ai/multi-query-attention-is-all-you-need-db072e758055>]

图 22: MQA: 各查询头仍独立, 但共享同一组键和值, 大幅压缩 KV 缓存。²²

MQA 的代价是键 / 值表示的多样性下降, 可能造成质量损失。对于极端受内存带宽限制的在线服务, 这种交换仍可能值得; 但不能只根据缓存缩小倍数判断总体收益。

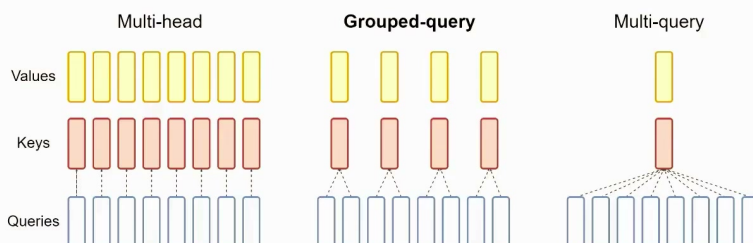
9.4 GQA: 系统效率与表达能力的折中

Grouped-Query Attention (GQA) 把查询头划分为若干组, 每组共享一对 K/V。它在 MHA 与 MQA 之间连续调节 n_{kv} : KV 头越少, 缓存越小; KV 头越多, 表示越丰富。很多现代模型采用 GQA, 因为它常能取得显著系统收益而质量损失很小。

²² 视频画面时间区间: 01:19:45–01:20:00。

Additional extensions – GQA

Don't go all the way to one dimension of KV – have fewer dims



Simple knob to control expressiveness (key-query ratio) and inference efficiency

More recently – MLA (multihead latent attention) from deepseek v2

图 23: MHA、GQA 与 MQA: 通过改变每组查询头共享的 KV 数量调节质量 / 内存折中。²³

部署指标必须分阶段报告

Prefill 应重点报告首 token 延迟、长提示吞吐和峰值注意力内存；Decode 应报告逐 token 延迟、并发容量、KV 缓存字节 / token 和带宽利用率。把两阶段平均成一个 tokens/s，容易掩盖真正瓶颈。

9.5 本章小结

Prefill 有大矩阵和高数据复用，Decode 则有严格顺序依赖并频繁读取 KV 缓存。MQA 和 GQA 不是纯粹的建模花样，而是直接面向 Decode 带宽与服务容量的架构设计。

10 长上下文：稀疏、滑动窗口与混合注意力

全局注意力允许任意 token 访问全部过去信息，但计算和缓存随上下文快速增长。长上下文设计的关键问题是：是否每一层、每一个 token 都需要全局通信？如果局部层能完成大部分短程处理，只需周期性注入全局层，就可能显著降低成本。

10.1 从稀疏模式到滑动窗口

稀疏注意力只计算预先选择的连接。最常见的滑动窗口让位置 i 只访问最近 w 个历史位置，把每层注意力计算从 $O(L^2)$ 降到近似 $O(Lw)$ ，同时把该层 KV 缓存限制在窗口内。

²³ 视频画面时间区间：01:20:45–01:21:00。

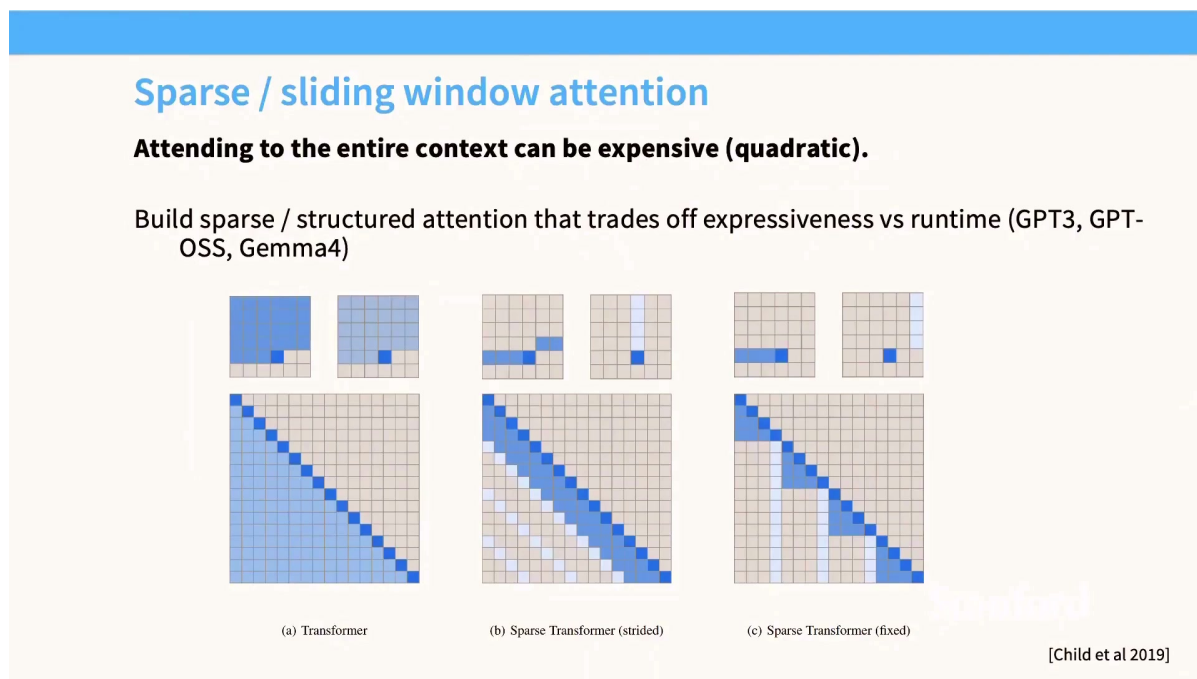


图 24: 全局、稀疏与滑动窗口注意力模式：减少连接数量即可降低长序列成本。²⁴

局部窗口的“层间传播”不等于直接检索

堆叠局部层会扩大理论感受野，但远处信息必须经过多层逐步传递，路径更长且可能失真。需要精确访问远端 token 的任务，通常仍需周期性全局层、检索机制或其他显式长程通道。

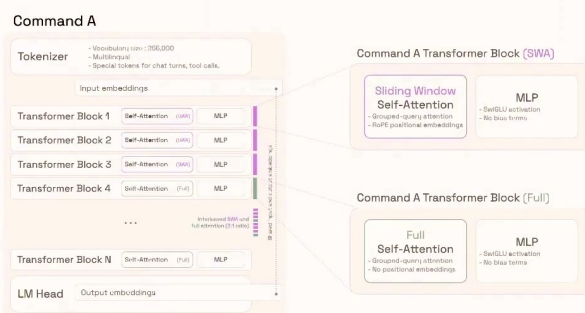
10.2 交错全局层与局部层

一种实用方案是让大多数层使用便宜的局部注意力，间隔若干层插入全局注意力。Command A 等模型展示了这种思路：局部层以 RoPE 和滑动窗口处理邻域模式，周期性全局层重新建立跨序列通信。这样，每个 token 先被局部计算充分加工，再通过全局层汇聚远程信息。

²⁴视频画面时间区间：01:25:15–01:25:30。

Current standard trick – interleave ‘full’ and ‘LR’ attention

From Cohere Command A – Every 4th layer is a full attention



Long-range info via NoPE, short-range info via RoPE + SWA.

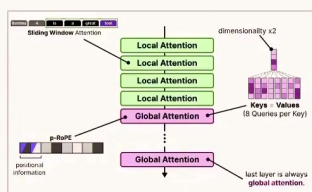
Other models – LLaMA 4, Gemma 3, Gemma 4, OLMo 3 does SWA+Full RoPE.

图 25: 全局 / 局部注意力交错: 用少量全局层维持远程通路, 用多数局部层控制成本。²⁵

10.3 混合架构仍在快速演化

近期模型把“便宜层”扩展到多种形式: 局部滑窗、不同频率的全局注意力, 甚至状态空间层与注意力层混合。它们共享同一原则: 高成本全局通信不必在每层发生。区别在于信息如何跨越局部窗口、位置编码施加在哪里、缓存如何管理, 以及硬件上是否有成熟内核。

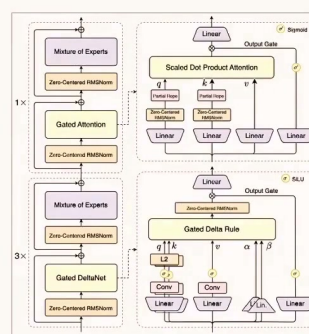
Other recent examples of interleaved attention



Gemma 4

Gradient clipping	1.0
Z-loss weight	10^{-5}
Weight decay on embeddings	No
Sliding window attention	3/4 of layers: 4,096 tokens
RoPE scaling	YaRN on full attn. layers
RoPE θ	$5 \cdot 10^4$
Layer norm applied to	Outputs

Olmo 3



Qwen 3.5 / Qwen 3 Next

<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-gemma-4>

图 26: 近期长上下文混合架构: Gemma、OLMo、Qwen 等采用不同的局部 / 全局或状态空间组合。²⁶

²⁵ 视频画面时间区间: 01:26:30–01:26:45。

长上下文评估不能只看最大长度

“支持 1M token” 是接口上限，不是有效利用能力。应分别测试短程保持、远程检索、多跳组合、位置鲁棒性、不同上下文长度下的质量退化，以及 Prefill / Decode 的真实成本。

10.4 本章小结

滑动窗口把每层成本从全局二次连接降为局部带状连接；交错架构用少量全局层补回远程通信。长上下文仍是设计变化最快的部分，必须联合评估信息可达性、训练分布、内核效率和缓存成本。

11 总结与延伸

11.1 讲者总结：共识、例外与变化最快的边界

课程最后回到架构调查表。今天的现代语言模型存在清晰共识：Pre-Norm 或其他经过稳定性验证的残差方案、RMSNorm、RoPE、门控 FFN，以及为部署优化的 GQA。与此同时，宽深比、词表、正则化和长上下文模式仍存在大量成功例外。

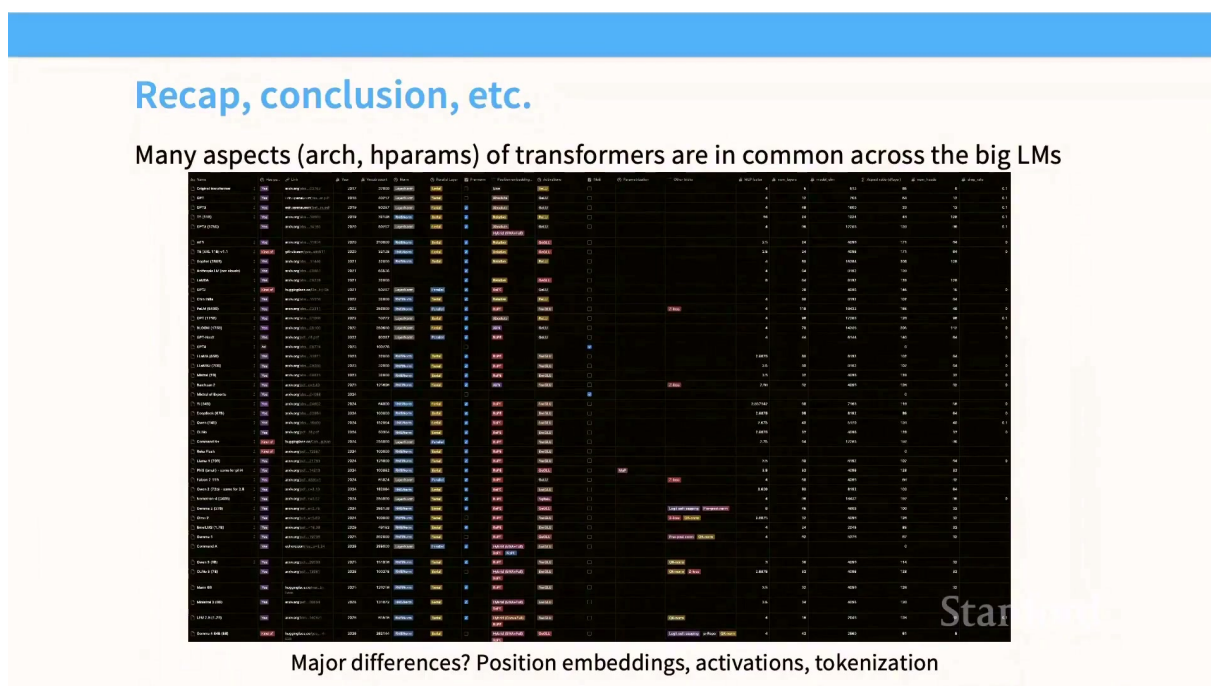


图 27：课程回顾：主流架构已出现稳定默认值，但位置与上下文处理仍快速变化。²⁷

11.2 一张可执行的设计清单

1. **先定义约束。**写清训练预算、目标硬件、序列长度、并发与质量门槛；没有约束就不存在“最佳架构”。

²⁶ 视频画面时间区间：01:28:00–01:28:15。

²⁷ 视频画面时间区间：01:28:45–01:29:03。

2. **建立成熟基线。**从 Pre-Norm / RMSNorm、RoPE、SwiGLU、GQA 等可靠默认值开始，先复现损失和吞吐。
3. **区分建模与系统指标。**同时测验证损失、FLOPs、墙钟时间、显存、通信、首 token 延迟和逐 token 延迟。
4. **把稳定性当一等指标。**监控两处 Softmax 的 logits、隐藏状态与梯度；在多随机种子下报告尖峰与发散率。
5. **逐项偏离并做等预算消融。**组件变化要控制数据、参数量、训练 token 和优化器，避免把品牌相关性误写成果。
6. **单独验证长上下文。**最大输入长度、远程信息利用率和服务成本是三件不同的事。

本讲最重要的综合判断

现代架构优化的核心，不只是增加表达能力，而是把有限的参数、计算、显存带宽与数值风险分配到最有价值的位置。很多“新组件”实际上是在重新安排数据搬运、缓存或危险内部尺度。

11.3 进一步推导：从组件思维转向约束思维

把全课放在一起，可以得到三条更一般的推论。

第一，**架构与系统不可分割。**RMSNorm、并行块、GQA、滑动窗口之所以重要，不只因为数学形式，也因为它们改变内存访问、并行度和缓存体积。理论 FLOPs 相同的两个模型，真实成本可能完全不同。

第二，**稳定性是规模化能力的一部分。**在小实验中偶发一次尖峰可能无关紧要；在持续数月的大训练中，尾部失败概率会直接决定项目经济性。z-loss 与 QK-Norm 代表一种成熟工程观：提前限制危险变量，而不是期待优化器自行修复。

第三，**上下文是下一轮架构分化的中心。**基础块已趋于同质化，模型间更显著的差别转向位置编码、局部 / 全局通信比例、KV 缓存和混合序列模块。评价这些设计时，应从“标称窗口有多长”转向“每单位成本能可靠利用多远的信息”。

11.4 建议练习

1. 在同一小型语言模型上比较 LayerNorm 与 RMSNorm：同时测损失、tokens/s 和归一化 kernel 时间。
2. 实现 MHA、GQA 与 MQA，在固定参数预算下测 KV 字节 / token、Decode 延迟及困惑度变化。
3. 记录训练过程中的输出 log-sum-exp、注意力 logits 最大值和梯度范数，再比较加入 z-loss 或 QK-Norm 后的尖峰分布。
4. 构造需要跨越局部窗口检索的信息，比较全局注意力、纯滑窗和交错全局 / 局部层的正确率与吞吐。

11.5 拓展阅读

- Stanford CS336 Spring 2026 课程主页: <https://cs336.stanford.edu/>
- Lecture 3 官方课件: https://github.com/stanford-cs336/lectures/blob/main/lecture_03.pdf
- RoFormer (RoPE 原始论文): <https://arxiv.org/abs/2104.09864>
- GLU Variants Improve Transformer: <https://arxiv.org/abs/2002.05202>
- GQA: Training Generalized Multi-Query Transformer Models: <https://arxiv.org/abs/2305.13245>
- FlashAttention (从 I/O 复杂度理解注意力): <https://arxiv.org/abs/2205.14135>

11.6 本章小结

可靠的现代基线让我们少走弯路，例外则提醒我们继续测量。模型设计的成熟路径是：明确约束、选择默认值、建立端到端指标、控制变量做消融，并把稳定性和部署成本放进与损失同等重要的验收表。